

Introduction

This document is the user guide for the Microchip implementation of the G3-PLC firmware stack.

The mechanisms and functionality defined in the G3-PLC specification are the basis of the entire firmware stack implementation. Therefore, it is highly recommended to use it as a reference. Basic concepts that are introduced by this G3-PLC specification are assumed to be known within this document.

In addition, the Microchip G3-PLC stack includes the G3 Hybrid Profile implementation, which is based on the Hybrid G3-PLC and RF Profile Annex to the G3-PLC specification.

Table of Contents

Introduction.....	1
1. General Architecture.....	5
1.1. PLC PHY Layer.....	7
1.2. RF PHY Layer.....	8
1.3. PLC Host Controller.....	8
1.4. PLC PAL (PHY Wrapper).....	8
1.5. RF PAL (PHY Wrapper).....	9
1.6. PLC MAC RT (MAC Real Time).....	9
1.7. PLC PAL 360/460 MAC RT (PLC MAC RT Wrapper).....	10
1.8. MAC Layer and MAC Wrapper.....	10
1.9. ADP Layer, ADP Conf and ADP API.....	10
1.10. Routing Module and Routing Wrapper.....	11
1.11. Common Utils Module.....	11
1.12. Crypto Module and Crypto Wrapper.....	11
1.13. HAL (Hardware Abstraction Layer).....	11
1.14. Coordinator Module.....	12
1.15. LBP Module.....	12
2. Understanding the Firmware Package.....	13
2.1. G3-PLC Workspace Structure.....	13
3. Understanding the G3-PLC FW Stack.....	20
3.1. Operation Modes (Operating System Support).....	20
3.2. Initializing and Starting G3-PLC Stack and User Application.....	20
3.3. Configuration Files.....	26
3.4. Persistent Data Storage.....	31
3.5. Firmware Update Process.....	33
4. Operating System Support.....	34
4.1. Overview.....	34
4.2. G3-PLC Process. Requirements, Handling and Usage of the G3-PLC FW Stack.....	34
5. Example Applications.....	35
5.1. ADP Examples.....	36
5.2. PLC PHY Examples.....	39
5.3. RF PHY Examples.....	40
5.4. Clock Configuration Example for Low Requirements Module.....	40
6. Microchip Implementation Particulars	42
6.1. Table Sizes and Buffer Count Configuration.....	42
6.2. Impedance Detection, Transmission States and Equalization.....	44
6.3. ZC Offset Configuration.....	49
6.4. Noise Detection and Compensation.....	50
6.5. Power-Down Detection.....	51
6.6. Voltage and Thermal Monitor for PLC Driver.....	54
7. API of PHY Layer.....	55

7.1. PIB Objects Specification and Access.....	55
8. API of DATA LINK Layer.....	61
8.1. PLC MAC SAP.....	61
8.2. PLC MAC Primitives.....	61
8.3. PLC MAC Real Time.....	62
8.4. RF MAC SAP.....	62
8.5. RF MAC Primitives.....	63
8.6. MAC Wrapper.....	63
8.7. MIB Objects Specification and Access.....	69
8.8. ADP SAP.....	102
8.9. ADP Data Primitives.....	104
8.10. ADP Management Service Primitives.....	105
8.11. ADPIB Objects Specification and Access.....	117
8.12. ADP Status Codes Description.....	129
9. Data Link Processes.....	134
9.1. Network Discovery Request.....	134
9.2. Network Join Request.....	134
9.3. Route/Path Discovery Request.....	135
9.4. Network Leave.....	137
9.5. Data Request.....	138
9.6. ADP Reset.....	139
9.7. GET/SET MIBs.....	139
10. LBP Module.....	141
11. LBP Module.....	142
11.1. LBP Coordinator.....	142
11.2. LBP Device.....	144
11.3. LBP IB List.....	146
12. Coordinator Module.....	148
12.1. Functional Description.....	148
12.2. API Listing.....	148
12.3. Brief of Provided Coordinator Module Example.....	151
13. Serialization with Embedded USI.....	152
13.1. USI Frame Format.....	152
13.2. USI G3 Protocols.....	153
13.3. Embedded USI Configuration.....	153
14. Supported Platforms.....	155
14.1. Supported MCU Families.....	155
14.2. Supported Devices.....	155
14.3. Supported Boards.....	155
15. Toolchain.....	156
15.1. General Prerequisites.....	156
15.2. Building the Applications.....	156

16. PICS.....	157
16.1. Band-plan Support.....	157
16.2. Feature Implementation Statement.....	157
16.3. Other Features.....	157
17. Abbreviations.....	158
18. References.....	159
19. Revision History.....	160
19.1. Revision B - 09/2023.....	160
19.2. Rev A - 11/2021.....	160
Microchip Information.....	161
The Microchip Website.....	161
Product Change Notification Service.....	161
Customer Support.....	161
Microchip Devices Code Protection Feature.....	161
Legal Notice.....	161
Trademarks.....	162
Quality Management System.....	163
Worldwide Sales and Service.....	164

1. General Architecture

The provided G3-PLC Firmware Stack follows a layered approach based on the G3-PLC specification.

The stack is a set of libraries and source code blocks implementing the different parts of G3. Blocks are separated by “wrappers”, which provide a clean separation between modules, a way to change stack table sizes, and even allow replacing a module with a different implementation of it.

The following figures show the stack architecture and its variants depending on the platform used:

Figure 1-1. Block Diagram of Microchip G3-PLC Stack (ATPL250)

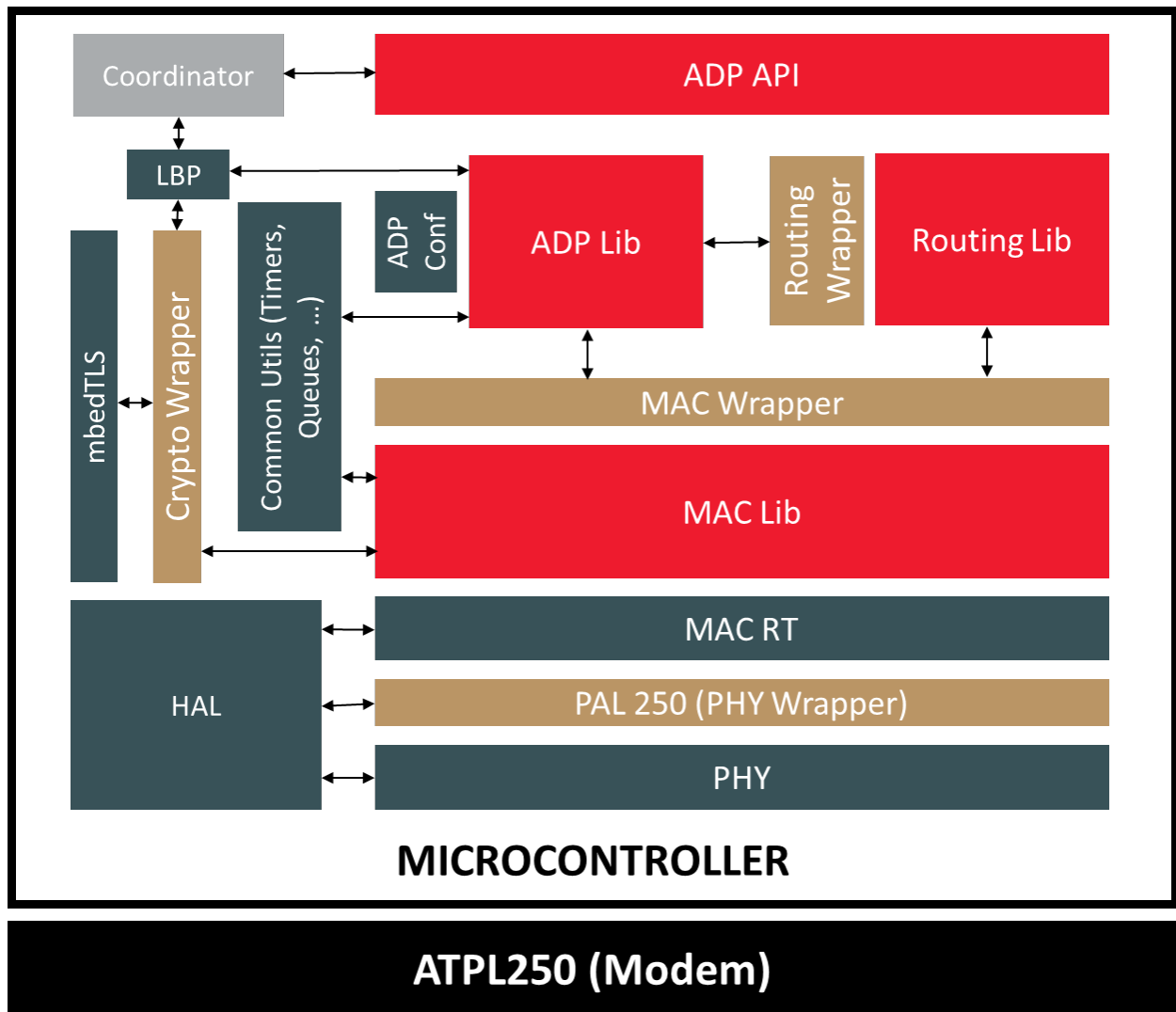


Figure 1-2. Block Diagram of Microchip G3-PLC Stack (PL360/PL460 MAC RT)

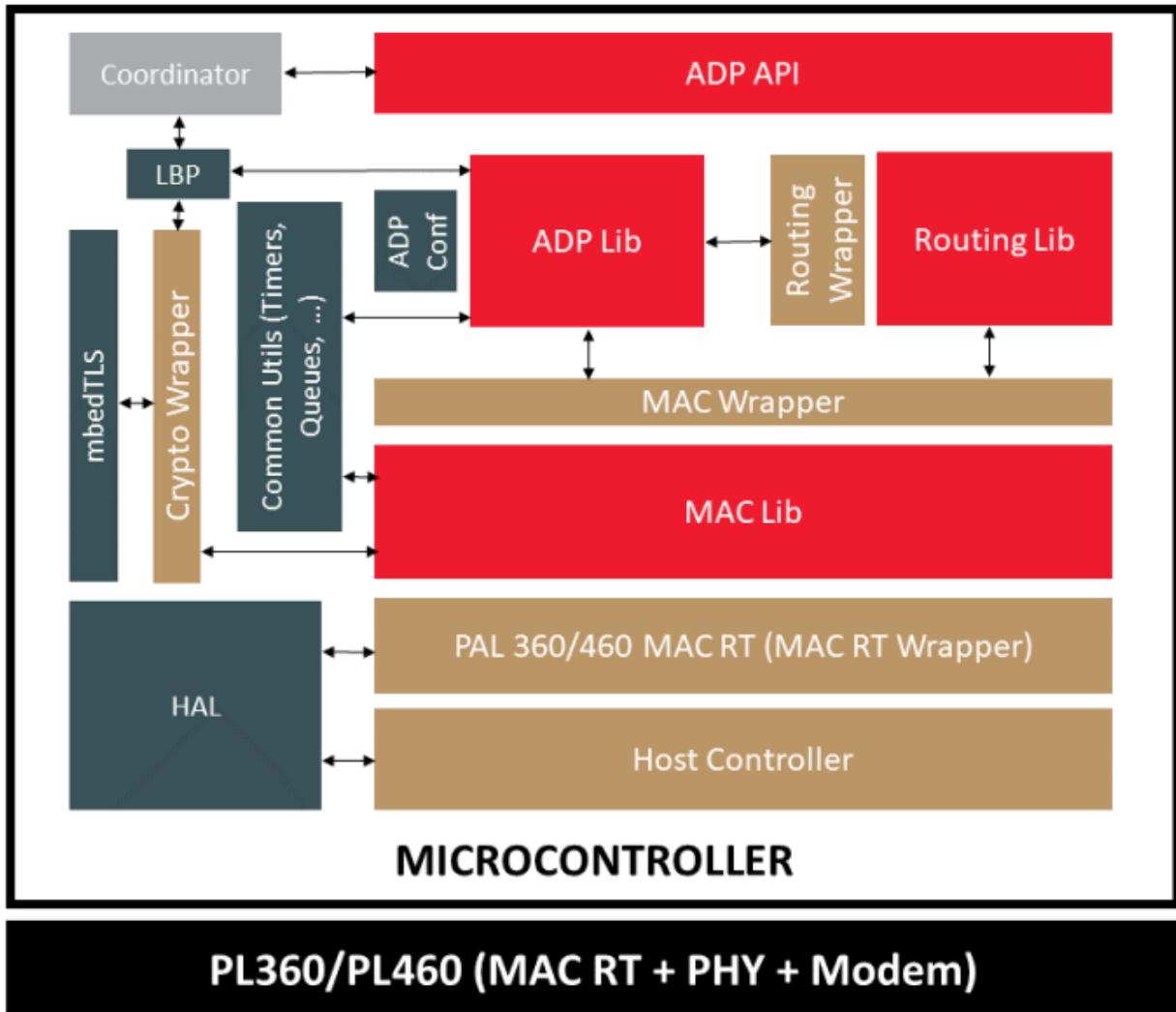
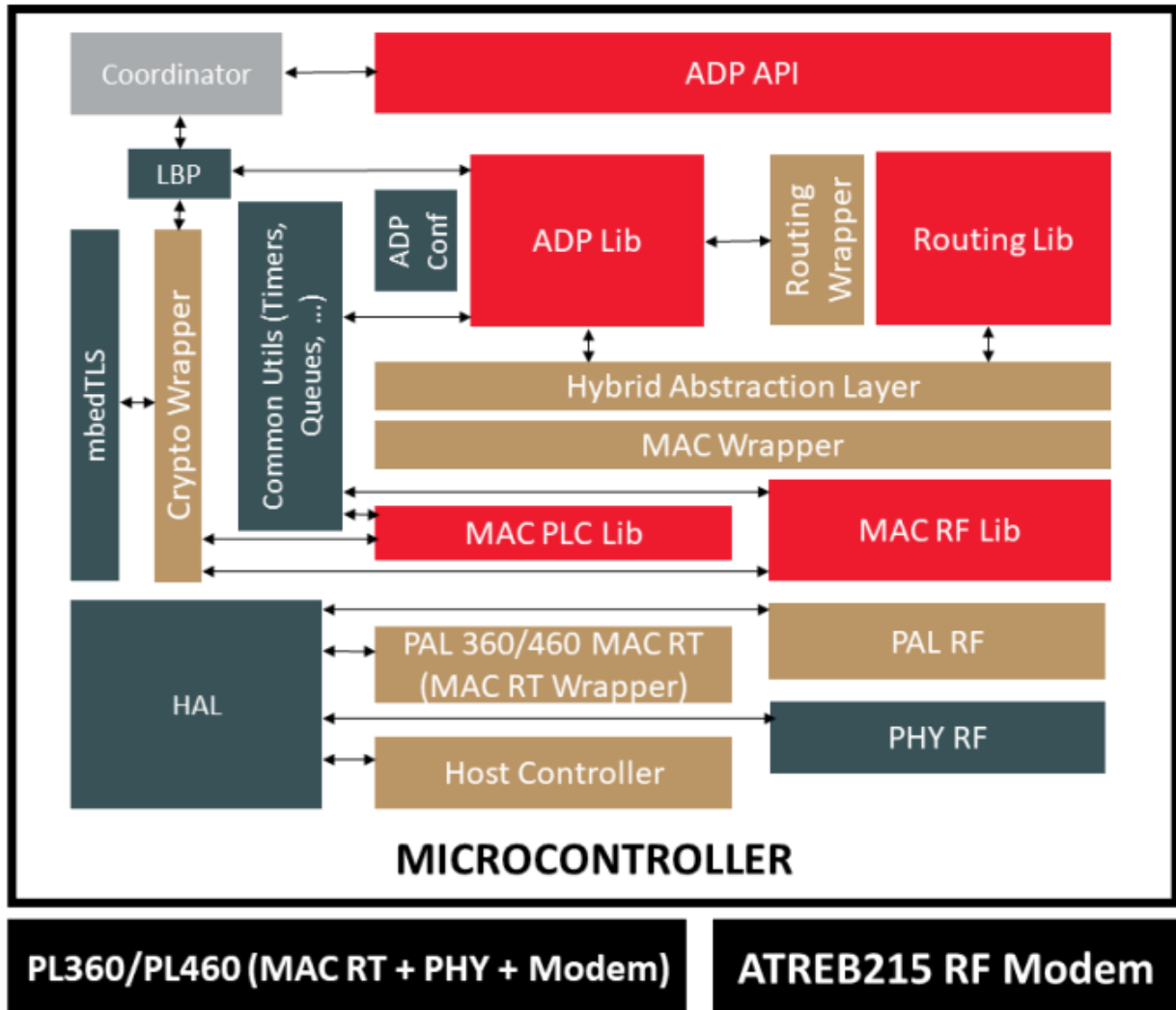


Figure 1-3. Block Diagram of Microchip G3-Hybrid PLC&RF Stack



1.1 PLC PHY Layer

The PLC PHY layer is in charge of frame transmission and reception. This layer is mainly interrupt-driven with events coming from the PLC modem.

In ATPL250 architecture, the PHY layer runs in the host microcontroller. The ATPL250 device generates interrupt events while transmitting and receiving frames. There are two special events that trigger data indication and data confirm primitives to be handled by upper layers: a frame reception and the end of a frame transmission.

In the case of PL360/PL460 architecture, PHY layer firmware is managed by the PL360/PL460 device; thus, data indication and data confirm events are managed by the Host Controller module in case of PHY layer applications, or internally for G3 stack applications, where **1.6. PLC MAC RT (MAC Real Time)** runs inside the PL360/PL460.

Apart from these events, the PHY layer API (either in the host microcontroller or through the Host Controller module) implements entry functions to transmit a frame using the PLC modem, to perform periodic tasks and to access the PHY Information Base (PIB) to read, write or modify parameters.

PL360/PL460 is a Flashless device, so its firmware has to be stored in the host microcontroller memory and downloaded at start-up. In the project examples provided, it is the linker and compiler that are configured to perform the allocation automatically. For further details, refer to the *PL360 Host Controller* document provided by Microchip.

1.2 RF PHY Layer

The RF PHY layer is in charge of frame transmission and reception over the air. This layer is mainly interrupt-driven with events coming from the RF transceiver.

The supported RF transceiver is AT86RF215, the PHY layer runs in the host microcontroller. The AT86RF215 device generates interrupt events while transmitting and receiving frames. There are two special events that trigger data indication and data confirm primitives to be handled by upper layers: a frame reception and the end of a frame transmission.

The PHY layer API implements entry functions to transmit a frame using the RF transceiver, request channel sensing, perform periodic tasks and to access the PHY Information Base (PIB) to read or write parameters.

1.3 PLC Host Controller

This block is only present in the PL360/PL460 platform.

It is in charge of controlling and monitoring all PL360/PL460 activities, as well as loading the binary at start-up and managing any unexpected behavior.

It is a complex module with its own documentation. Refer to the *PL360 Host Controller* document provided by Microchip.

1.4 PLC PAL (PHY Wrapper)

This layer abstracts the interface of the [1.1. PLC PHY Layer](#) developed by Microchip and provides an interface compliant with the G3-PLC PHY layer. For more details, refer to the G3 PLC Spec [www.g3-plc.com/]. In other words, this layer acts as the interface between the currently developed PHY and MAC layers, and is in charge of communicating both layers properly.

It implements primitives to inform the MAC layer of events coming from the PLC, namely PhyDataConfirm, PhyAckConfirm, PhyDataIndication and PhyAckIndication.

Apart from these tasks, PHY API will directly address function calls from MAC to transmit a frame, PhyDataRequest and PhyAckRequest to PHY layer. The same will occur with MAC calls to PLME functions (PhyPlmeSetRequest, PhyPlmeSetConfirm, PhyPlmeGetRequest, PhyPlmeGetConfirm, PhyPlmeSetTrxStateRequest, PhyPlmeSetTrxStateConfirm, PhyPlmeCsRequest, PhyPlmeCsConfirm, PhyPlmeResetRequest), which will be directly addressed to PHY functions related to PIB access.

Please note that the semantics of the PHY primitives included in this firmware stack cover the requirements of the G3-PLC PHY specs and provide more information:

- PhyInitialize has to be called to initialize the PHY layer
- PhyEventHandler has to be called to perform periodic tasks
- PhyGetTime retrieves the value of the internal PHY timer used to control RX and TX times
- PhyGetToneMapResponseData is used by the MAC layer to retrieve information from the PHY layer to fill a Tone Map Response frame
- PhySetToneMask sets the Tone Mask used by the Phy layer
- PhyGetLegacyMode retrieves whether PHY layer is running in legacy mode or not
- PhyGetParam and PhySetParam are used to access PIB objects not present in the standard PhyPlmeGetRequest and PhyPlmeSetRequest primitives

For further details, refer to the *pal.h* header file.

In case of the PL360/PL460 platform, this interface is internal to the PL360/PL460 device, as a link between the host and the PL360/PL460 is managed at the MacRT level. See [1.7. PLC PAL 360/460 MAC RT \(PLC MAC RT Wrapper\)](#) for further information.

1.5 RF PAL (PHY Wrapper)

This layer abstracts the interface of the [1.2. RF PHY Layer](#) developed by Microchip and provides an interface similar to the one specified for the G3-PLC PHY layer. This layer acts as the interface between the currently developed PHY and MAC layers, and is in charge of communicating both layers properly.

It implements primitives to inform the RF MAC layer of events coming from the RF, namely `PalRfDataConfirm` and `PalRfDataIndication`.

Apart from these tasks, RF PAL will directly address function calls from MAC to transmit a frame (`PalRfTxRequest`) or cancel a programmed transmission (`PalRfTxCancel`) to the PHY layer.

Other RF PAL API functions include:

- `PalRfInitialize` initializes the RF PHY layer
- `PalRfEventHandler` calls the RF PHY Event handler, which checks for events to be notified to the upper layers
- `PalRfGetPhyTime` retrieves the value of the internal RF PHY timer used to control the RX and TX times
- `PalRfResetRequest` resets the RF PAL and PHY layers



Important: The `PalRfInitialize` function is where the PHY layer is initialized according to chosen modulation (FSK or OFDM), frequency band, channel number, operation mode and so on. Refer to the files: `ieee_15_4_sun_fsk.h` and `ieee_15_4_sun_ofdm.h`, located under the `/common/components/rf/at86rf215/` folder to check for different configuration modes available.

For further details, refer to the `pal_rf.h` header file.

1.6 PLC MAC RT (MAC Real Time)

This block is the lowest part of the G3 PLC MAC layer and, as its name indicates, contains all the tasks that require a low latency attention.

It is provided as source code for the ATPL250 platform, out of the G3 MAC library.

In the PL360/PL460 platform, it is integrated in the binary. This significantly relaxes the time requirements on the Host MCU because, as stated above, all the low latency requirements are in this layer and, thus, are managed by the PL360/PL460 device.

The functionalities included in this block are:

- CRC check
- ACK generation if requested
- CSMA
- ARQ
- Segmentation and reassembly
- Network scan. Generation and processing of BCN and BREQ frames
- Transmission robustness management. Generation and processing of TMR frames

1.7 PLC PAL 360/460 MAC RT (PLC MAC RT Wrapper)

This module is the equivalent to the [1.4. PLC PAL \(PHY Wrapper\)](#) layer when the [1.6. PLC MAC RT \(MAC Real Time\)](#) runs inside the PL360/PL460 device.

It is a wrapper to the PLC MAC RT, which offers the same API as that module, but makes use of the [1.3. PLC Host Controller](#) to take control of the MAC RT running inside PL360/PL460.

1.8 MAC Layer and MAC Wrapper

The PLC MAC layer is provided by Microchip as a compiled library.

Taking into account that PLC MAC RT is already a separated module, the PLC MAC layer has the following functions:

- Security. Frame Encryption and Frame Decryption/Validation
- MAC IB (Information Base) management

The RF MAC layer is provided by Microchip as a compiled library. This layer has the following functions:

- Channel access arbitration
- Network scan. Generation and processing of BCN and BREQ frames
- Frame Acknowledgment and re-transmission management
- Security. Frame Encryption and Frame Decryption/Validation
- POS (Personal Operating Space) management by means of Information Elements
- MAC IB (Information Base) management

The MAC Wrapper is a source code block that acts as an interface with both the PLC and RF MAC layers. The reasons for having such a wrapper include:

- Its interface offers the same functionality as PLC and RF MAC APIs, so upper layers must access any of the MAC layers through the Wrapper, removing any reference to the MAC header files and, thus, isolating the MAC libraries from the upper layers.
- Table sizes and G3 Specification compliance are defined in this wrapper and are later initialized at run time in the MAC libraries. This allows unique PLC and RF MAC libraries for any Device type and target Specification compliance.

The MAC Wrapper includes the Hybrid Abstraction Layer, a source code block that acts as interface between the Adaptation Layer ([1.9. ADP Layer, ADP Conf and ADP API](#)) and both the PLC and RF MAC layers.

The role of this layer is to act as a common input/output area for both the PLC and RF MAC Layers, making the Hybrid Profile as transparent as possible to the ADP and the upper layers.

It manages the Requests coming from ADP, Routing and LBP modules, and forwards them to one or both of the MAC layers. In the opposite direction, manages the Confirm and Indication primitives coming from any of the MAC layers and forwards them to the ADP, Routing or LBP module, indicating the MAC layer(s) that generated the event.

The MAC Wrapper interface is defined in [8.6. MAC Wrapper](#).

1.9 ADP Layer, ADP Conf and ADP API

The Adaptation Sublayer (ADP) is provided by Microchip as a compiled library. It has the following functions:

- IPv6 headers compression / decompression
- Fragmentation / reassembly of the IPv6 packets
- Controlling broadcast / multicast propagation

- Routing a message into the network, with the help of the Routing Module
- ADP IB (Information Base) management

The ADP Conf is a source code module where some Tables and Data Buffers are declared and are later referenced at run time in the ADP library. This allows a unique ADP library for any device type and network size, by changing the size and number of such Tables and Buffers.

The ADP API is the entry point for upper layers to the G3 stack, and, thus, any access is done through this API and no other module is directly accessed.

The ADP API interface is defined in [8.8. ADP SAP](#), [8.9. ADP Data Primitives](#), [8.10. ADP Management Service Primitives](#) and [8.11. ADPIB Objects Specification and Access](#).

1.10 Routing Module and Routing Wrapper

The Routing Module is provided by Microchip as a compiled library. It implements the LOADng protocol, as specified in Annex D of the G3 specification.

The purpose of this module is the discovery and maintenance of the routes into the network.

The Routing Wrapper is a source code block that acts as an interface with the Routing Module. The reasons for having such a wrapper include:

- Its interface gives access to all Routing functionality required by other modules, so these modules access the Routing Module through the Wrapper, removing any reference to the Module itself and, thus, isolating the Routing library from other modules.
- Table sizes are defined in this wrapper and are later initialized at run time in the Routing library. This allows a unique Routing library for any Device type and Network size.
- In a solution where Routing is not used, the Routing library will not be included in the final system. The wrapper provides the proper interface and return values in case the Routing library is not present.

1.11 Common Utils Module

This Module contains shared resources such as Timers, Queue Management, Type Conversion, Random Number Generation and Logging capabilities.

It is used by the ADP, MAC and Routing libraries, as well as by their auxiliary modules. It is provided as source code.

1.12 Crypto Module and Crypto Wrapper

The Crypto Wrapper contains the necessary API to encrypt/decrypt at the ADP and MAC layers, independently from the specific implementation below.

Provided examples make use of mbedTLS, an ARM's standard crypto library implementation. This library is provided as a reference, but can be changed by another crypto engine with the proper Crypto Wrapper adaptation.

The provided mbedTLS was adapted to use the HW capabilities of the microcontroller, if available.

1.13 HAL (Hardware Abstraction Layer)

Besides the G3-PLC stack, Microchip provides an open source module for the Hardware Abstraction Layer (HAL). This module provides function API-based service to the higher-level layers that allows them to perform hardware-oriented operations independent of actual hardware details.

Note: The provided HAL source code is only an implementation example. Users should modify the code according to their hardware and specifications.

There are different groups of functions, e.g., to initialize HW, write parameters in non-volatile memory, Power-Down detection, timer interrupt management, etc.

1.14 Coordinator Module

The Coordinator module is part of a G3 PAN Coordinator, and, thus, is not present in the G3 PAN Device examples.

This module is in charge of EAP-PSK on the Coordinator side, and, thus, decides whether a Device is accepted in the network or not.

Note: The provided module is a basic example, not intended to be a complete module in a real Coordinator. This module is out of the G3 spec, and it is provided for testing purposes.

1.15 LBP Module

LoWPAN Bootstrap Protocol (LBP) and its role is to manage the creation of a network (in coordinator) or the joining to the network (in devices), by performing tasks, such as authentication and encryption key distribution.

The module has some shared functionality for both the coordinator and the device, such as encoding/decoding of the LBP frames, and some separated modules which are present or not depending on the node type, to manage the joining process from both sides.

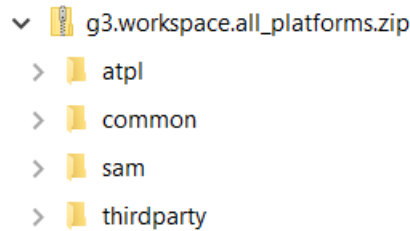
This module interacts with both the Coordinator Module (if present) and the ADP Library.

2. Understanding the Firmware Package

The Microchip G3-PLC firmware package is provided as a single zip file containing all the code and libraries needed to run G3 example applications on supported boards. This file is named after “G3 Workspace”. The package contains G3-specific code along with common ASF (Advanced Software Framework) modules and drivers that are required for this G3-PLC firmware to work. This chapter provides some general guidelines on how the different software modules are structured.

2.1 G3-PLC Workspace Structure

Figure 2-1. G3 Workspace



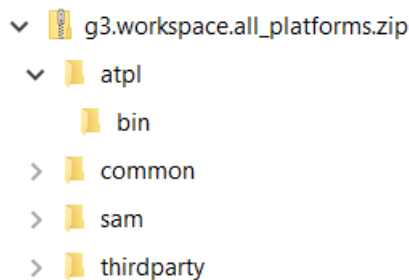
The workspace package is divided into four folders: *atpl*, *common*, *sam* and *thirdparty*.

This chapter locates and briefly explains G3-specific code inside the workspace. General purpose modules used by G3 are provided in the workspace but not highlighted here.

The contents and purpose of each folder is detailed in the following sections.

2.1.1 atpl folder

Figure 2-2. atpl/bin folder

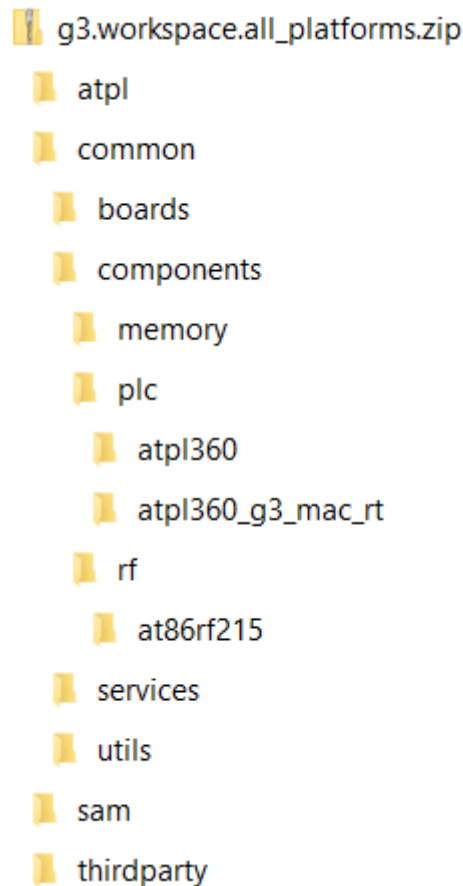


The *atpl* folder contains only one element, the *bin* sub-folder.

This folder contains the PL360/PL460 binary files. A tool for binary encryption is also provided and located in this folder.

2.1.2 common folder

Figure 2-3. common/components/plc & rf folders



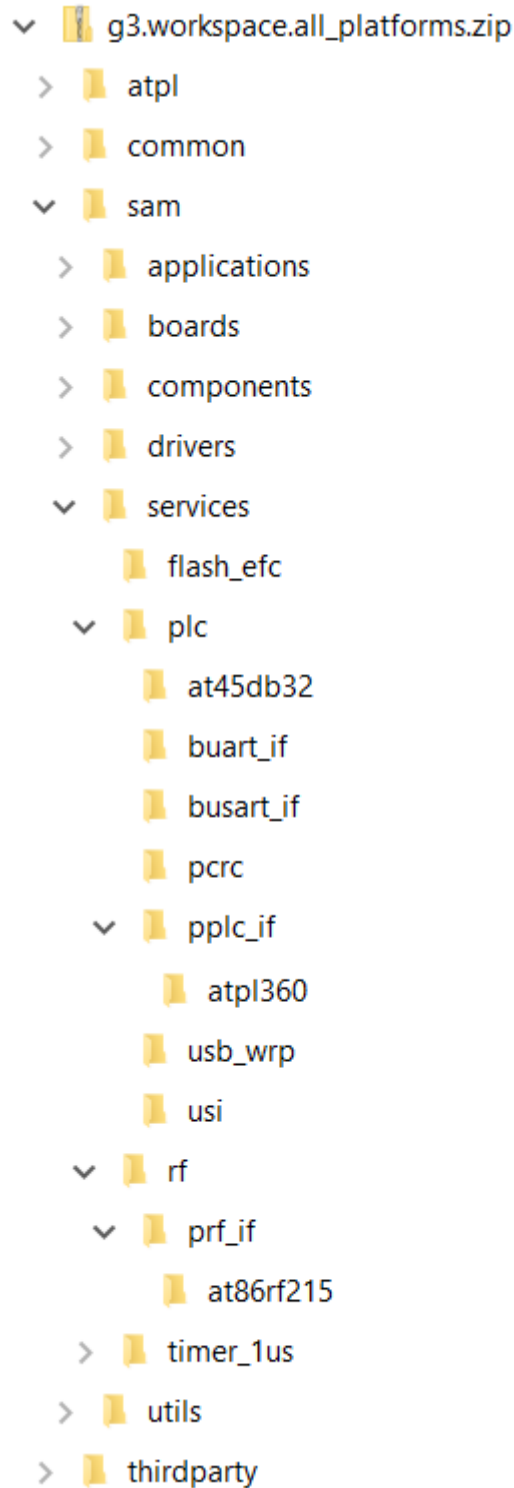
The common folder contains general purpose modules used by the G3 stack, and adds a specific one for the PL360/PL460 device, the *common/components/plc* folder.

This folder contains the [1.3. PLC Host Controller](#) module to perform and monitor all transactions with the PL360/PL460 device. As PL360/PL460 running PHY layer (for PHY level applications) or running PHY + MAC RT layers (for G3 applications) have different Host Controllers, two sub-folders are present, one for each implementation.

The Hybrid Profile introduces the *common/components/rf* folder, where the AT86RF215 Controller is located, which includes the RF PHY layer and associated modules.

2.1.3 sam folder

Figure 2-4. sam/services/plc & rf folders



The *sam* folder contains modules shared for all SAM devices that are used by the G3 stack, and adds a specific one containing code used by PLC stacks, the *sam/services/plc* folder.

Services are modules that are placed above drivers, and whose intent is to perform specific tasks using such drivers to add an extra abstraction layer to facilitate some operations to the users.

at45db32 service manages read/write/erase operations for the at45db32 Flash memory, instead of having to access the SPI driver directly.

buart_if and *busart_if* (named after buffered uart and buffered usart) offer buffered tx/rx access to UART and USART peripherals, handling UART/USART drivers without interrupting their users until the whole buffer is transmitted or received.

pcrc service is included in all PLC applications, but not used by G3 in particular, thus, is beyond the scope of this guide.

pplc_if service is in charge of managing all SPI transactions with the PLC transceiver (PL360/PL460), providing a simplified API to the modules that interact with the transceiver.

prf_if service is in charge of managing all SPI transactions with the RF transceiver, providing a simplified API to the modules that interact with the transceiver.

usb_wrp service is the wrapper to use the USB port as a serial interface.

usi service (Universal Serial Interface), is an HDLC-type encapsulation module used by some G3 applications where the board running G3 is connected through a Serial Link to a Host PC or micro-controller that runs its counterpart (USI Host) to interact with G3 stack, abstracting the Serial connection.

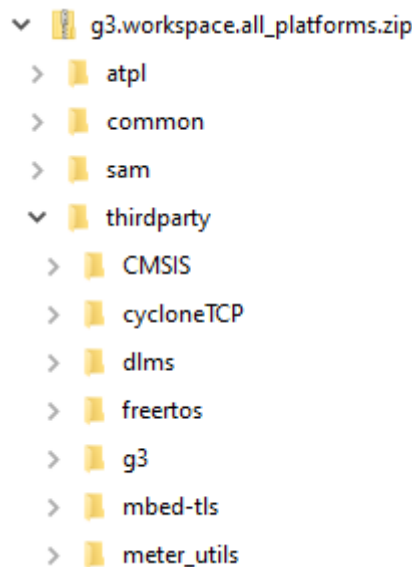
The *sam* folder contains also a Bootloader in both Project and Binary formats to be used along with PL360G55CB-EK and PL360G55CF-EK boards. These project and binary can be found in the folder: `\sam\applications\serial_bootloader`.

This bootloader allows the target boards to be reprogrammed without the need of additional HW programming tools. Only a connection through USB or UART and a PC tool are needed.

Note: If using the USB interface with Windows versions prior to 10, it is necessary to install a driver in the PC to create a Virtual COM port that will be used for the programming process. For this virtual port to be created, the computer will ask for a driver the first time the board is connected. The driver can be found in the folder: `\common\services\usb\class\cdc\device\atmel_devices_cdc.inf`.

2.1.4 thirdparty folder

Figure 2-5. thirdparty folder

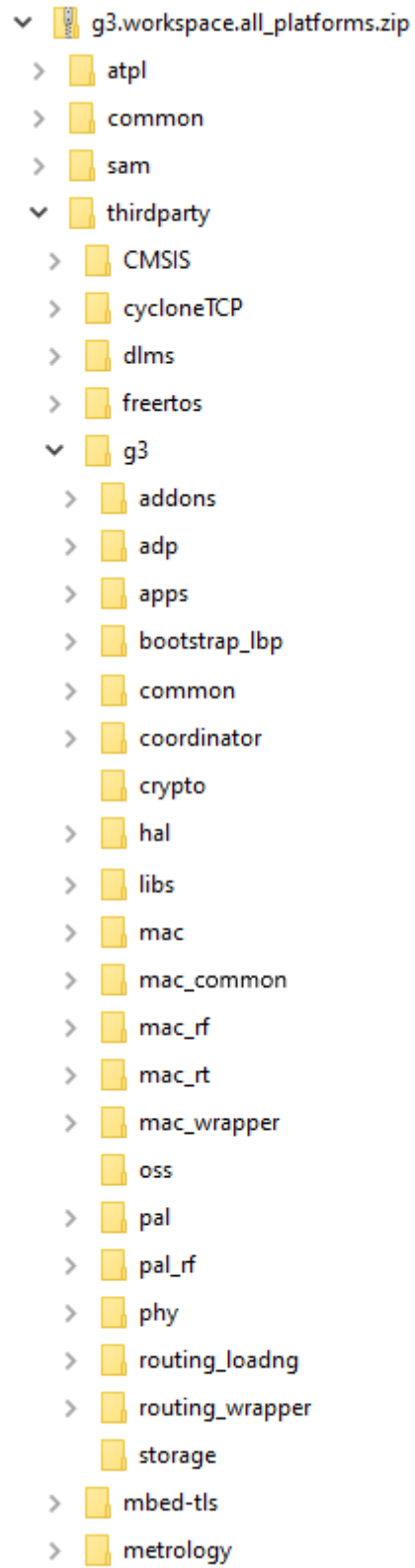


This folder is where code not related to specific modules/drivers/services is placed. In the G3 workspace, the following folders are included:

- *CMSIS*: where Cortex-M system files and libraries can be found.
- *cycloneTCP*: contains an IPv6 layer implementation from a third party, distributed under the license terms defined by the provider.
- *dlms*: contains an example implementation of a DLMS client and server, for G3 demonstration purposes.
- *freertos*: this RTOS is included as an example on how to integrate G3 stack in this kind of embedded solutions.
- *g3*: contains all G3 code and most of its auxiliary modules.
- *MBED-TLS*: is the default cryptographic library that G3 relies on to perform security duties.
- *metrology/meter_utils*: metrology-related code, used in projects including metrology functionality.

2.1.4.1 g3 folder

Figure 2-6. thirdparty/g3 folder



As previously stated, the *g3* folder contains all the G3 code and most of its auxiliary modules.

The folder structure reflects the G3 Architecture, as presented in [1. General Architecture](#). To summarize the modules already presented and present the new ones, the different sub-folders are listed here:

- *addons*: contains serialization layers to access ADP and MAC layers through a Serial Link using USI protocol.
- *adp*: contains the ADP library, configuration file and API header files, as presented in [1.9. ADP Layer, ADP Conf and ADP API](#) and detailed in subsections of [8. API of DATA LINK Layer](#).
- *apps*: contains G3 stack example applications; see [5.1. ADP Examples](#) for detailed information.
- *bootstrap_lbp*: contains the Local Bootstrap Protocol module common to the Coordinator and Device as presented in [1.15. LBP Module](#).
- *common*: contains auxiliary utils as presented in [1.11. Common Utils Module](#).
- *coordinator*: contains the Coordinator module implementation example as presented in [1.14. Coordinator Module](#).
- *crypto*: contains the cryptography wrapper and auxiliary files as presented in [1.12. Crypto Module and Crypto Wrapper](#).
- *hal*: contains Hardware Abstraction Layer as presented in [1.13. HAL \(Hardware Abstraction Layer\)](#).
- *libs*: contains the G3 stack libraries, which include ADP libs, PLC MAC lib, RF MAC lib and Routing libs.
- *mac*: contains header files, which define PLC MAC API, detailed in subsections of [8. API of DATA LINK Layer](#).
- *mac_common*: contains common parts for both PLC and RF MAC layers.
- *mac_rf*: contains header files, which define RF MAC API, detailed in subsections of [8. API of DATA LINK Layer](#).
- *mac_rt*: contains MAC Real Time source code, as presented in [1.6. PLC MAC RT \(MAC Real Time\)](#).
- *mac_wrapper*: contains the MAC layer Wrapper, as presented in [1.8. MAC Layer and MAC Wrapper](#) and detailed in [8.6. MAC Wrapper](#).
- *oss*: contains the Operating System Support, explained in [4. Operating System Support](#).
- *pal*: contains PHY Abstraction Layers for both PL360 and ATPL250 devices, as presented in [1.4. PLC PAL \(PHY Wrapper\)](#) and [1.7. PLC PAL 360/460 MAC RT \(PLC MAC RT Wrapper\)](#).
- *pal_rf*: contains PHY Abstraction Layers for the RF215 device, as presented in [1.5. RF PAL \(PHY Wrapper\)](#).
- *phy*: contains the PHY layer example applications, detailed in [5.2. PLC PHY Examples](#).
- *routing_loadng*: contains LOADng protocol API files, as presented in [1.10. Routing Module and Routing Wrapper](#).
- *routing_wrapper*: contains Routing module wrapper, as presented in [1.10. Routing Module and Routing Wrapper](#).
- *storage*: contains an auxiliary module, which handles G3 non-volatile data storage requirements; see [3.4. Persistent Data Storage](#) for more info.

3. Understanding the G3-PLC FW Stack

The following chapter explains how the G3-PLC FW stack has to be initialized, started and called periodically. It also explains how to integrate a User Application over the G3 stack.

The basic procedure for performing this integration is:

- Initializing and starting the FW stack
- Developing user application requirements
- Modifying the configuration files according to the application requirements

3.1 Operation Modes (Operating System Support)

There are two different operation modes available:

- Microcontroller Operation mode
- RTOS Operation mode

The operation mode is defined in the *conf_oss.h* file under the following define sentence:

```
/* OS Support */
#define OSS_USE_FREERTOS
```



Important: For configuring the FW stack to operate in RTOS mode, define `OSS_USE_FREERTOS`. If `OSS_USE_FREERTOS` is not defined, the FW is executed in Microcontroller Operation mode. (See [OSS Layer Configuration – conf_oss.h](#).)



Tip: The default RTOS included in the stack is FreeRTOS. Also note that the RTOS mode is provided as an example of integration of the stack inside an OS for demonstration purposes only. It must not be taken as the starting point of a development, and Microchip does not ensure the same performance and stability as the examples running in Microcontroller mode.

This compilation constant changes the way code is executed in the *oss_if.c* file, which contains the code in charge of controlling the program flow. This file contains all the calls to initialization functions, calls to time control functions and calls to periodic task handler functions. All this is done in a different way depending on the operation mode, but the idea of controlling the program flow from this file is the same.



In the PL360 platform, where MAC RT runs inside the device, and for PLC-only devices, the time constraint for calling the G3 periodic task can be relaxed up to 20 ms, as time-critical tasks are handled inside the PL360.

For Hybrid devices, a 1-ms call rate has to be kept to ensure the PHY RF duties are correctly handled.

3.2 Initializing and Starting G3-PLC Stack and User Application

Here is an example, *dlms_app_coord*, of how the G3 PLC stack is configured and started.

The program entry point function is simple:

```
void main( void )
{
```

```

oss_task_t x_task;

/* Initialize OSS */
oss_init();

/* Register Task */
x_task.task_init = app_init;
x_task.task_process = app_process;
x_task.task_lms_timer_cb = app_timers_update;
oss_register_task(&x_task);

/* Register Dispatcher Task */
x_task.task_init = dispatcher_app_init;
x_task.task_process = dispatcher_app_process;
x_task.task_lms_timer_cb = dispatcher_timers_update;
oss_register_task(&x_task);

/* Register Ping Task */
x_task.task_init = ping_app_init;
x_task.task_process = ping_app_process;
x_task.task_lms_timer_cb = ping_app_timers_update;
oss_register_task(&x_task);

/* Register UDP Responder Task */
x_task.task_init = udp_responder_app_init;
x_task.task_process = udp_responder_app_process;
x_task.task_lms_timer_cb = udp_responder_app_timers_update;
oss_register_task(&x_task);

/* Start OSS */
oss_start();
}

```

Three clear tasks are visible here:

- Initialization
- Task registration of different modules for further execution
- Start execution

These three steps are explained further in the following sections.

3.2.1 First Step: Initialization

```

void oss_init(void)
{
    /* Prepare the hardware */
    platform_init_hw();

    memset(&sx_oss_init_tasks, 0, sizeof(sx_oss_init_tasks));
    memset(&sx_oss_proc_tasks, 0, sizeof(sx_oss_proc_tasks));
    memset(&sx_oss_updt_tasks, 0, sizeof(sx_oss_updt_tasks));
    memset(&sx_oss_proc_prio_tasks, 0, sizeof(sx_oss_proc_prio_tasks));

    suc_oss_num_task_registered = 0;
}

```

First of all, the stack initializes the hardware platform according to its requirements by calling the *platform_init_hw()* function.

When using hardware different than the hardware supported by Microchip, HAL files may have to be changed according to user needs. All these files are available as source code for this purpose. In case Microchip evaluation boards are used, hardware will be correctly initialized and the user does not need to take care of this.

Then, task containers are reset and the number of registered tasks set to 0 to ensure a correct registration later.

3.2.2 Second Step: Task Registering

The OSS file (*oss_if.c*) is in charge of controlling all the program flow, as seen in [Operation Modes \(Operating System Support\)](#).

To call user application functions from this file, such functions have to be registered as tasks. The task prototype contains four fields, which the user will assign to callback functions:

- *task_init* field: to assign a callback function to perform initialization duties, such as variable initialization.
- *task_process* field: to assign a callback function to be called periodically by OSS, such as an application event handler. The rate at which this event handler will be called can be configured through the compilation constant `OSS_APP_EXEC_RATE`, which can be found in the *oss_if.h* file.
- *task_1ms_timer_cb* field: to assign a callback function to be called from a 1-ms timer interrupt. This is typically used for time control short duties requiring precision because it is called from timer interrupt, so the user has to avoid long processing time on it. (This is intended for simple duties such as incrementing a millisecond counter variable).
- *task_process_prio* field: to assign a callback function for high priority processes of the task. This high priority process is called in every program loop, instead of using the `OSS_APP_EXEC_RATE` value.

If a task does not require one of the offered fields, it can be set to *Null*, and OSS will not call any callback function for such task. For example, if a task does not need time control at all, the *task_1ms_timer_cb* field can be set to *Null* before the task registration.

3.2.3 Third Step: Start Program Execution

The program execution is started using *oss_start* function. The Microcontroller mode will be used to explain the different functions and processes involved in program execution. The OS mode can be easily followed in the provided code and it is not explained here, as functionality is exactly the same but performing tasks in an RTOS way.

```
void oss_start(void)
{
    /* Set up timer interrupt and user defined callback */
    platform_set_ms_callback(& oss_1ms_timer_handler);
    platform_led_cfg_blink_rate(OSS_LED_BLINK_RATE);
    platform_cfg_call_process_rate(OSS_G3_STACK_EXEC_RATE);
    platform_cfg_call_app_process_rate(OSS_APP_EXEC_RATE);

    #if defined(PLATFORM_PDD_INTERNAL_SUPPLY_MONITOR) ||
    defined(PLATFORM_PDD_EXTERNAL_VOLTAGE_DIVIDER)
        /* Power down detection initialization */
        platform_init_power_down_det();
    #endif

    #ifdef OSS_ENABLE_IPV6_STACK_SUPPORT
        /* Initialize IPv6 stack */
        netInit();
    #endif

    #ifdef NUM_PORTS
        /* Initialize USI */
        usi_init();
    #endif

    /* Task-registered initialization */
    _oss_execute_tasks(OSS_TASK_INIT);

    /* Timers may interfere with task initialization, init after */
    platform_init_1ms_timer();

    /* main loop */
    while (1) {
        /* Reset watchdog */
        RESET_WATCHDOG;

        if (platform_flag_call_process()) {
            /* G3 stack process */
            #ifdef OSS_G3_ADP_MAC_SUPPORT
                /* Internally calls MAC and PHY Event handlers */
                AdpEventHandler();
            #endif
        }
    }
}
```

```

    }

    if (platform_flag_call_app_process()) {
        /* Task-registered processes */
        _oss_execute_tasks(OSS_TASK_PROCESS);

        #ifdef NUM_PORTS
        usi_process();
        #endif

        /* Handle TCP/IP events */
        #ifdef OSS_ENABLE_IPv6_STACK_SUPPORT
        netTask();
        #endif
    }

    /* LED blink */
    platform_led_update();

    /* Task-registered PRIO processes */
    _oss_execute_tasks(OSS_TASK_PROCESS_PRIO);
}
}

```

This function performs some interesting tasks, which are explained in the next subsections.

3.2.3.1 Time Control Initialization

```

/* Set up timer interrupt and user defined callback */
platform_set_ms_callback(&_oss_lms_timer_handler);
platform_led_cfg_blink_rate(OSS_LED_BLINK_RATE);
platform_cfg_call_process_rate(OSS_G3_STACK_EXEC_RATE);
platform_cfg_call_app_process_rate(OSS_APP_EXEC_RATE);

```

First of all, the function `platform_set_ms_callback` configures a callback to be called every time an HW interrupt raises (which is defined and handled in `hal.c` file). This callback function is called every 1 ms, independently of the platform below, and it is in charge of managing the time base for the G3 stack.

The next function, `platform_led_cfg_blink_rate`, configures the rate at which an activity LED blinks in the board periodically, just to indicate that program is running. The rate is defined by the constant `OSS_LED_BLINK_RATE`, located in file `conf_oss.h`, and its value is indicated in milliseconds. This is an auxiliary function that does not affect the G3 stack functionality at all.

Finally, functions `platform_cfg_call_process_rate` and `platform_cfg_call_app_process_rate` are used to define the rate at which the G3 stack and user application event handlers are called. They are defined using the constants `OSS_G3_STACK_EXEC_RATE` and `OSS_APP_EXEC_RATE` located in the `oss_if.h` file. If running MAC RT in the PL360, the value is overridden, re-defining constants with a new value in the `conf_oss.h` file.

3.2.3.2 Power-Down Detector Initialization

```

#if defined(PLATFORM_PDD_INTERNAL_SUPPLY_MONITOR) ||
defined(PLATFORM_PDD_EXTERNAL_VOLTAGE_DIVIDER)
/* Power down detection initialization */
platform_init_power_down_det();
#endif

```

The G3 stack needs some information to be stored in non-volatile memory so that it can be restored after a Power-Down. For this purpose, a Power-Down detection mechanism is implemented in the HAL module. To use this Power-Down detector, either `PLATFORM_PDD_INTERNAL_SUPPLY_MONITOR` or `PLATFORM_PDD_EXTERNAL_VOLTAGE_DIVIDER` has to be defined in the `conf_hal.h` file. When it is defined, all needed hardware is initialized using function `platform_init_power_down_det`.

For further information regarding how Power-Down detection is performed, see [6.5. Power-Down Detection](#).

3.2.3.3 IPv6 Stack Support

```
#ifdef OSS_ENABLE_IPV6_STACK_SUPPORT
/* Initialize IPv6 stack */
netInit();
#endif
```

The G3 protocol is meant to use an IPv6 layer on top of it in most of the applications. IPv6 is considered out of the scope of the G3 stack; therefore, it is not part of the Microchip G3 library.

An implementation of a third party of an IPv6 stack is provided to demonstrate the use of DLMS on top of it and the link between IPv6 and the G3 stack.

The use of this IPv6 is optional so the user can build an application directly over the G3 stack without the IPv6 layer, but as mentioned before, this is not the usual case. The use of this layer depends on the definition of the constant `OSS_ENABLE_IPV6_STACK_SUPPORT`, located in file `conf_oss.h`. In the provided example, the IPv6 layer is needed, and, thus, this constant is defined by default.

3.2.3.4 USI Initialization

```
#ifdef NUM_PORTS
/* Initialize USI */
usi_init();
#endif
```

If a Universal Serial Interface (USI) is used, it is initialized here using the `usi_init` function. To call this function, `NUM_PORTS` must be defined in the `conf_usi.h` file.

The examples provided include an embedded sniffer functionality that requires the USI. The `conf_usi.h` file in the project can be inspected to see a configuration example.

For more information about what USI is and how to use it, refer to [13. Serialization with Embedded USI](#).

3.2.3.5 Registered Tasks Initialization

```
/* Task-registered initialization */
_oss_execute_tasks(OSS_TASK_INIT);
```

As seen in [3.2.2. Second Step: Task Registering](#), it is possible to set a pointer to a callback function where a registered task performs its initialization duties.

The function `_oss_execute_tasks(OSS_TASK_INIT)` executes initialization callbacks for all registered tasks.



Important: It is required that one of the registered tasks initializes the G3 stack using the function `AdpInitialize`. This is not done directly by OSS because one of the parameters to set in the G3 stack initialization is the pointer to the callback functions that the G3 stack calls when notifications are reported from the stack to the application, and such callback functions have to be implemented by the application. This is why the application must call the G3 stack initialization function.

For more information regarding the `AdpInitialize` function, refer to [Initialization Function](#).

3.2.3.6 Timer Start-Up

```
/* Timers may interfere with task initialization, init after */
platform_init_lms_timer();
```

After all of the initialization tasks are performed, the 1-ms timer interrupt is configured and enabled.

Once enabled, all callbacks (both internal and user-defined) registered to the timer interrupt will be called every 1 ms from the interrupt.

3.2.3.7 Main Loop

After all initializations seen in previous sections, the program enters in the main infinite loop. The following recursive processes are performed in this main loop.

3.2.3.7.1 Reset Watchdog

On every main loop execution cycle, the watchdog timer is reloaded. The watchdog timer is used to detect program execution anomalies and reset the processor if any occur.

To disable the watchdog, the constant `CONF_BOARD_KEEP_WATCHDOG_AT_INIT`, located in the `conf_board.h` file, has to be undefined.

3.2.3.7.2 Process Call Check

```
if (platform_flag_call_process()) {  
    ...  
}
```

```
if (platform_flag_call_app_process()) {  
    ...  
}
```

The call of periodic functions, or event handlers, of the stack and app layers is done depending on the result value of the functions `platform_flag_call_process` and `platform_flag_call_app_process`. As described in [Time Control Initialization](#), a call rate for event handlers is configured through the constants `OSS_G3_STACK_EXEC_RATE` and `OSS_APP_EXEC_RATE`. **Both are configured to 1-millisecond period by default, but for the G3 examples on the PL360 platform, it is overridden due to relaxed time constraints.**

In summary, these functions will return *true* when `OSS_G3_STACK_EXEC_RATE` and `OSS_APP_EXEC_RATE` are elapsed from the previous call, and, thus, event handlers of different layers will be called.

3.2.3.7.3 Registered Tasks Event Handler

```
/* Task-registered processes */  
_oss_execute_tasks(OSS_TASK_PROCESS);
```

If a pointer is set to a function that is called periodically as an event handler (as in [3.2.2. Second Step: Task Registering](#)), it will be called inside the `_oss_execute_tasks (OSS_TASK_PROCESS)` function.

The function will call the event handlers for all the registered tasks in the same order they were registered.

3.2.3.7.4 USI Event Handler

```
#ifdef NUM_PORTS  
    usi_process();  
#endif
```

As in [USI Initialization](#), if the `NUM_PORTS` constant is defined, the USI event handler will be called here.

3.2.3.7.5 G3 Stack Event Handler

```
#ifdef OSS_G3_ADP_MAC_SUPPORT  
    /* Internally calls MAC and PHY Event handlers */  
    AdpEventHandler();  
#endif
```

If constant `OSS_G3_ADP_MAC_SUPPORT` is defined (in `conf_oss.h`), the G3 stack event handler is called.

The function `AdpEventHandler` is the event handler for the ADP layer. This function internally manages the call to the lower layers of the stack (MAC, PAL and PHY), so the user does not have to worry about managing all these handlers.



Important: As stated previously, this function has to be called every 1 ms when using ATPL250 platform to meet the time constraints defined in G3.

One reason for not defining this constant could be, for example, when an application is running directly over the PHY layer and the PHY event handler is called directly from the application event handler.

For more information regarding the *AdpEventHandler* function, refer to [Event Handler Function](#).

3.2.3.7.6 IPv6 Event Handler

```
#ifdef OSS_ENABLE_IPV6_STACK_SUPPORT
    netTask();
#endif
```

As explained in [IPv6 Stack Support](#), the provided example makes use of a third party IPv6 stack. When the *OSS_ENABLE_IPV6_STACK_SUPPORT* constant is defined, its event handler will be called here.

3.2.3.7.7 Registered Tasks Event Handler

Regardless of the result from *platform_flag_call_app_process()* function, high priority processes of the registered tasks are called in every program loop.

```
/* Task-registered PRIO processes */
_oss_execute_tasks(OSS_TASK_PROCESS_PRIO);
```

If a pointer is set to a function that is called periodically as a high priority event handler (as in [3.2.2. Second Step: Task Registering](#)), it will be called inside the *_oss_execute_tasks* (*OSS_TASK_PROCESS_PRIO*) function.

The function will call the high priority event handlers for all the registered tasks in the same order they were registered.

3.2.3.7.8 Activity LED Blinking

```
platform_led_update();
```

Finally, the main loop checks whether the activity LED has to blink using the *platform_led_update* function. The blinking of the LED is decided depending on the *OSS_LED_BLINK_RATE* constant, as described in [Time Control Initialization](#).

3.3 Configuration Files

The stack contains a variety of configuration files, which allow some features to be active or not, or to change some behaviors.

Each provided application contains a folder for each supported device and associated evaluation board. This folder contains the projects for the corresponding device and board, and the configuration files for those projects. Apart from a particular device or board configuration files, the following configuration files are available:

- *conf_at86rf.h* (Only in projects supporting the Hybrid Profile)
- *conf_atpl360.h* (Only in projects using PL360 device)
- *conf_board.h*
- *conf_bs.h* (Only in projects for Coordinator)
- *conf_buart_if.h*
- *conf_busart_if.h*
- *conf_clock.h*

- `conf_oss.h`
- `conf_hal.h`
- `conf_pplc_if.h`
- `conf_prf_if.h` (Only in projects supporting the Hybrid Profile)
- `conf_project.h`
- `conf_tables.h`
- `conf_timer_1us.h` (Only in projects supporting the Hybrid Profile)
- `conf_uart_serial.h`
- `conf_usb.h`
- `conf_usi.h`
- `FreeRTOSConfig.h`

The meaning of these configuration files is described in more detail in the following sections.

3.3.1 RF215 Configuration – `conf_at86rf.h`

Constants used for RF215 configuration. Available constants are:

- `AT86RF_PART` – Selects between AT86RF215 and AT86RF215M parts
- `AT86RF215_DISABLE_RF24_TRX` – Disables 2.4 GHz band
- `AT86RF215_DISABLE_RF09_TRX` – Disables SubGHz band
- `AT86RF215_ENABLE_AUTO_FCS` – Enables automatic calculation and append of FCS (Frame Check Sequence) in transmission and verification on reception
- `AT86RF215_MAX_PSDU_LEN` – Defines maximum PSDU length for buffers memory allocation
- `AT86RF215_NUM_RX_BUFFERS` – Number of reception buffers, useful in case RX data is collected at a low rate from upper layers
- `AT86RF215_NUM_TX_PROG_BUFFERS` – Number of programmed transmission buffers, useful in case more than one frame has to be programmed for a future transmission
- `AT86RF_ADDONS_ENABLE` – Enables RF215 add-ons, which add sniffer capabilities

3.3.2 PL360/PL460 Configuration – `conf_atpl360.h`

Constants used for PL360/PL460 configuration. Available constants are:

- `ATPL360_ADDONS_ENABLE` – Enables PL360/PL460 add-ons, which include sniffer capabilities and/or serial access to PHY API
- `ATPL360_RST_WAIT_MS` – Wait period before downloading PL360/PL460 binary retries if download verification fails
- `ATPL360_WB` – Defines work band for PL360/PL460 (`ATPL360_WB_CENELEC_A`, `ATPL360_WB_CENELEC_B`, `ATPL360_WB_FCC`, `ATPL360_WB_ARIB`). Not used on projects with Mac RT running in PL360/PL460, where band dependency is handled inside PL360/PL460 binary.

3.3.3 Board Configuration – `conf_board.h`

This file defines the peripherals used and their configuration, such as:

- Watchdog Timer can be enabled/disabled using constant `CONF_BOARD_KEEP_WATCHDOG_AT_INIT`.
- Console UART, if used, is defined here:

```
#define CONSOLE_UART    UART1
#define CONSOLE_UART_ID ID_UART1
```

- External slow clock usage is enabled by defining constant `CONF_BOARD_32K_XTAL`

- Support for different peripherals (UART, SPI, TWI), and for external component control (LCD, external memory) are defined here.

3.3.4 Bootstrap Module Configuration – `conf_bs.h`

Configuration of Bootstrap and Coordinator. Contains:

- `G3_COORDINATOR_PAN_ID` – Defines PAN ID for the network to be created by the Coordinator
- `MAX_LBDS` – Number devices that are allowed to join the network
- `LBS_INVALID_SHORT_ADDRESS` – Address reserved for the Coordinator and considered invalid for devices
- `INITIAL_KEY_INDEX` – Initial Key Index (of the 2 available indexes) to use when setting GMK

3.3.5 Buffered UART Configuration – `conf_buart_if.h`

Configuration of HW timers and buffer sizes for the UARTs.

3.3.6 Buffered USART Configuration – `conf_busart_if.h`

Configuration of HW timers and buffer sizes for the USARTs.

3.3.7 Clock Configuration – `conf_clock.h`

Configuration of the clock source and clock tree of the microcontroller.

- System clock source is configured among different options:
 - External Crystal
 - Internal RC
 - PLL
- System clock prescaler is chosen
- PLL source, multiplier and divider are selected, if a PLL is used
- USB clock source is configured (on devices supporting it)
- Coprocessor clock is configured (on dual core devices)

Configuration provided on G3 examples are intended to take full advantage of MCU and Evaluation Board capabilities. For an alternative configuration, check [5.4. Clock Configuration Example for Low Requirements Module](#).

3.3.8 OSS Layer Configuration – `conf_oss.h`

On this file, the user can decide to run the program in Microcontroller mode or use an OS, which layers are used in the project, and the periodicity of some tasks.

- `OSS_USE_FREERTOS` – Defining this will cause the program to run in OS mode (using FreeRTOS as the supported and included RTOS). By default this is not defined, so Microcontroller mode will be used.
- `OSS_LED_BLINK_RATE` – Sets the blinking rate of the activity LED. This may be used to set a different rate to different applications to distinguish them when running on different boards.
- If the user wants to override the frequency at which the Application and G3 Stack Event Handlers are called, the values of `OSS_APP_EXEC_RATE / G3_APP_PROCESS_EXEC_RATE` and `OSS_G3_STACK_EXEC_RATE / G3_PROCESS_EXEC_RATE` have to be set in this file. When constants are defined in this file, the default value of 1 ms is overridden by the number of milliseconds set in these constants. The provided applications for Mac RT running in the PL360 Device use this method to relax the calling rate; refer to them for an example. Note that there are 2 cases where 1ms call rate has to be kept:
 - Devices running Hybrid Profile, as RF PHY requires low latency.

- Embedded Sniffer activation when working in FCC band, in order to not lose frames in the Sniffer Tool.
- The activation or deactivation of different layers is done by:
 - OSS_ENABLE_IPv6_STACK_SUPPORT – Adds an IPv6 layer to the running stack. See [IPv6 Stack Support](#) for more information.
 - OSS_G3_ADP_MAC_SUPPORT – Makes the ADP and MAC layers of the G3 stack to run in the current application. As explained in a previous section, this may be confusing because the user may expect that these layers have to always run, but this is left configurable to be able to run applications directly over the PHY layer (which would not be strictly G3 applications).

3.3.9 HAL Configuration – conf_hal.h

This file contains HW configuration parameters related to:

- Timers
- Power-Down detection
- LCD signaling (in case LCD support is available)
- G3 data storage
- EUI64 initialization source

3.3.10 PPLC Interface Configuration – conf_pplc_if.h

This file specifies the configuration of the hardwired interface between the microcontroller and the G3-PLC modem:

- Configuration of SPI module
- Interruption signaling
- Reset signaling
- LDO Enable signaling
- Standby control (if connection available)
- Thermal Warning Monitor pin definition (on PL460 based projects)
- TX Enable signaling (on PL460 based projects)
- ADC Channel definition for Vdd Monitor (on PL460 based projects)

The G3-PLC stack requires an external interrupt from the transceiver. For the ATPL250 device, the interrupt pin EINT is used. For the PL360, the firmware binary of the device configures GPIO3 (pin PA3 of PL360) as the external interrupt line. This file associates the external interrupt output of the transceiver with the stack running in the host microcontroller.

For further information about how the interrupt line of the PL360 works, refer to the PL360 Host Controller documentation.

3.3.11 PRF Interface Configuration – conf_prf_if.h

This file specifies the configuration of the hardwired interface between the microcontroller and the RF transceiver:

- Configuration of SPI module, including max length of transaction for buffer allocation
- Interruption signaling
- Reset signaling
- LED signaling

The G3-PLC stack requires an external interrupt from the transceiver. This file associates the external interrupt output of the transceiver with the stack running in the host microcontroller.

3.3.12 Project Configuration – `conf_project.h`

This file contains bandplan definition:

- `CONF_BAND_CENELEC_A` – 36 subcarriers [35,9375 - 90,625] kHz
- `CONF_BAND_CENELEC_B` – 16 subcarriers [98,4375 - 121,875] kHz
- `CONF_BAND_FCC` – 72 subcarriers [154,6875 - 487,5] kHz
- `CONF_BAND_ARIB` – 54 subcarriers [154,6875 - 403,125] kHz

On projects where multiple binaries for PL360 device are linked, so working band can be dynamically changed, constants `CONF_MULTIBAND_FCC_CENA` or `CONF_MULTIBAND_FCC_CENB` are defined here.

Particular PLC Coupling configurations, such as using a special branch configuration, are also defined on this file. For example `CONF_ENABLE_C11_CFG` has to be defined if `COUP_011` is used, or in PL460 projects, `CONF_ENABLE_PL460_FCC_1_5B_CFG` can be defined for a particular FCC configuration.

On projects requiring restoring IBs after a Power-Down, constant `ENABLE_PIB_RESTORE` is defined here.

For PL360G55CB-EK and PL360G55CF-EK boards, this file contains some macros for easy configuration of USI and Console ports on USB and MikroBUS:

```

/* Port mapping for USI and Console */

/* USI. Select One or None of the following options */
#define USI_ON_MIKROBUS_USART
/* #define USI_ON_USB */

/* Console. Select One or None of the following options */
/* #define CONSOLE_ON_MIKROBUS_USART */
#define CONSOLE_ON_USB

```

Using these macros, `conf_usi.h` and `conf_usart_serial.h` files automatically adapt their definitions to map ports in the desired way so the user does not need to manually modify them.



Tip: This file is where the user needs to include any compilation constant related to the user application code. It is a generic file local to the project that the user may extend depending on individual needs.

3.3.13 Tables size configuration – `conf_tables.h`

Some of the tables and buffers used in the G3 stack were left size-configurable for user convenience.

The tables and buffers sizes that can be configured in this file are the following:

- `CONF_COUNT_ADP_BUFFERS_1280` – Number of ADP TX buffers of size 1280 bytes
- `CONF_COUNT_ADP_BUFFERS_400` – Number of ADP TX buffers of size 400 bytes
- `CONF_COUNT_ADP_BUFFERS_100` – Number of ADP TX buffers of size 100 bytes
- `CONF_ADP_FRAGMENTED_TRANSFER_TABLE_SIZE` – Number of simultaneous fragmented Rx frames
- `CONF_ADP_ROUTING_TABLE_SIZE` – ADP Routing Table size
- `CONF_ADP_ROUTING_SET_SIZE` – ADP Routing Set size
- `CONF_LOADNG_RREP_GENERATION_TABLE_SIZE` – LOADng RREP Generation Table size
- `CONF_MAC_POS_TABLE_ENTRIES_RF` – RF MAC POS Table size
- `CONF_MAC_DSN_TABLE_ENTRIES_RF` – RF MAC Data Sequence Number Table size

- `CONF_NET_MEM_POOL_BUFFER_COUNT` – IPv6 Buffer Count
- `CONF_MAX_NEIGHBOUR_DLMS_SNAPSHOT_SIZE` – DLMS Neighbour Table Snapshot size
- `CONF_MAX_ROUTING_DLMS_SNAPSHOT_SIZE` – DLMS Routing Table Snapshot size
- `CONF_DLMS_MAX_DEV_NUM` – DLMS Supported Devices Table size (only for Coordinator)

More detailed information can be found in [6.1. Table Sizes and Buffer Count Configuration](#).

3.3.14 1 μ s Timer Configuration – `conf_timer_1us.h`

HW TC peripheral selection to be used as the source for the 1 μ s Timer Service.

3.3.15 Serial USART Service Configuration – `conf_uart_serial.h`

Definition of the console UART.

3.3.16 USB CDC Configuration – `conf_usb.h`

Constants used for USB Communication Device Configuration. Only used on boards where a native USB port is available.

These constants are tightly related with USB Virtual COM port configuration and are not intended to be modified.

3.3.17 Universal Serial Interface Configuration – `conf_usi.h`

If serialization is used for the project, this file contains the USI port and protocol definition and configuration.

`ENABLE_SNIFFER` constant is defined in this file to use embedded sniffer functionality.

3.3.18 Generic FreeRTOS Peripheral Control Functions Configuration – `FreeRTOSConfig.h`

FreeRTOS configuration that the user can modify according to their individual needs.



Take into account the requirements, handling and usage of the Microchip G3-PLC FW Stack prior to changing the RTOS configuration. Carefully read [Operating System Support](#).

3.3.19 Global Configuration Affecting all Projects – `conf_global.h`

Configuration files previously presented are project-scope files; therefore, there is a file for each of the available projects so configuration can be different from one project to another.

There is a special configuration file, `conf_global.h`, that is common to all projects. The idea is to store parameters here that are likely to be the same for different projects.

Currently, there are only one parameter to be configured in this file:

- `ENABLE_ROUTING` – The routing enabling parameter has to be inline with the inclusion or not in the project of Routing Library. If Routing library is not included, this parameter has to be set to 0 in order to correctly handle Routing requests from G3 on Routing Wrapper. By default, routing is enabled in CENELEC-A, CENELEC-B and FCC G3 modes of operation.

3.4 Persistent Data Storage

Requirements & Roles

The G3 stack requires that a certain number of IB values are preserved after a reset or even after a device is power cycled.

The IBs to be preserved are the following (as named in the Microchip G3 Stack API files):

- `MAC_WRP_PIB_FRAME_COUNTER`

- `MAC_WRP_PIB_FRAME_COUNTER_RF`
- `ADP_IB_MANUF_DISCOVER_SEQUENCE_NUMBER`
- `ADP_IB_MANUF_BROADCAST_SEQUENCE_NUMBER`



The G3 stack is not responsible for this task. It is an external module that has to be aware of a reset or power-down, to perform the read/storage/recover/write sequence of the IBs.

The G3 stack offers a helper callback function, *AdpUpdNonVolatileDataIndication*, which is called every time any of these IBs changes, so the upper layer can always have the latest values just in case they need to be recovered. This callback function only informs that the values have changed, but it is the upper layer that has to retrieve the IB values and keep the record.

Proposed Approach

The methods used to achieve this data preservation are up to the final user, but Microchip provides an example of implementation in the *Storage* module, which, in turn, makes use of the *HAL* module to access low-level hardware. The approach is a combination of data records on both GPBRs and Flash memory, where GPBRs are used in case of a microcontroller reset, and Flash memory is used in case of Power-Down.

Power-Down detection is done by the *HAL* module, see [6.5. Power-Down Detection](#) for details. The *HAL* module allows a callback function to be defined, which will be called when a Power-Down is detected.

Along with the IBs mentioned above, some extra data is stored:

- A counter incremented on each start-up, used to increase randomness in number generation on successive system start-ups, in microcontrollers that do not include a built-in TRNG block
- A version number and a CRC used to ensure the integrity of the stored data

Data Storage/Recovery Algorithm

Below are the steps followed to ensure data preservation:

1. During execution, and upon *AdpUpdNonVolatileDataIndication* callback, the Storage module retrieves the IBs from the G3 stack and stores them on GPBRs. This way we ensure that in case of a reset (other than Power-Down), we can retrieve information from these GPBRs.
2. When the Power-Down Callback function is called, the data is saved to Flash memory for further recovery.
3. Upon start-up, the reset source is checked and one of the following actions is taken:
 - a. In case of a Power-Down reset, Flash memory has the latest info written on Power-Down detection and GPBRs are empty. Then, data is recovered from Flash memory and, after validation, it is written to GPBRs and set to G3 IBs.
 - b. In case of any other reset cause, GPBRs contain the latest information and Flash does not. Then, data is recovered from GPBRs and set to G3 IBs.
4. Back to step 1.

Flash Storage Options

The provided storage implementation example has different options to choose the target Flash memory where the G3 data will be stored.

This option is set on the provided examples on the *conf_hal.h* file, where one of the following options has to be defined:

- `CONF_STORAGE_INTERNAL_FLASH` – Uses microcontroller internal Flash to store data

- *CONF_STORAGE_USER_SIGNATURE* – Uses a microcontroller special Flash area called User Signature to store data
- *CONF_STORAGE_AT45DBX_FLASH* – Uses an external Flash chip mounted on some Microchip EKs to store data

The Flash memory area is read and erased at start-up. The erase operation is done at start-up in order to save time on Power-Down, when time is critical and, thus, only the write operation is performed there.

The time for the writing operation in the different Flash options is shown in the table below.

Config Option	Writing Time (µs)
<i>CONF_STORAGE_INTERNAL_FLASH</i>	150
<i>CONF_STORAGE_USER_SIGNATURE</i>	270
<i>CONF_STORAGE_AT45DBX_FLASH</i>	50

3.5 Firmware Update Process

G3-PLC specification does not define a standard firmware update process. The stack is upgradeable using application-based mechanisms.

The user application is responsible for performing the memory management and steps for the firmware update process.

4. Operating System Support

4.1 Overview

User applications can run over an operating system, which means that the G3-PLC stack implementation needs to support that.

The OSS intends to transform the Microcontroller mode operation into a task-mode operation typical of operating systems. In order to do that, it creates and manages a single task where all active layers and interfaces are included. The user does not need to take care of controlling how the G3-PLC stack is running and can create their applications normally. The current implementation of the OSS is based on FreeRTOS but the user can modify it appropriately to use any other RTOS.

4.2 G3-PLC Process. Requirements, Handling and Usage of the G3-PLC FW Stack

When RTOS is used, the user has to take into account the requirements of different tasks running, to configure parameters such as the stack for each one of the tasks.

The given implementation takes into account the needed resources for the provided example, but any modification may have a direct impact on these needs. Therefore, the FreeRTOS configuration file may have to be modified.



CAUTION Take into account that the provided RTOS support is just an example of integration of the stack in an RTOS. It is functional, but Microchip does not ensure the same reliability and performance as the Microcontroller mode implementation, as the latter is the reference implementation.

5. Example Applications



Note that all provided application examples were configured to work on our evaluation boards. Every firmware project must define a Board Support Package (BSP) according to the hardware to be used.

Along with the G3-PLC FW stack, some application examples are provided to show how to use the stack.

Examples can be divided into two groups:

- ADP examples, using the whole stack (ADP + MAC + PHY), to evaluate Microchip's G3 solution as a whole
- PLC PHY examples, using only the PLC PHY layer, to evaluate the physical layer features, or to be used connected to a PC tool for demonstration purposes. For PL360/PL460, MAC RT examples are also provided for the same purpose.

Example applications are provided for CENELEC-A, CENELEC-B and FCC working bands on PLC. A suffix in the project name indicates the band it is for:

- PL360/PL460 supports CENELEC-A, CENELEC-B and FCC bands by using different binary files, one for each band.
 - Suffix “_cen_a” indicates that the CENELEC-A binary is linked in the application and loaded to PL360/PL460 at start-up.
 - Suffix “_cen_b” indicates that the CENELEC-B binary is linked in the application and loaded to PL360/PL460 at start-up.
 - Suffix “_fcc” indicates that the FCC binary is linked in the application and loaded to PL360/PL460 at start-up.
 - Projects for the PL360G55CB board only support CENELEC-B and the behavior is the same as in “_cen_b” ones.
 - Projects using PL460 can support 2 bands at the same time (FCC and CENELEC-A or FCC and CENELEC-B). In these projects, both binaries are linked in the application and the one loaded to PL460 is decided at run time, depending on the Host Controller. The one loaded by default is determined by the band definition in the *conf_project.h* file. These projects include the suffix _fcc_cen_a or _fcc_cen_b respectively. A particular case of this behaviour is the PL360G55CF board, which supports CENELEC-A and FCC bands, and works in the same way as explained (dynamic selection and loading of binary file).
- Projects supporting the G3 Hybrid Profile add the “_hybrid” suffix at the end of their name.

The band configuration on band-specific projects is already pre-defined in the provided *conf_project.h* file, and changing it will cause malfunctions. On the other hand, on multiband projects, a default band is pre-defined in the provided *conf_project.h* file, but it is possible to change it to the other band if desired.



Attention: As stated above, provided examples are configured to take full advantage of Evaluation Board capabilities, ignoring aspects such as power consumption that a final implementation will take care of. For an example of an alternative clock configuration for low requirements, see [5.4. Clock Configuration Example for Low Requirements Module](#).

5.1 ADP Examples

5.1.1 DLMS App

The DLMS App is an example that shows how the G3 API along with an IPv6 stack can be used in a typical DLMS application. This application is provided for device and coordinator, implementing a DLMS server and client respectively. The coordinator performs a continuous data collection that generates data traffic in the G3 network. Besides DLMS traffic, the application can be configured to generate ICMPv6 traffic through PINGs to other nodes.

Coordinator

Application header file *app_dlms_coord.h* contains some configuration constants for the coordinator which depend on the number of devices connected and the number of hops required to reach the coordinator if complex setups are used. These setup-dependent constants, which can be changed by the user, are:

- *DLMS_APP_ENABLE_PATH_REQ*. Enables the path request to all devices from the coordinator. If enabled, a path request round will be executed before each cycle. This is not defined by default. If defined, the constants *TIME_MAX_WAIT_PREQ_CFM* and *TIME_MAX_BETWEEN_PREQ_WITHOUT_DATA* set the timeout to wait for a Confirm and a Response to the Path Request respectively.
Note: As the structure to hold the path request is quite memory consuming, when this definition is enabled, the number of supported devices may have to be lowered in order to fit in RAM memory.
- *DLMS_TIME_WAIT_RESPONSE*. Time in milliseconds to consider a data timeout when no response is received from a device and continue with the next data request. Depending on the setup, and especially when hops and routes are used, the round trip time of a data request-response may be high. The default value is 50 seconds. The Route Repair process can be triggered if communication fails at some point, and the Timeout for such process is defined by the G3 Spec to 40 seconds by default, so the application timeout is chosen to take this into account.
- *DLMS_APP_WAIT_REG_NODES*. When defined, whenever a bootstrap or routing activity is detected in the network, the timer that waits for *TIMER_WAITING_START* is reloaded, so cycles would only start after the *TIMER_WAITING_START* time without bootstrap or routing traffic. This avoids needing to know beforehand the time it takes to build the network so as to wait before cycles start. Defined by default.
- *DLMS_TIME_WAITING_IDLE*. Time in milliseconds to wait until the application starts sending data requests to nodes present in the network. Depending on *DLMS_APP_WAIT_REG_NODES* constant (explained above), this time can be an absolute time starting when the coordinator is powered-up, or can be dynamically reloaded if network activity is detected. When this timer expires, DLMS traffic starts, taking into account only the devices present in the network at that moment. Therefore, if the *DLMS_APP_WAIT_REG_NODES* is not defined, the *DLMS_TIME_WAITING_IDLE* time has to be long enough to let all devices perform their bootstrap process correctly. Bootstrap process time per device is deeply dependent on the number of devices joining the network (the more devices, the more collisions and thus the more delay until next try), and also on the number of hops needed to perform the join process (hops increase the time to reach coordinator and get a response). A time of one minute per device may fit almost any setup (it is more than enough in simple setups, but just to give a general rule). Default value is 120 seconds.
- *DLMS_TIME_BETWEEN_CYCLES*. Sets the waiting time between one DLMS Cycle and the Next.
- *TIMER_BETWEEN_MESSAGES*. Sets the waiting time between different messages inside a DLMS Cycle.

The file *app_ping_coord.h* contains the constant *DLMS_APP_ENABLE_PING_CYCLE*, which enables sending ping frames to all devices before each cycle. Not defined by default.

Device

The file *app_ping_dev.h* contains the constant *DLMS_APP_ENABLE_PING_ALARM*, which enables sending ping frames periodically from the devices, using the interval definitions contained in this same file. Not defined by default.

Common

Both Device and Coordinator applications can configure the G3 stack and IPv6 parameters in order to be able to run G3 Conformance and Performance tests. On the provided projects, there is a compilation constant *APP_CONFORMANCE_TEST* available on the *g3_app_config.h* file which enables this configuration at start-up, but the final user can configure this behaviour at run time if preferred, modifying the application code by calling configuration functions upon a particular trigger.

5.1.2 Meter/Modem Apps

The Meter and Modem concept and background

The objective of the Meter and Modem applications is to demonstrate how to implement, in 2 separated devices communicating through a Serial Port, the DLMS and Metrology objects on one side, and the PLC Communications on the other. This is typically the distribution when there is an already functional Meter, to which a communications module is connected, whatever the media used for such communications.

Serial Communication Protocol between the Meter and Modem

The Meter and Modem applications are provided with 2 different communication protocols between them:

- Simple Management, also called “Transparent Mode”
- DLMS Management, which implements an HDLC protocol over the Serial Line

Any of these 2 options is enabled by defining one of the following Compilation Constants in the *g3_app_config.h* file:

```
#define SIMPLE_MGMT
#define DLMS_MGMT
```

Only one of them must be defined. DLMS management is the option enabled by default in the given example projects.

The Meter and Modem Roles and Exchanged Information

Regardless of the protocol used between the Meter and Modem, the information exchanged between devices is the same.

The Meter application is Reactive and does nothing until a request is received from the Modem.

The Modem is the Active part, and it is responsible for sending requests to the Meter (either requests from the Modem itself or coming from a remote node through an available communication medium) and sending the Response back through the medium where the request was received.

The G3 stack needs an EUI64 to be used as an Extended Address before Joining a Network, and this is the first task of the Modem, to request the Meter for a 64-bit string called “MeterID”, which will be used for that purpose.

After joining the Network using the MeterID, the Modem will wait for requests from a remote node, transfer those requests to the Meter using the selected protocol through the Serial Line, and send back the response obtained from the Meter to the remote node that originated the request.

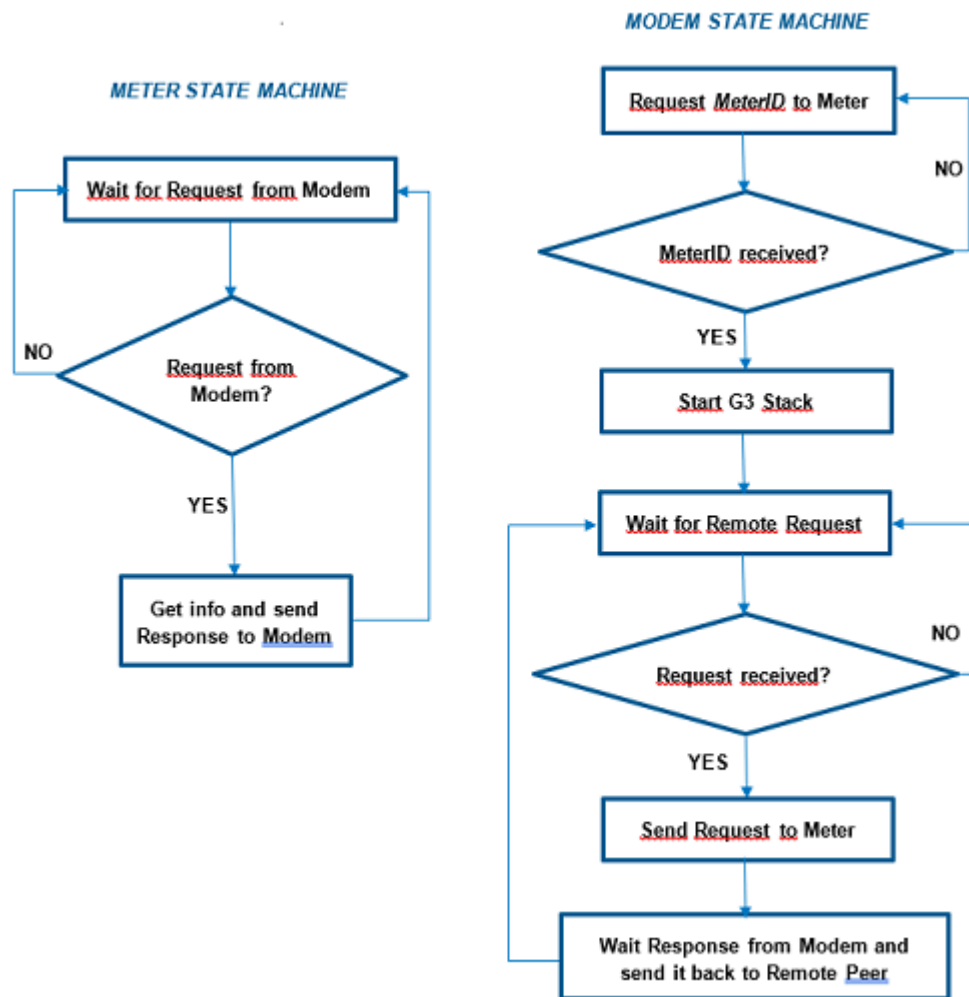
The given example uses DLMS as Application Layer on the Meter, so DLMS requests will be sent from the Modem to the Meter, and DLMS responses are what the Meter will send to the Modem.

For more information on how data is exchanged between devices, step through the provided example applications.

State Machines on the Meter and Modem

The State Machines running on the Meter and Modem devices can be depicted as follows:

Figure 5-1. Meter and Modem State Machines



5.1.3 ADP & MAC Serialization App

The ADP and MAC serialization is an application example that brings access to the ADP, MAC and Coordinator API through a serial connection. This application is useful for users who want to make intensive test of the stack or want to run the upper layers in another CPU.

The application allows control of the G3 PLC Stack at different levels. The user can make use of the ADP API (standard access) or access directly MAC Wrapper API as a shortcut for some tests. Serialization is also available to the provided Coordinator module in case the user wants to control the Bootstrap phase on the Coordinator side.

Serialization is done by means of [13. Serialization with Embedded USI](#).

5.2 PLC PHY Examples

5.2.1 PHY Tester

The PHY Tester tool is an application example that allows checking of the complete performance of the Microchip G3 PHY Layer on PLC boards. This example requires a board and a PC tool (PLC PHY Tester tool), which has to be installed in the user's host PC to interface with the boards.

The PLC PHY Tester tool controls the devices by means of a USI serialization. Users can perform PHY communication tests to check the capabilities of the PLC transceiver.

The example provided offers the serial interface configured through a certain UART or USB (depending on platform) at 230400 bps by default; the user can change these settings by means of [3.3.17. Universal Serial Interface Configuration – conf_usi.h](#).

5.2.2 PHY Sniffer

The PHY Sniffer is an application example that uses the PHY layer to monitor PLC frames in the G3 network and send them via USI serialization. The PLC Sniffer PC tool is also provided to interface with the board and interpret the G3 frames. This example requires only one board, and a G3 network to be monitored.

The example provided offers the serial interface configured through a certain UART or USB (depending on platform) at 230400 bps by default; the user can change these settings by means of [3.3.17. Universal Serial Interface Configuration – conf_usi.h](#).

5.2.3 TX Console

Because of the timing restrictions in the connection with the PC, the PLC PHY Tester tool may present limitations in applications or tests that require a very short time interval between consecutive frame transmissions.

The PHY Tx Test Console is an application example that demonstrates the complete performance of the Microchip G3 PHY Layer avoiding timing limitations in the PC host. That way, users can perform more specific PHY tests (e.g., short time interval between consecutive frames).

The PHY Tx Test Console application offers an interface to the user by means of a command console. In this console, users can configure several transmission parameters, such as modulation, frame data length and time interval between frames.

The example provided uses the console UART as a serial interface, configured through a certain UART or USB (depending on platform) at 921600 bps by default; the user can change these settings by means of [3.3.3. Board Configuration – conf_board.h](#) and [3.3.15. Serial USART Service Configuration – conf_uart_serial.h](#).

5.2.4 PHY PLC & Go

This example is intended to show a simple application running on top of the G3 PHY layer.

It presents a point-to-multipoint chat application on a Serial Console. When a message is written and sent from a device, all other devices receive it and print it on the Console.

The PLC & Go Application Note is available for download on the Microchip website and describes this example in more detail.

5.2.5 MAC RT PLC & Go Plus

This example is intended to show a simple application running on top of the MAC RT layer when it is running on PL360/PL460.

It presents a point-to-multipoint chat application on a Serial Console. When a message is written and sent from a device, all other devices receive it and print it on the Console.

The PLC & Go Plus Application Note is available for download on the Microchip website and describes this example in more detail.

CAUTION

Notice: This is not exactly a PHY application, and thus is not located under the '/thirdparty/g3/phy/atpl360/apps', but under '/thirdparty/g3/apps' instead.

5.3 RF PHY Examples

5.3.1 Hybrid PLC & RF PHY Tester

The Hybrid PLC & RF PHY Tester tool is an application example that allows checking of the complete performance of both Microchip G3 PLC & RF PHY Layers. This example requires a board and a host PC to interface with, where the user will run test scripts on top of a serialization library provided by Microchip.

Test scripts control the devices by means of a USI serialization. Users can perform PHY communication tests to check the capabilities of the selected transceiver, PLC or RF.

The example provided offers the serial interface configured through a certain UART or USB (depending on platform) at 230400 bps by default; the user can change these settings by means of [3.3.17. Universal Serial Interface Configuration – conf_usi.h](#).

5.3.2 Hybrid PLC & RF Sniffer

The Hybrid PLC & RF Sniffer is an application example that uses both RF and PLC PHY layers to monitor frames in a G3 Hybrid network and send them via USI serialization. The Microchip PLC Sniffer Tool, from version 2.0.3, is able to decode both PLC and RF frames, and is also provided with the example. This example requires only one board, and a G3 network to be monitored.

In case the board used has only one of the interfaces available, the example will run as a single medium sniffer, showing only PLC or RF frames. The same applies if it is used on a legacy PLC Network instead of a Hybrid Network, where only PLC frames will be shown.

The example provided offers the serial interface configured through a certain UART or USB (depending on platform) at 230400 bps by default; the user can change these settings by means of [3.3.17. Universal Serial Interface Configuration – conf_usi.h](#).

5.4 Clock Configuration Example for Low Requirements Module

As stated in chapters above, provided examples are configured to take full advantage of Evaluation Boards capabilities, ignoring aspects such as power consumption.

Microchip Kits mount external oscillators to provide CLK signal to their microcontrollers. By means of PLLs, the work frequency of the microcontroller is set to its maximum (or close to it) in provided examples, to get the highest performance as possible. But this is not a requirement of the G3 stack itself.

This chapter provides the changes in configuration files needed to be able to work at a lower frequency, using one of the internal RCs available. This leads to a decrease in Power Consumption and the possibility to remove the external crystal oscillator, both desired aspects in case a simpler solution is the target. For illustration purposes, a SAMG55-based platform is chosen to present the changes in configuration files.

Changes in conf_clock.h File

By default, projects are configured to use an external oscillator and a PLL to raise working frequency. In addition, a second PLL is used to generate the USB clock:

```
#define CONFIG_SYSCLK_SOURCE      SYSCLK_SRC_PLLACK
#define CONFIG_SYSCLK_PRESCALER  SYSCLK_PRESC_1

/* Configure PLL0 to get MCK at 96 MHz */
```

```
#define CONFIG_PLL0_SOURCE    PLL_SRC_SLCK_XTAL
#define CONFIG_PLL0_MUL      2930
#define CONFIG_PLL0_DIV      1

/* Configure PLL1 to get USB CLK at 48 MHz */
#define CONFIG_PLL1_SOURCE    PLL_SRC_SLCK_XTAL
#define CONFIG_PLL1_MUL      1465
#define CONFIG_PLL1_DIV      1

#define CONFIG_USBCLK_SOURCE   USBCLK_SRC_PLL1
#define CONFIG_USBCLK_DIV     1
```

The alternative configuration to use the internal 24 MHz RC is the following:

```
#define CONFIG_SYSCLK_SOURCE    SYSCLK_SRC_MAINCK_24M_RC
#define CONFIG_SYSCLK_PRE     SYSCLK_PRE_1
```

No PLLs are used, as 24 MHz internal RC does not support connection to PLL, it is used directly as MCK; and USB CLK is not configured, as it requires 48 MHz to work properly.

Changes in `conf_pplc_if.h` file

Reducing MCK to 24 MHz has an impact on default SPI speed configuration. SPI uses DMA in the provided examples, and this limits the speed at which the SPI clock can be configured, related to MCK.

Default SPI speed is configured at 8 MHz in provided examples:

```
#define PPLC_CLOCK            8000000
```

But this has to be reduced when MCK is set to 24 MHz, so this definition has to be changed to:

```
#define PPLC_CLOCK            4000000
```

Working at 4 MHz on SPI will make transactions take more time, but correct communication is ensured at this rate.

With this Clock and SPI Configuration, we have a fully functional G3 Stack running at 24 MHz, SPI transactions at 4 MHz, and without USB support, which is a perfectly valid approach for a PLC module with no extra requirements.

6. Microchip Implementation Particulars

This section describes some particularities of the Microchip G3 implementation, mainly referring to Physical Layer capabilities and features which make the difference between this product and others.

6.1 Table Sizes and Buffer Count Configuration

To improve G3 stack configuration capabilities and for user convenience, some G3 stack tables and buffer counts can be modified and set at compilation time. To do so, they are declared in auxiliary source files.

The configuration of these parameters depends on both user needs and on memory availability in the target platform.

These values can be set in the *conf_tables.h* file, present in all provided example applications.

The table sizes and buffer count configured in this file are the following:

Number of ADP Transmission Buffers

ADP defines a structure of buffers for transmission of three different sizes: 1280 bytes, 400 bytes and 100 bytes.

The reason for these buffer sizes are the following:

- 1280 bytes is the maximum size of an IPv6 packet coming from the application layer.
- 400 bytes is the maximum size that G3 MAC layer accepts. Typically used for short application IPv6 frames.
- 100 bytes is enough for most frames coming from inside the G3 stack, such as LOADng or LBP frames.

These buffer sizes are fixed; however, the number of buffers for each size is configurable through definitions. Default configuration is the following, depending on the Node type:

Coordinator

```
#define CONF_COUNT_ADP_BUFFERS_1280    1
#define CONF_COUNT_ADP_BUFFERS_400     8
#define CONF_COUNT_ADP_BUFFERS_100     8
```

Device

```
#define CONF_COUNT_ADP_BUFFERS_1280    1
#define CONF_COUNT_ADP_BUFFERS_400     3
#define CONF_COUNT_ADP_BUFFERS_100     3
```

Number of Simultaneous Fragmented Receptions

When receiving a fragmented frame, the ADP layer tracks fragments in a table for further reassembly. This table size, thus, defines the number of simultaneous fragmented frames that can be processed. Default configuration is the following, depending on the Node type:

Coordinator

```
#define CONF_ADP_FRAGMENTED_TRANSFER_TABLE_SIZE  3
```

Device

```
#define CONF_ADP_FRAGMENTED_TRANSFER_TABLE_SIZE  1
```

LOADng Routing Tables

The Routing Table and Routing Set functionality is well-defined in the G3 specification (and beyond the scope of this guide), but not the size that these tables have to have. In addition, the number of simultaneous Route Requests where a Route Reply has to be issued can also be configured. Default configuration is the following, depending on the Node type:

Coordinator

```
#define CONF_ADP_ROUTING_TABLE_SIZE      2000
#define CONF_ADP_ROUTING_SET_SIZE        400
#define CONF_LOADNG_RREP_GENERATION_TABLE_SIZE  10
```

Device

```
#define CONF_ADP_ROUTING_TABLE_SIZE      150
#define CONF_ADP_ROUTING_SET_SIZE        30
#define CONF_LOADNG_RREP_GENERATION_TABLE_SIZE  3
```

MAC Layer Tables

RF POS Table functionality is well-defined in the G3 specification (and beyond scope of this guide), but not its size. In addition, the Data Sequence Number Table, used to avoid processing duplicate frames, can also be configured in size. Default configuration is the following, depending on the Node type:

Coordinator

```
#define CONF_MAC_POS_TABLE_ENTRIES_RF      (500)
#define CONF_MAC_DSN_TABLE_ENTRIES_RF      (16)
```

Device

```
#define CONF_MAC_POS_TABLE_ENTRIES_RF      (100)
#define CONF_MAC_DSN_TABLE_ENTRIES_RF      (8)
```

IPv6 Stack Buffer Count

Although outside of the G3 stack scope, the number of buffers used in the IPv6 stack can also be configured here for unification purposes. Default configuration is the following, depending on the Node type:

Coordinator

```
#define CONF_NET_MEM_POOL_BUFFER_COUNT      16
```

Device

```
#define CONF_NET_MEM_POOL_BUFFER_COUNT      4
```

Nodes Supported at DLMS Level

Also outside of the G3 stack scope, the number of devices supported by a DLMS client can also be configured here. On Coordinator applications, such a DLMS Client is provided, so DLMS queries can be sent to Devices to simulate a real Data traffic in a typical network. The number of DLMS Servers that can be supported by the provided client is configurable:

```
#define CONF_DLMS_MAX_DEV_NUM              250
```

Routing and Neighbour Table Snapshots at DLMS Level

Also outside of the G3 stack scope, some tables have a mirrored snapshot on the provided DLMS server, which can also be configured here. Devices have a snapshot of their Routing and Neighbour Tables to provide information upon certain DLMS queries. Default configuration is the following:

```
#define CONF_MAX_NEIGHBOUR_DLMS_SNAPSHOT_SIZE  75
#define CONF_MAX_ROUTING_DLMS_SNAPSHOT_SIZE    150
```

6.2 Impedance Detection, Transmission States and Equalization

6.2.1 What are Transmission States?

Transmission States are configurations applied to the PHY layer in order to obtain the desired signal level depending on the user needs.

Depending on impedance, transmission parameters change in order to obtain the better performance/efficiency from the output driver.

Three different states are defined for this purpose:

- 0 - HI_STATE. Transmission is configured to comply with normative with the best performance/efficiency assuming there is a high-impedance ($\text{imp} > 20 \text{ Ohm}$), either auto-detected or fixed by configuration.
- 1- LO_STATE. This is an intermediate state suitable for all impedance ranges when the required transmission level is low (controlled by upper layers).
- 2 - VLO_STATE. Transmission is configured to comply with normative with the best performance/efficiency assuming there is a very low-impedance ($\text{imp} < 10 \text{ Ohm}$), either auto-detected or fixed by configuration.

The user may force one of these states on the PHY layer or use the recommended operation mode, which is Automatic Impedance Detection and Adaptation.

6.2.2 Forced Transmission Modes

As stated before, transmission modes are optimized for an impedance range, and out of such range, they do not work properly in terms of performance/efficiency.

Let us suppose we have set a signal injection objective of 110 dBuV for both HI and VLO modes.

The following figures show the response of forced modes in an impedance range from 0 to 50 ohms.

Figure 6-1. Forced Transmission Modes vs. Impedance

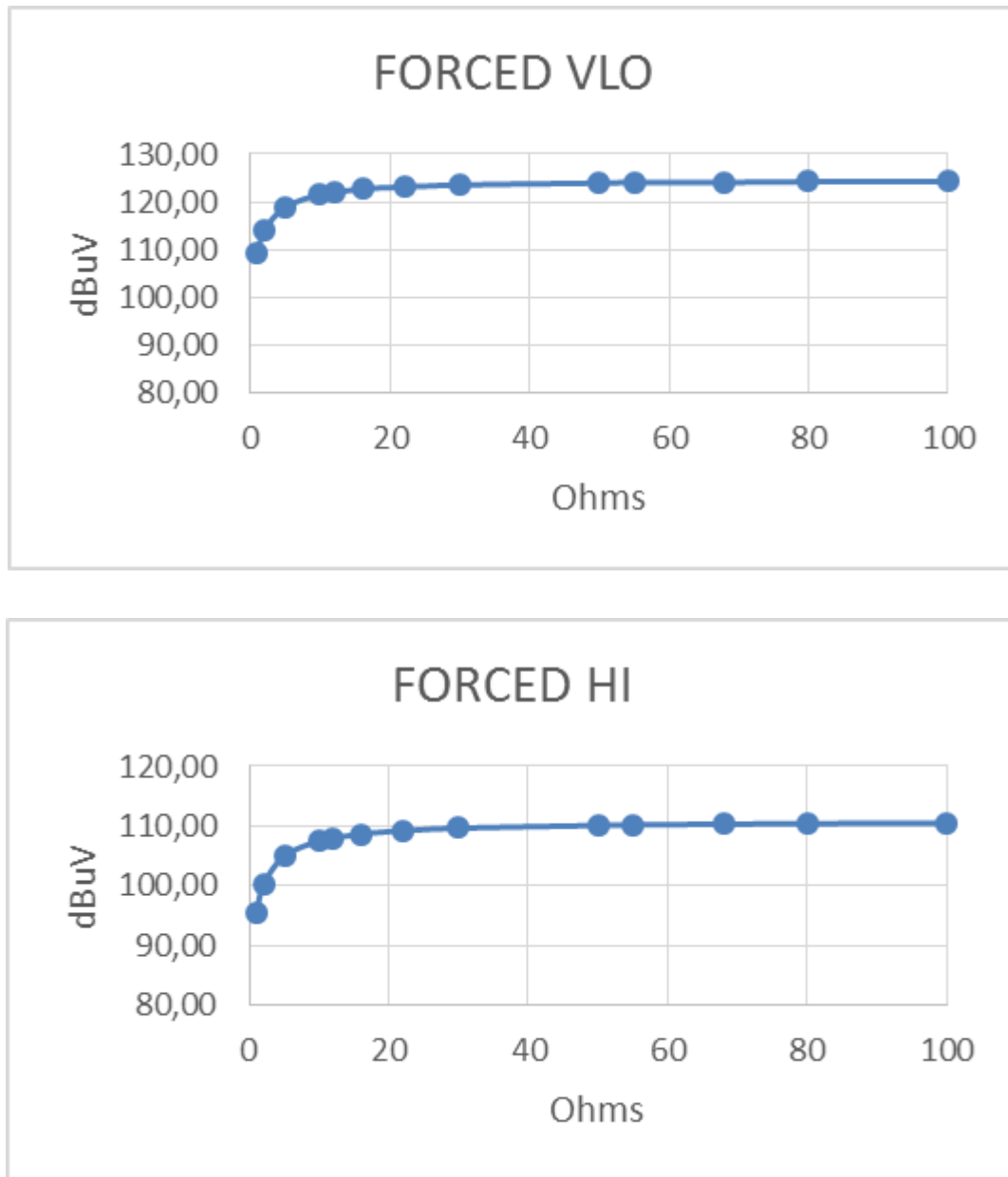


Figure shows that each mode is injecting the desired signal in its intended range, but out of such range, each mode is not achieving the injection objectives. Also can be seen that the response has a much more stable trend for impedance above 20 ohms, and below that, the transmission driver is more dependent on impedance detected.

Figures show the response when both State and Gain are Fixed. There is also a configuration where State is fixed, but slight changes in Gain are allowed (see [7.1.21. PHY_ID_CFG_AUTODETECT_IMPEDANCE \(0x0161\)](#) for details), but such slight changes will not be enough to meet injection objective.

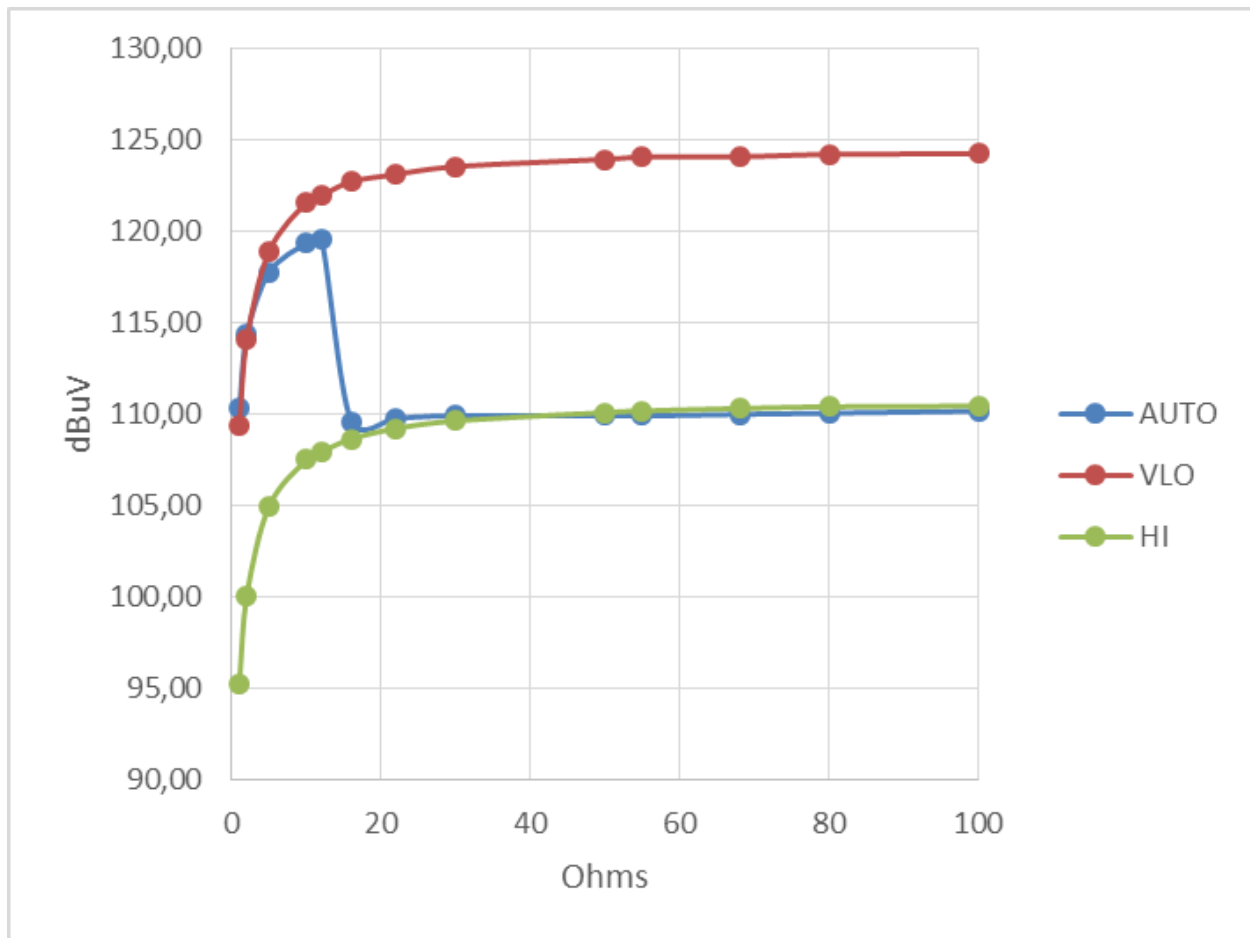
To meet objective in all impedance range (with some limitations in low impedance as explained above), we have to rely on Impedance Detection and Adaptation algorithm implemented in the Physical layer.

6.2.3 Impedance Detection and Adaptation Algorithm

The Physical layer implements an algorithm that monitors the output signal level and infers the impedance value against what it is transmitting to. Depending on the inferred impedance, the Physical layer sets the proper transmission state to obtain the desired output level. Enabled or not, this detection and adaptation is controlled through the PIB [PHY_PARAM_CFG_AUTODETECT_BRANCH](#) (see PIB for details about auto-detect configuration).

The following figure shows the response in AUTO_STATE_VAR_GAIN (1) mode, compared to the forced modes described before:

Figure 6-2. Impedance Adaptation vs. Forced Modes



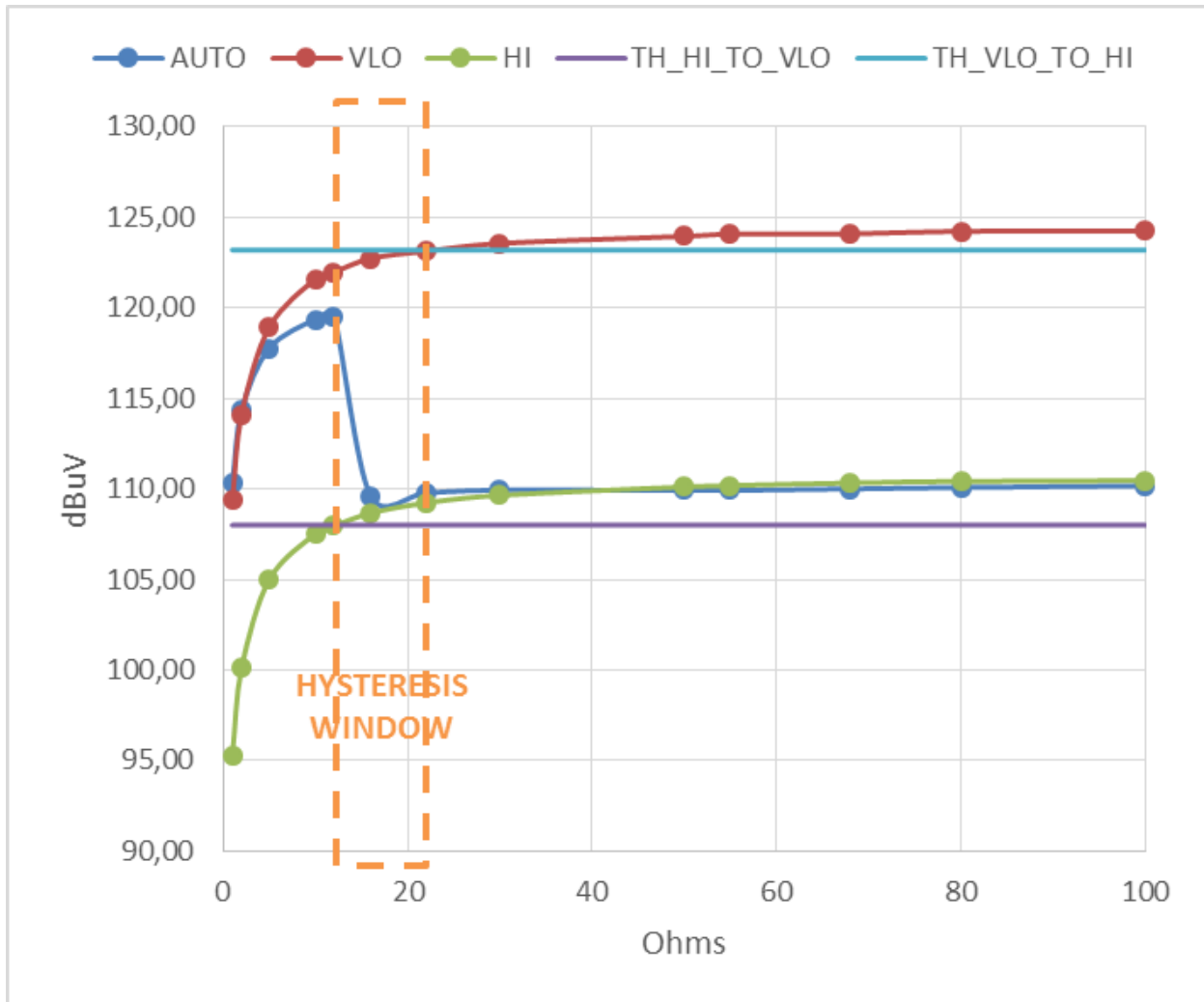
This adaptation is achieved by defining thresholds to switch from HI to VLO mode, and vice versa, which determine the points where transmission state is changed.

These thresholds are defined such that the switch from HI to VLO is done in a lower impedance than the switch from VLO to HI, obtaining a hysteresis window to avoid continuous switching between states which will lead to a continuous change in signal injection, which is an undesirable scenario.

The usage of LO mode is reserved to transmission with a forced attenuation, as explained later in this section.

The following figure illustrates such thresholds and the hysteresis window they achieve:

Figure 6-3. Thresholds and Hysteresis Window



As shown in the figure, the Auto response was obtained by sweeping from high to low impedance; this is why the switch is done near the lower value of the hysteresis window (some point between 10 and 15 ohms). If the sweep is done in the opposite way, from low to high impedance, the transition will be done near the higher value of the hysteresis window. Therefore, the curve will be slightly different inside such window. It will follow the VLO curve inside the window, and, then, switch to the HI curve just above 20 ohms.

It also shows that the Auto curve does not follow the exact trend of the forced ones. This is because the algorithm performs a fine tuning in each transmission according to the detected signal value, trying to adjust more closely to the signal injection objectives.

In this algorithm, the LO state is reserved to be used when the upper layers set an attenuation value (which in G3 is determined by the Neighbour Table [8.7.25. MAC_WRP_PIB_NEIGHBOUR_TABLE \(0x0000010A\)](#) information in TxGain field) greater than a certain value (determined by the transceiver used), where the algorithm does not have enough resolution to properly determine the switching between states. Thus, it uses the LO state, which is a static state, whose configuration is aimed to this high attenuation circumstance, and considered valid in all the impedance range. As soon as the upper layer requests a transmission with less attenuation, the algorithm will be back to the mode switching between HI and VLO depending on the inferred impedance.

6.2.4 Equalization

Usually the output transmission driver has a non-flat response inside the working band (ripple). Depending on the difference between carriers, we may fall in a response out of the specification which has to be corrected to meet constraints.

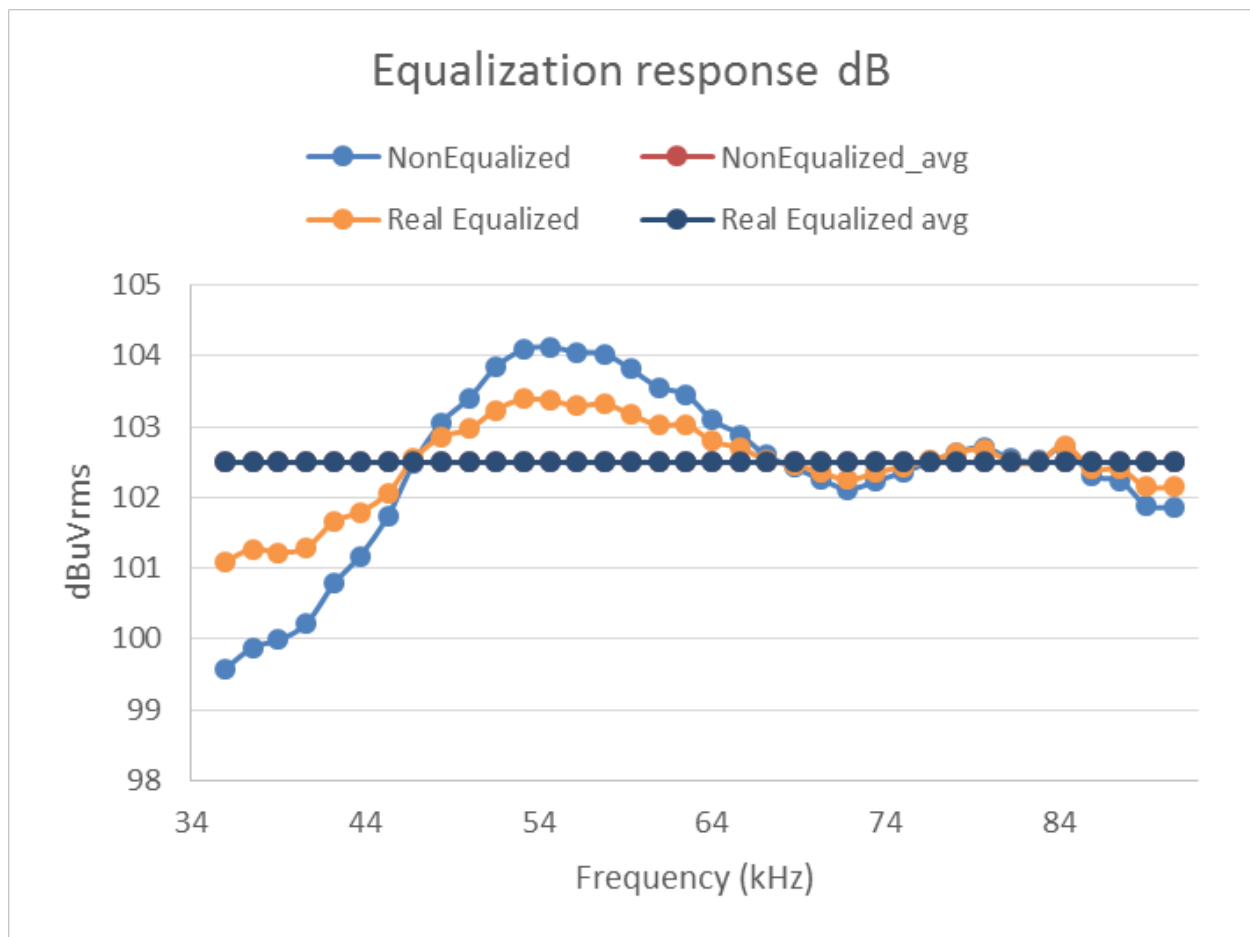
The Physical layer is able to perform this equalization by means of signal pre-distortion, which compensates the effect of the external driver, reducing the ripple, to give a final response closer to a flat one.

Equalization has a tight relation with transmission states seen in previous sections; in fact, each transmission state defines a particular equalization. As configurations are different for each state, the response is also different, which requires different equalization.

An important aspect to take into account is that the objective of the equalization is to reduce ripple, giving a flatter response, but keeping the average signal injection. Therefore, care must be taken in order to reduce signal in some carriers and increase it in others, resulting in the same response after integrating the signal level inside the operation band.

The following figure shows an example of equalization in the G3 Cenelec-A band:

Figure 6-4. Equalization Example



The figure shows how the ripple is reduced after equalization, keeping the same signal average in band (overlapped curves).

6.2.5 Calibration of Signal Level and Equalization

In the provided Physical layer, the transmission level and equalization parameters are configured to work properly with the provided HW (EKs).

The final user may need to calibrate all these parameters when working on their final HW. This is not a trivial step, so Microchip provides a tool which will perform the necessary tests and obtain the values that the user will overwrite in the provided Physical layer in order to meet expectations, after requesting that the user introduce the desired targets in terms of signal level and ripple.

The process to follow and the tool usage not within the scope of this guide, and will be provided in a separate document.

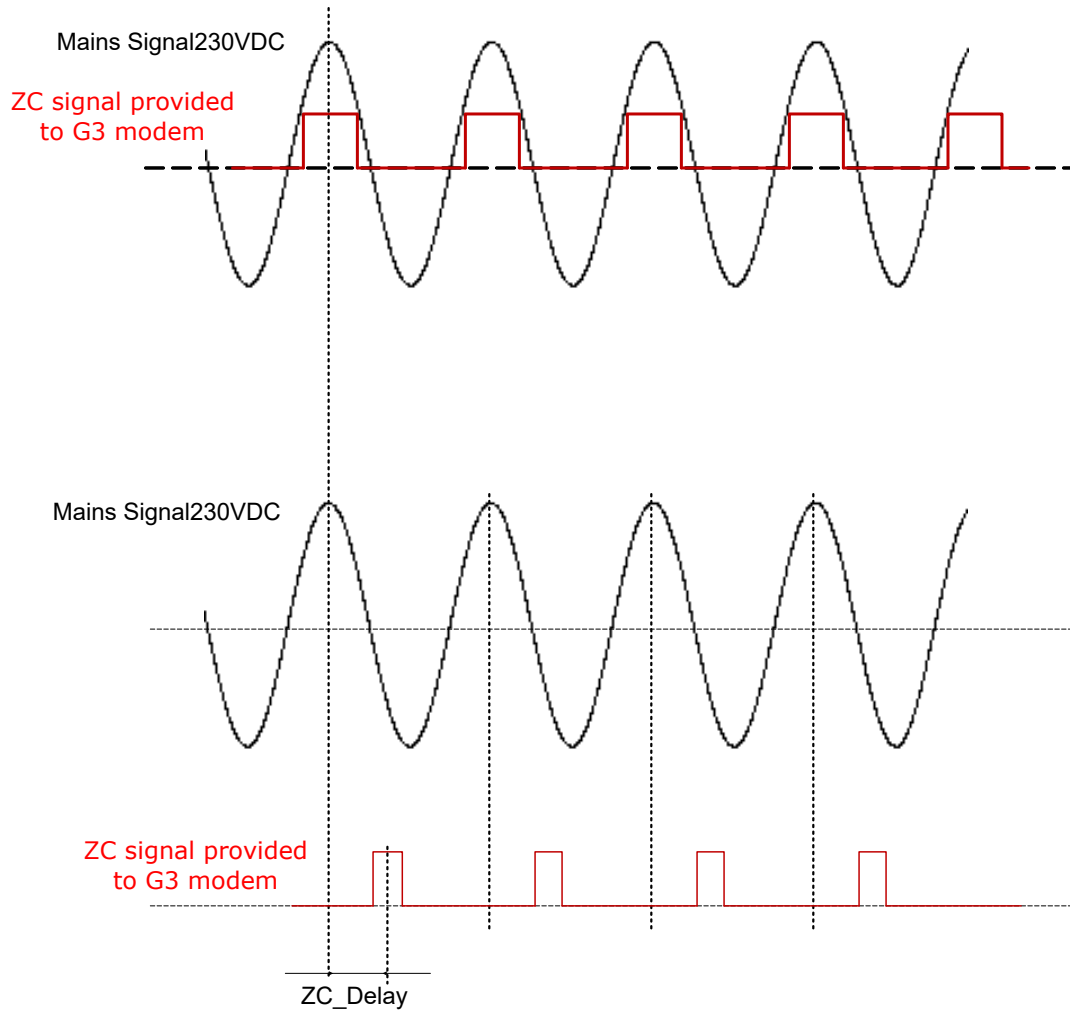
6.3 ZC Offset Configuration

The PHY layer calculates the electrical phase difference between transmitter and receiver for a given frame. To do so, both transmitter and receiver have to be able to read the Zero-Cross time of the mains.

PL360/PL460 have a dedicated input pin for connection to a signal which provides such mains Zero-Cross. Due to design reasons, the signal on the input pin may not be fully synchronized with the real mains Zero-Cross.

The following figure shows two different cases of the ZC signal provided to the modem.

- The first one shows a typical response of a Zener-based circuit to detect Zero-Cross, where the highest point of mains signal and the middle point of the pulse are aligned. In this case no configuration is needed, as there is no delay between these points.
- Below is an example where the provided ZC signal comes from an external device, which may have a delay in generating the pulse, and whose pulse width is not related to mains frequency. The figure shows the time that has to be measured to be set as ZC delay in the modem parameters, so further calculations refer to the real mains ZC.

Figure 6-5. ZC Delay Calculation

This is configured via PIB (ATPL360_REG_ZC_CONF_DELAY, ATPL360_REG_ZC_CONF_INV). Please refer to the PL360 Host Controller documentation for more details.

The default value of this parameter is configured according to Microchip EK boards requirements. The user can change it in its implementation if necessary.

6.4 Noise Detection and Compensation

Microchip G3 stack is able to detect and compensate narrow band and impulsive noises. As both detection and compensation are different depending on noise type, they will be explained in different sections.

6.4.1 Narrow Band Noise

Narrow band noise is continuously monitored in the line, at a given time rate with some exceptions affecting this period.

This functionality can be enabled/disabled through IB [PHY_PARAM_ENABLE_AUTO_NOISE_CAPTURE](#). It is enabled by default.

The periodicity is set through [PHY_PARAM_TIME_BETWEEN_NOISE_CAPTURES](#) IB. The default value is 1 second.

The compensation of narrow band noise is done through a notch filter that is configured in the same frequency as narrow band noise is detected, to reduce its amplitude in reception stage.

The information regarding whether a notch filter is configured and in which carrier can be retrieved by upper layers by means of [PHY_PARAM_RRC_NOTCH_ACTIVE](#) and [PHY_PARAM_RRC_NOTCH_INDEX](#) IBs.

The effect of narrow band noise in the MAC layer may be a sub-band cancellation in the next Tone Map Response, but this decision is not fixed. It will be evaluated by the Tone Map decision algorithm depending on SNR in the carriers near the narrow band noise. For more information about Tone Map Response functionality, please refer to G3 Specification.

6.4.2 Impulsive Noise

Impulsive noise is monitored during frame reception, where SNR of different symbols are compared to decide whether an impulsive noise is present. This functionality is always present and cannot be configured, as it has no drawbacks in frame reception itself.

The compensation of impulsive noise will be done by MAC layer in the following Tone Map Responses.

PL360/PL460 applies different techniques to improve robustness against impulsive noise that may not require the use of ROBO modulation, but a combination of modulation/tonemapping.

For more information about Tone Map Response functionality, refer to G3 Specification.

6.5 Power-Down Detection

Requirements

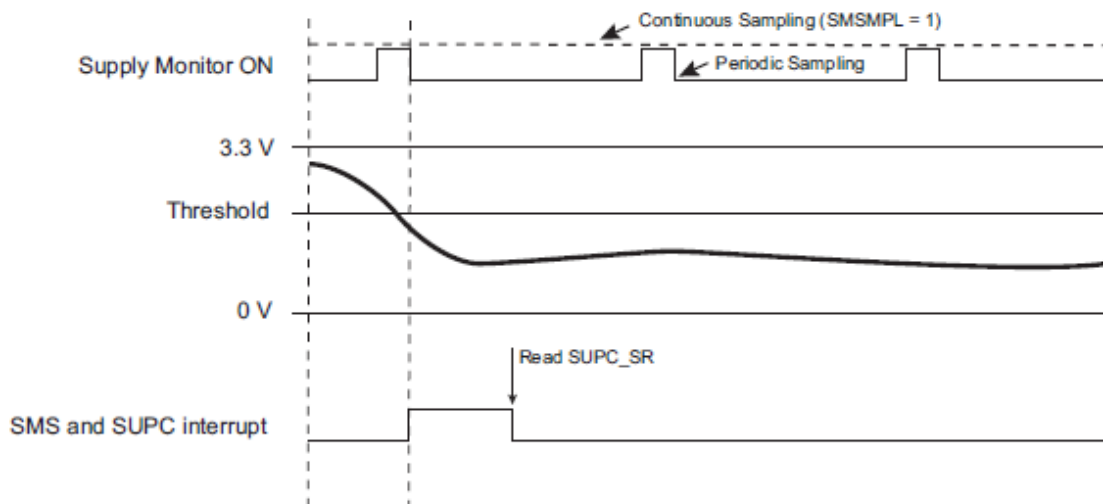
Applications using the G3 stack need to be aware of when a device is powered down, in order to store some information before power is completely lost.

This section explains the Power-Down detection approach used in the G3 examples, although other alternative methods are presented as well. Regarding the data storage, please refer to [3.4. Persistent Data Storage](#).

Approach 1. Supply Monitor.

The default configuration of the provided G3 examples makes use of the Supply Controller, a HW block present in all SAM devices, which can be configured to trigger an interrupt when a certain (configurable) threshold is reached.

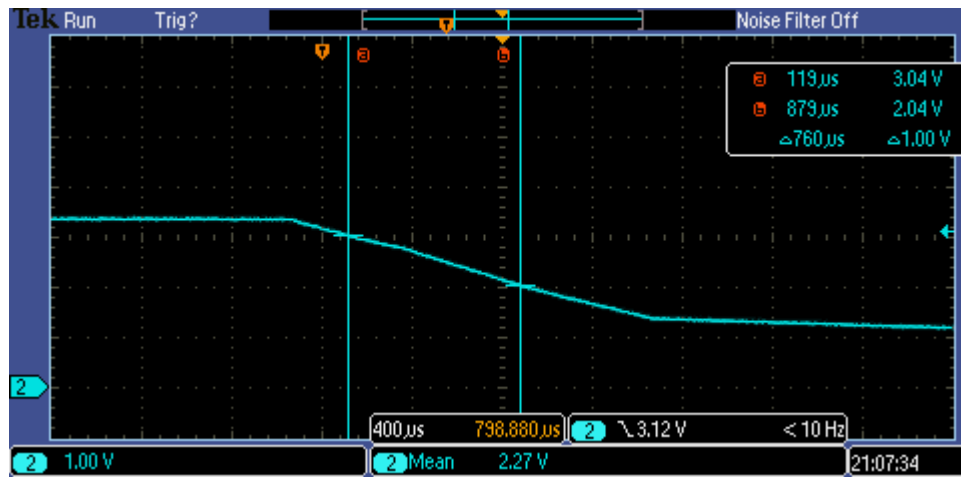
Figure 6-6. Supply Monitor



The default configuration in the examples is Continuous Sampling and Threshold = 3V. The response time of this approach (Continuous Sampling) is one cycle of SLCK (32kHz), which gives 31.25 μ s.

The supply monitor is the fastest solution in terms of response time, and the less CPU consuming option, as everything is managed by HW until interrupt is triggered. The drawback is that the time to perform the storage is short. The following capture shows the 3V3 falling slope measured in a PL360G55CB_EK supplied by USB, which is considered the worst case.

Figure 6-7. PL360G55CB EK 3V3 Fall



The capture shows that, from Power-Down detection on 3V, there are about 900 µs until voltage falls to 2V. The time required to store data in User Signature plus the worst case in detection time sum up to 300 µs.



This time measurements and constraints are based on Microchip EKs and the selected storage implementation. Any change in software or hardware has an impact and the validation of the approach is required.

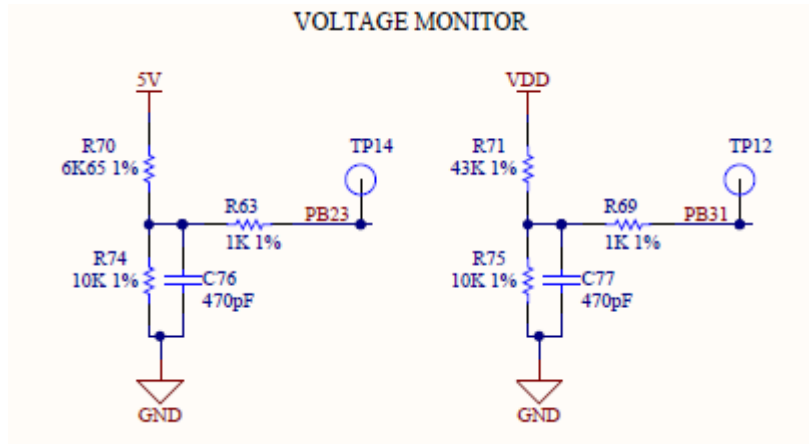
The conclusion is that this approach is OK for the provided examples in the available EKs, but all this information regarding times and voltages has to be taken into account when working on a new design, both HW and SW.

Approach 2. External Voltage Divider.

An alternative method, available in some Microchip PLC EKs, is to have a voltage divider connecting one of the power supplies of the board to a microcontroller GPIO with ADC capabilities.

Knowing the ADC value in the range of valid power supply voltages, a threshold is defined where it is estimated that the voltage has reached a value low enough to consider that the supply is about to fail (Power-Down).

In the provided implementation, the ADC is connected to a Timer Counter peripheral which periodically triggers ADC Conversions, which output value is compared to the threshold to detect whether power is falling down.

Figure 6-8. Voltage Divider

As seen in the picture, taken from ATPL250-EK documentation, 2 options are provided, connected to 5V and 12V (VDD) respectively. The provided implementation uses the VDD option to detect power fail.

The response time in this case depends on the rate at which the timer is configured to trigger conversions. The default rate is configured to a conversion every 10 ms, which is thus the response time. Although this may look like a poor response time, it may be fast enough in designs where VDD and 3V3 supplies are mounted in cascade (such as in ATPL250_EK) and the capacitors in VDD provide an extra time before the 3V3 supply is affected by the Power-Down.

Figure 6-9. ATPL250AMB 12V to 3V3 Fall

The picture shows the instant when the ADC output value is below the threshold (corresponding to 10V in VDD in the blue line), and the time when the 3V3 supply is affected (yellow line falling). Time between these events is about 55 ms, this is why a 10 ms response time is enough.

This option, having a worse response time, and being more CPU consuming (each ADC conversion triggers an interrupt where threshold is compared by SW) than approach 1, could still be the better option in cases where tasks to perform on Power-Down take more time, as we are in the range of milliseconds (instead of microseconds as in approach 1).

Alternative Approaches.

The provided examples offer the 2 approaches explained above, but the decision on how to detect the Power-Down is up to the final user, and different alternatives can be used.

It is possible that the design where G3 will be integrated has also a means of detecting Power-Down due to external needs. Then, the circuitry detecting the Power-Down can be connected to the microcontroller running G3 and configure an interrupt in such PIO which will then be in charge of the data storage.

Or even the SW solution of which G3 is a part has already a routine to perform some tasks at Power-Down, whatever the means of detection. Then the G3 Data Storage can be done along with such tasks, and no dedicated Power-Down detection will be necessary for G3.

Implementation and Configuration

The implementation of the presented approaches can be found in the HAL (Hardware Abstraction Layer) module in the provided G3 examples, as it is platform-dependent and implementations may differ depending on the microcontroller used, or the capabilities of the external circuitry.

The configuration of the method used and its corresponding parameters is done in the *conf_hal.h* file of the examples.

Any change that final user may test that is related to available approaches, or implementation of new ones, will be done in those files.

6.6 Voltage and Thermal Monitor for PLC Driver

On the PL460, being a device with an integrated PLC Tx Driver, it is important to ensure that the supply voltage level of the amplification stage is within the expected range.

To ensure this, the provided examples implement a voltage monitor using an analog input in the host micro-controller, which checks if the voltage is out of range, and, in case it is detected, uses an output pin connected to PL460 to indicate that transmission is not allowed. This is done by sampling the voltage and comparing it to determined thresholds (which depend on the circuitry used for the Monitor). When transmission is disabled by the pin, the PL460 aborts any transmission in progress or programmed, and new transmission requests are denied. This requires 2 PIOs in the Host microcontroller, one Analog Input to sample the voltage, and one Digital Output to indicate to the PL460 that transmission is not allowed.

The PL460 also implements a thermal monitor, which indicates through a Pin if the Die Temperature is above 110°C. The provided examples configure a Digital Input in the host microcontroller to detect this temperature limit and, thus, not request more transmissions to the PL460 until the temperature is back to safe levels. As stated, this requires an additional PIO in the microcontroller configured as a digital input to get the thermal warning from the PL460. If this pin is not used by the host microcontroller, the PL460 has another internal pin that indicates if the die temperature is above 120°C to disable transmissions in extreme temperature conditions.

For more details on the required PIOs and their configuration, refer to the [3.3.10. PPLC Interface Configuration – conf_pplc_if.h](#) file on any of the provided examples for the PL460 platform.

7. API of PHY Layer

PL360/PL460 does not require a PHY layer in the Host microcontroller, it is self-contained in PL360/PL460 device, and access to it is managed by the Host Controller component. For further information regarding this component, please refer to the PL360 Host Controller documentation.

However, as PHY PIB attributes can be accessed through a dedicated MAC layer IB, they are described here for reference.

7.1 PIB Objects Specification and Access

Following is a list of available PIBs with a brief description and possible values.

7.1.1 PHY_ID_INFO_PRODUCT (0x0100)

Product identifier. Depending on Device.

Access: Read only.

Value Range: 10-character string.

Default Value: String identifying the Device for which code is compiled.

7.1.2 PHY_ID_INFO_MODEL (0x010A)

Model identifier.

Access: Read Only.

Value Range: 0-65535.

Default Value: Depending on used model.

7.1.3 PHY_ID_INFO_VERSION (0x010C)

PHY layer version identifier.

Access: Read Only.

Value Range: 0-4294967295.

Default Value: Depending on used version.

7.1.4 PHY_ID_TX_TOTAL (0x0110)

Correctly transmitted frames counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.5 PHY_ID_TX_TOTAL_BYTES (0x0114)

Transmitted bytes counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.6 PHY_ID_TX_TOTAL_ERRORS (0x0118)

Total transmission errors counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.7 PHY_ID_BAD_BUSY_TX (0x011C)

Transmission failure due to already in transmission counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.8 PHY_ID_TX_BAD_BUSY_CHANNEL (0x0120)

Transmission failure due to busy channel counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.9 PHY_ID_TX_BAD_LEN (0x0124)

Transmission failure due to incorrect length (too short - too long) counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.10 PHY_ID_TX_BAD_FORMAT (0x0128)

Transmission failure due to wrong format (Tone-map not valid) counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.11 PHY_ID_TX_TIMEOUT (0x012C)

Timeout error during transmission counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.12 PHY_ID_RX_TOTAL (0x0130)

Total received frames counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.13 PHY_ID_RX_TOTAL_BYTES (0x0134)

Total received bytes counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.14 PHY_ID_RX_RS_ERRORS (0x0138)

Frames discarded by Reed Solomon block counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.15 PHY_ID_RX_EXCEPTIONS (0x013C)

Frames discarded due to decoding errors during reception counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.16 PHY_ID_RX_BAD_LEN (0x0140)

Frames discarded due to incorrect length (too short - too long) counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.17 PHY_ID_RX_BAD_CRC_FCH (0x0144)

Frames discarded due to incorrect CRC in header counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.18 PHY_ID_RX_FALSE_POSITIVE (0x0148)

Frames discarded due to incomplete preamble synchronization counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.19 PHY_ID_RX_BAD_FORMAT (0x014C)

Frames discarded due to unexpected values in header counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

7.1.20 PHY_ID_TIME_BETWEEN_NOISE_CAPTURES (0x0158)

Time between noise captures (in ms) to detect tonal noise.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 60000.

7.1.21 PHY_ID_CFG_AUTODETECT_IMPEDANCE (0x0161)

Autodetect impedance state, where possible values are:

- 0 - FIXED_STATE_FIXED_GAIN. Transmission state is fixed as impedance detection is not allowed. Transmission gain is kept constant.
- 1- AUTO_STATE_VAR_GAIN. Transmission state may change depending on impedance detected. This may lead to big changes in transmission gain in order to comply with normative.
- 2 - FIXED_STATE_VAR_GAIN. Transmission state is fixed but impedance detection is allowed to perform little transmission gain changes for better signal quality.

Access: R/W.

Value Range: 0-2.

Default Value: 1- AUTO_STATE_VAR_GAIN.

7.1.22 PHY_ID_CFG_IMPEDANCE (0x0162)

Transmission mode corresponding to impedance state, where possible values are:

- 0 - HI_STATE. Transmission is configured to comply with normative with the best performance/efficiency assuming there is a high-impedance ($imp > 20 \text{ Ohm}$), either auto-detected or fixed by configuration
- 1- LO_STATE. This is an intermediate state suitable for all impedance range when the required transmission level is low (controlled by upper layers)
- 2 - VLO_STATE. Transmission is configured to comply with normative with the best performance/efficiency assuming there is a very low-impedance ($imp < 10 \text{ Ohm}$), either auto-detected or fixed by configuration

Access: R/W.

Value Range: 0-2.

Default Value: 0 - HI_STATE.

7.1.23 PHY_ID_RRC_NOTCH_ACTIVE (0x0163)

Indicates whether a Notch filter is active, used to compensate effect of a tonal noise.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

7.1.24 PHY_ID_RRC_NOTCH_INDEX (0x0164)

Indicates the carrier index where the Notch filter is active (if any), 0 is set if no filter is active.

Access: R/W.

Value Range: 0, [23-58 Cen A], [33-104 FCC], [33-86 ARIB].

Default Value: 0.

7.1.25 PHY_ID_ENABLE_AUTO_NOISE_CAPTURE (0x0166)

Enables a periodic tonal noise detection so a Notch filter will be applied where noise is detected if needed. Period is set in [7.1.20. PHY_ID_TIME_BETWEEN_NOISE_CAPTURES \(0x0158\)](#) parameter.

Access: R/W.

Value Range: 0-1.

Default Value: 1.

7.1.26 PHY_ID_DELAY_NOISE_CAPTURE_AFTER_RX (0x0167)

If set, the next periodic tonal noise detection is canceled after a successful frame reception, and delayed a [7.1.20. PHY_ID_TIME_BETWEEN_NOISE_CAPTURES \(0x0158\)](#) period. The purpose is not to interfere with frame receptions when there is traffic in the network.

Access: R/W.

Value Range: 0-1.

Default Value: 1.

7.1.27 PHY_ID_PLC_DISABLE (0x016A)

Disables PLC Tx/Rx for device. Data indication primitives are not generated, and data confirms are issued after a data request without actually transmitting.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

7.1.28 PHY_ID_NOISE_PEAK_POWER (0x016B)

Returns detected noise peak power (in dBuV) on last noise capture, in case a peak was detected (PHY_PARAM_RRC_NOTCH_ACTIVE == 1). If no peak was detected, it returns the noise average in band measured on last noise capture.

Access: R/W.

Value Range: 0-255.

Default Value: 0.

7.1.29 PHY_ID_LAST_MSG_LQI (0x016C)

LQI value of the last received frame.

Access: R/W.

Value Range: 0-255.

Default Value: 0.

7.1.30 PHY_ID_LAST_MSG_DURATION (0x016F)

Duration of last received frame, in ms.

Access: R/W.

Value Range: 0-65535.

Default Value: 0.

7.1.31 PHY_ID_TONE_MAP_RSP_ENABLED_MODS (0x0174)

When selecting the modulation to be notified in a Tone Map Response frame, PHY layer is asked to take the decision. With this PIB we can limit the decision range, enabling or disable different modulations through the 8 bits of the PIB. Bit order is 8PSK_D, 8PSK_C, QPSK_D, QPSK_C, BPSK_D, BPSK_C, ROBO_D, ROBO_C. 1 means enable and 0 means disable. For example, the value to allow only differential modulations would be 0xAA, and to allow only coherent ones, it would be 0x55.

Access: R/W.

Value Range: 0-255.

Default Value: 255 (All Modulations Enabled).

7.1.32 PHY_ID_FORCE_NO_OUTPUT_SIGNAL (0x0175)

Disables Output Signal Power when transmitting. This mode is used in some common impedance measurement tests and it is offered for the final user to be able to perform such tests using the real PHY layer and not a modified one only for those tests.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

7.1.33 PHY_ID_RESET_PHY_STATS (0x0176)

Resets all PHY Statistics PIBs at once.

Access: W.

Value Range: 8-bits. Any value triggers statistics reset.

Default Value: N/A.

7.1.34 PHY_ID_PREAMBLE_NUM_SYNCP (0x0177)

Number of SYNCP symbols to be used when building the preamble of PLC frames. Also used in preamble detection on reception.

Access: R/W.

Value Range: 5-12.

Default Value: 8.

8. API of DATA LINK Layer

This section details the API of the DATA LINK layer modules and their auxiliary files.

8.1 PLC MAC SAP

The functions detailed here (initialization and event handler) are not part of the G3 spec.

These are functions specific to the Microchip implementation of the G3 stack. The ADP layer is in charge of calling these functions through the MAC Wrapper module, so the user does not need to care about calling them.

8.1.1 PLC MAC Initialization Function

The *MacInitialize* function is used to initialize the PLC MAC and layers below.

```
void MacInitialize(struct TMacNotifications *pNotifications, uint8_t u8Band, struct TMacTables *pTables)
```

Parameters:

<i>*pNotifications</i>	Structure with callbacks used to notify MAC specific events (if NULL the layer is de-initialized)
<i>u8Band</i>	Frequency band. This has to match the hardware. Use any one of the constants: - BAND_CENELEC_A - BAND_CENELEC_B - BAND_FCC - BAND_ARIB
<i>*pTables</i>	Pointer to PLC MAC Tables structure. Tables themselves are declared and initialized in Mac Wrapper module so users can change their size to fit their needs.

8.1.2 PLC MAC Event Handler Function

The *MacEventHandler* function is called periodically, to execute the PLC MAC layer State Machine and, thus, process pending Data Requests, Confirms and Indications. This function will also call the event handler of the layers below.

```
void MacEventHandler(void)
```

8.2 PLC MAC Primitives

All the functions and the structures presented in this section are defined in the *MacApi.h*, *MacDefs.h*, *MacCommonDefs.h* and *MacMib.h* files. Refer to these files for details of structure contents and types.

8.2.1 PLC Mac Data Primitives

DATA PRIMITIVE	Microchip Function
MCPS-DATA.request	void MacMcpsDataRequest (struct TMcpsDataRequest *pParameters)
MCPS-DATA.confirm	void (*MacMcpsDataConfirm)(struct TMcpsDataConfirm *Parameters)
MCPS-DATA.indication	void (*MacMcpsDataIndication)(struct TMcpsDataIndication *pDataIndication)
<i>Beyond Spec</i>	void (*MacMcpsMacSnifferIndication)(struct TMcpsMacSnifferIndication *pParameters)

8.2.2 PLC Mac Management Primitives

MANAGEMENT PRIMITIVE	Microchip Function
MLME-GET.request	enum EMacStatus MacMlmeGetRequestSync (enum EMacPibAttribute eAttribute, uint16_t u16Index, struct TMacPibValue *pValue)
MLME-GET.confirm	Not implemented, as MacMlmeGetRequestSync is synchronous and thus returns the value in the same call

MLME-SET.request	enum EMacStatus MacMlmeSetRequestSync (enum EMacPibAttribute eAttribute, uint16_t u16Index, const struct TMacPibValue *pValue)
MLME-SET.confirm	Not implemented, as MacMlmeSetRequestSync is synchronous and thus returns the result in the same call
MLME-RESET.request	void MacMlmeResetRequest (struct TMLmeResetRequest *pParameters)
MLME-RESET.confirm	void (*MacMlmeResetConfirm)(struct TMLmeResetConfirm *pParameters)
MLME-SCAN.request	void MacMlmeScanRequest (struct TMLmeScanRequest *pParameters)
MLME-BEACON-NOTIFY.indication	void (*MacMlmeBeaconNotify)(struct TMLmeBeaconNotifyIndication *pParameters)
MLME-SCAN.confirm	typedef void (*MacMlmeScanConfirm)(struct TMLmeScanConfirm *pParameters)
MLME-START.request	void MacMlmeStartRequest (struct TMLmeStartRequest *pParameters)
MLME-START.confirm	void (*MacMlmeStartConfirm)(struct TMLmeStartConfirm *pParameters)
MLME-COMM-STATUS.indication	typedef void (*MacMlmeCommStatusIndication)(struct TMLmeCommStatusIndication *pParameters)

8.3 PLC MAC Real Time

The lowest level of the PLC MAC layer, where strict response time duties reside, is separated from the MAC library and provided as a separated module.

This is a strong advantage when working with the PL360/PL460 device, as this module is integrated in the code running inside the transceiver with the following advantages:

- Response time requirements of the G3 stack dramatically decrease. The G3 periodic call (*AdpEventHandler* function), can be called every 20 ms, instead of the required 1-ms rate when MAC RT runs in the Host microcontroller.
- Reduction in both code and data memory footprint in the Host microcontroller.

MAC Real Time duties are as follows:

- CRC check
- ACK generation if requested
- CSMA
- ARQ
- Segmentation and reassembly
- Network scan. Generation and processing of BCN and BREQ frames
- Transmission robustness management. Generation and processing of TMR frames

8.4 RF MAC SAP

The initialization and event handler functions are not part of the G3 Hybrid spec.

These are functions specific to the Microchip implementation of the G3 stack. The ADP layer is in charge of calling these functions through the MAC Wrapper module, so the user does not need to care about calling them.

8.4.1 RF MAC Initialization Function

The *MacInitializeRF* function is used to initialize the RF MAC and layers below.

```
void MacInitializeRF(struct TMacNotificationsRF *pNotifications, struct TMacTablesRF *pTables)
```

Parameters:

- | | |
|------------------------|---|
| <i>*pNotifications</i> | Structure with callbacks used to notify RF MAC specific events (if NULL the layer is de-initialized) |
| <i>*pTables</i> | Pointer to RF MAC Tables structure. Tables themselves are declared and initialized in the Mac Wrapper module so the user can change their size to fit their needs |

8.4.2 RF MAC Event Handler Function

The *MacEventHandlerRF* function is called periodically, to execute the RF MAC layer State Machine, and, thus, process pending Data Requests, Confirms and Indications. This function will also call the event handler of the layers below.

```
void MacEventHandlerRF(void)
```

8.5 RF MAC Primitives

All the functions and the structures presented in this section are defined in the *MacRfApi.h*, *MacRfDefs.h*, *MacCommonDefs.h* and *MacRfMib.h* files. Refer to these files for details of structure contents and types.

8.5.1 RF MAC Data Primitives

DATA PRIMITIVE	Microchip Function
MCPS-DATA.request	void MacMcpsDataRequestRF (struct TMcpsDataRequest *pParameters)
MCPS-DATA.confirm	void (*MacMcpsDataConfirm)(struct TMcpsDataConfirm *Parameters)
MCPS-DATA.indication	void (*MacMcpsDataIndication)(struct TMcpsDataIndication *pDataIndication)
<i>Beyond Spec</i>	void (*MacMcpsMacSnifferIndication)(struct TMcpsMacSnifferIndication *pParameters)

8.5.2 RF MAC Management Primitives

MANAGEMENT PRIMITIVE	Microchip Function
MLME-GET.request	enum EMacStatus MacMlmeGetRequestSyncRF (enum EMacPibAttributeRF eAttribute, uint16_t u16Index, struct TMacPibValue *pValue)
MLME-GET.confirm	Not implemented, as MacMlmeGetRequestSyncRF is synchronous and thus returns the value in the same call
MLME-SET.request	enum EMacStatus MacMlmeSetRequestSyncRF (enum EMacPibAttributeRF eAttribute, uint16_t u16Index, const struct TMacPibValue *pValue)
MLME-SET.confirm	Not implemented, as MacMlmeSetRequestSyncRF is synchronous and thus returns the result in the same call
MLME-RESET.request	void MacMlmeResetRequestRF (struct TMLmeResetRequest *pParameters)
MLME-RESET.confirm	void (*MacMlmeResetConfirm)(struct TMLmeResetConfirm *pParameters)
MLME-SCAN.request	void MacMlmeScanRequestRF (struct TMLmeScanRequest *pParameters)
MLME-BEACON-NOTIFY.indication	void (*MacMlmeBeaconNotify)(struct TMLmeBeaconNotifyIndication *pParameters)
MLME-SCAN.confirm	typedef void (*MacMlmeScanConfirm)(struct TMLmeScanConfirm *pParameters)
MLME-START.request	void MacMlmeStartRequestRF (struct TMLmeStartRequest *pParameters)
MLME-START.confirm	void (*MacMlmeStartConfirm)(struct TMLmeStartConfirm *pParameters)
MLME-COMM-STATUS.indication	typedef void (*MacMlmeCommStatusIndication)(struct TMLmeCommStatusIndication *pParameters)

Note that the types specific to the RF MAC layer have the “RF” suffix on their name. Some of the RF MAC API functions share the types with PLC MAC Type definitions, and, thus, no suffix is present on their names.

8.6 MAC Wrapper

This layer is created with these goals:

- To serve as a clean architectural boundary between the MAC and ADP layers. All requests from the ADP layer to MAC, likewise all callbacks from MAC layer to ADP, are routed through this layer.
- To define MAC layer tables and their size. This way, the MAC library is independent from the table sizes, and, thus, can be used for different purposes (such as, Device or Coordinator), or sizes can be accommodate to fit custom needs of final users. So it acts as a *MAC library configuration* file.

- To contain the Hybrid Abstraction Layer, which abstracts upper layer from which MAC layers are present.

Although not considered main goals, having this wrapper as a separate layer accessible outside of the G3 libraries also permits:

- Being used for debug purposes.
- Monitoring the traffic between layers.
- Implementing beyond-spec features when events occur between the ADP and MAC layers.



The correct operation between the ADP and MAC layers is ensured if the MAC wrapper is kept as is. The user may add code in this layer, but always at their own risk and under their responsibility. If the user adds any code, it must always be added after the code already present in the functions to ensure primary operations are done first, and added code is executed only after such primary operation is finished.

As stated above, the Hybrid Abstraction Layer (HyAL) has the role of abstracting the upper layers (ADP, LOADng, LBP) from having 2 different MAC layers (PLC and RF) in the Hybrid Profile, or just one MAC (PLC or RF).

The intention when the Hybrid Profile was defined, was to keep those upper layers as agnostic as possible from the fact that there may be only one MAC layer or two MAC layers below. Thus, a unique API was needed regardless of the MAC layers present.

To do so, HyAL has the API of an IEEE® 802.15.4 MAC layer, but adding a new parameter to some of the primitives, the MediaType, which is explained in the following sections.

Note that in those sections, when it is stated that *it is forwarded to both MAC layers* or *it is received from both MAC layers*, it has to be understood as a general case where both MAC layers are present, and in case only one MAC layer is available, forwarding and reception only applies to such MAC layer.

Refer to the [1. General Architecture](#) for a complete view of the layers on the G3 stack.

8.6.1 MAC Wrapper SAP

8.6.1.1 MacWrapperInitialize

```
void MacWrapperInitialize(struct TMacWrpNotifications *pNotifications, uint8_t u8PlcBand);
```

This function initializes the PLC and RF MAC layers and the callbacks for upper layers.

Parameters:

<i>pNotifications</i>	Structure with callbacks used to notify ADP with events coming from both PLC and RF MAC layers
<i>u8PlcBand</i>	Working PLC band (should be inline with the hardware)

```
struct TMacWrpNotifications {
    MacWrpDataConfirm m_MacWrpDataConfirm;
    MacWrpDataIndication m_MacWrpDataIndication;
    MacWrpResetConfirm m_MacWrpResetConfirm;
    MacWrpBeaconNotify m_MacWrpBeaconNotifyIndication;
    MacWrpScanConfirm m_MacWrpScanConfirm;
    MacWrpStartConfirm m_MacWrpStartConfirm;
    MacWrpCommStatusIndication m_MacWrpCommStatusIndication;
    MacWrpSnifferIndication m_MacWrpSnifferIndication;
};

uint8_t u8PlcBand;
ADP_BAND_CENELEC_A = 0
ADP_BAND_CENELEC_B = 1
```

```
ADP_BAND_FCC = 2
ADP_BAND_ARIB = 3
```

8.6.1.2 MacWrapperEventHandler

```
void MacWrapperEventHandler(void);
```

Performs periodic tasks of Mac Wrapper Layer and calls RF and PLC MAC Event Handlers.

8.6.2 MAC Wrapper Primitives

8.6.2.1 MacWrapper-DATA.request

```
void MacWrapperMcpsDataRequest(struct TMacWrpDataRequest *pParameters);
```

The Data Request primitive is the equivalent of MCPS-DATA.request, but with the *MediaType* field added to the parameters structure. *MediaType* usage is detailed in [8.6.3. Media Type Definition](#).

By receiving this command, the MAC Wrapper forwards the frame to the appropriate MAC sublayer, based on the *MediaType* and *DstAddr* parameters. For unicast transmissions (*DstAddr* different from 0xFFFF):

- If *MediaType* is 0x00, the MAC Wrapper sends a MCPS-DATA.request to the PLC MAC. If the transmission is successful, a MacWrapper-DATA.confirm is generated with status SUCCESS and *MediaType* = 0x00. If the transmission fails (MCPS-DATA.confirm from PLC MAC with status different from SUCCESS), a second attempt will be performed on the backup medium.
- If *MediaType* is 0x01, the MAC Wrapper sends a MCPS-DATA.request to the RF MAC. If the transmission is successful, a MacWrapper-DATA.confirm is generated with status SUCCESS and *MediaType* = 0x01. If the transmission fails (MCPS-DATA.confirm from RF MAC with status different from SUCCESS), a second attempt will be performed on the backup medium.
- If *MediaType* is 0x03, the MAC Wrapper sends a MCPS-DATA.request to the PLC MAC. On reception of the corresponding MCPS-DATA.confirm from the PLC MAC, a MacWrapper-DATA.confirm is generated, with status = MCPS-DATA.confirm status and *MediaType* = 0x00.
- If *MediaType* is 0x04, the MAC Wrapper sends a MCPS-DATA.request to the RF MAC. On reception of the corresponding MCPS-DATA.confirm from the RF MAC, a MacWrapper-DATA.confirm is generated, with status = MCPS-DATA.confirm status and *MediaType* = 0x01.

For broadcast transmissions (*DstAddr* = 0xFFFF) the MAC Wrapper sends a MCPS-DATA.request to the PLC MAC and to the RF MAC:

- If the transmission is successful on either of the two media, a MacWrapper-DATA.confirm is generated with status SUCCESS and *MediaType* = 0x02.
- If transmission is not possible on any media, a MacWrapper-DATA.confirm is generated with status set to the proper error and *MediaType* = 0x02.

8.6.2.2 MacWrapper-DATA.confirm

```
typedef void (*MacWrpDataConfirm)(struct TMacWrpDataConfirm *pParameters);
```

The Data Confirm primitive is the equivalent of MCPS-DATA.confirm, but with the *MediaType* field added to the parameters structure. *MediaType* usage is detailed in [8.6.3. Media Type Definition](#).

The MAC Wrapper generates a MacWrapper-DATA.confirm following the receipt of (at least) a MCPS-DATA.confirm from the layer below.

In case of broadcast transmission (*DstAddr* = 0xFFFF), only one MacWrapper-DATA.confirm is generated after receiving confirmation both from PLC MAC and RF MAC.

8.6.2.3 MacWrapper-DATA.indication

```
typedef void (*MacWrpDataIndication)(struct TMacWrpDataIndication *pParameters);
```

The Data Indication primitive is the equivalent of the MCPS-DATA.indication, but with the *MediaType* field added to the parameters structure. MediaType usage is detailed in [8.6.3. Media Type Definition](#).

The MAC Wrapper generates a MacWrapper-DATA.indication following the receipt of a MCPS-DATA.indication, either from MAC PLC or MAC RF.

A duplicate detection is implemented to drop indications for the same frame coming from a second medium, if already received on the other.

8.6.2.4 MacWrapper-SCAN.request

```
void MacWrapperMlmeScanRequest(struct TMacWrpScanRequest *pParameters);
```

By receiving this command, the MAC Wrapper initiates the scan procedure sending a MLME-SCAN.request both to the PLC MAC and to the RF MAC.

8.6.2.5 MacWrapper-SCAN.confirm

```
typedef void (*MacWrpScanConfirm)(struct TMacWrpScanConfirm *pParameters);
```

The MAC Wrapper generates a MacWrapper-SCAN.confirm following the receipt of a MLME-SCAN.confirm from both the PLC MAC and the RF MAC.

The status returned in MacWrapper-SCAN.confirm is SUCCESS if the scan procedure was successful on at least one of the two interfaces.

8.6.2.6 MacWrapper-BEACON-NOTIFY.indication

```
typedef void (*MacWrpBeaconNotify)(struct TMacWrpBeaconNotifyIndication *pParameters);
```

The Beacon Notify Indication primitive is the equivalent of MLME-BEACON-NOTIFY.indication, but with the *MediaType* field added to the PAN Descriptor structure. MediaType usage is detailed in [8.6.3. Media Type Definition](#).

When receiving a MLME-BEACON-NOTIFY.indication, either from RF MAC or from PLC MAC, the MAC Wrapper fills the MAC Wrapper PAN Descriptor structure setting the MediaType properly and sends the MacWrapper-BEACON-NOTIFY.indication command to the higher layer.

8.6.2.7 MacWrapper-COMM-STATUS.indication

```
typedef void (*MacWrpCommStatusIndication)(struct TMacWrpCommStatusIndication *pParameters);
```

The Comm Status Indication primitive is the equivalent of MLME-COMM-STATUS.indication, but with the *MediaType* field added to the parameters structure. MediaType usage is detailed in [8.6.3. Media Type Definition](#).

When receiving a MLME-COMM-STATUS.indication, either from RF MAC or from PLC MAC, the MAC Wrapper forwards the event through the MacWrapper-COMM-STATUS.indication command, setting the MediaType properly.

8.6.2.8 MacWrapper-START.request

```
void MacWrapperMlmeStartRequest(struct TMacWrpStartRequest *pParameters);
```

When receiving a MacWrapper-START.request primitive, the MAC Wrapper invokes the start requests of both RF and PLC MAC layers.

8.6.2.9 MacWrapper-START.confirm

```
typedef void (*MacWrpStartConfirm)(struct TMacWrpStartConfirm *pParameters);
```

The MAC Wrapper generates a MacWrapper-START.confirm following the receipt of a MLME-START.confirm from both RF and PLC MAC layers.

The status returned in MacWrapper-START.confirm is SUCCESS if the start procedure was successful on both interfaces. Alternately, if the procedure failed on either one of the two interfaces or both, the first received error code value is returned.

8.6.2.10 MacWrapper-RESET.request

```
void MacWrapperMlmeResetRequest(struct TMacWrpResetRequest *pParameters);
```

When receiving a MacWrapper-RESET.request primitive, the MAC Wrapper is reset and, then, the reset requests for both RF and PLC MAC layers are invoked.

8.6.2.11 MacWrapper-RESET.confirm

```
typedef void (*MacWrpResetConfirm)(struct TMacWrpResetConfirm *pParameters);
```

The MAC Wrapper generates a MacWrapper-RESET.confirm following the receipt of a MLME-RESET.confirm from both RF and PLC MAC layers.

The status returned in MacWrapper-RESET.confirm is SUCCESS if the start procedure was successful on both interfaces. Alternately, if the procedure failed on either one of the two interfaces or both, the first received error code value is returned.

8.6.2.12 MacWrapper-GET.request

```
enum EMacWrpStatus MacWrapperMlmeGetRequestSync(enum EMacWrpPibAttribute eAttribute, uint16_t ul6Index, struct TMacWrpPibValue *pValue);
```

By receiving this command, the MAC Wrapper inspects the parameters to check whether the target MAC IB belongs to the PLC MAC, the RF MAC or the Common IB, and, then, forwards the request to the corresponding MAC layer.

This Request is Synchronous, thus the result and value are returned as part of the function call, instead of waiting for a Confirm.

8.6.2.13 MacWrapper-SET.request

```
EMacWrpStatus MacWrapperMlmeSetRequestSync(enum EMacWrpPibAttribute eAttribute, uint16_t ul6Index, const struct TMacWrpPibValue *pValue);
```

By receiving this command, the MAC Wrapper inspects the parameters to check whether the target MAC IB belongs to the PLC MAC, the RF MAC or the Common IB, and, then, forwards the request to the corresponding MAC layer.

This Request is Synchronous, thus the result is returned as part of the function call, instead of waiting for a Confirm.

8.6.2.14 MacWrapper-SNIFFER.indication

```
typedef void (*MacWrpSnifferIndication)(struct TMacWrpSnifferIndication *pParameters);
```

When receiving a MLME-SNIFFER.indication, either from RF MAC or from PLC MAC, the MAC Wrapper forwards the event through the MacWrapper-SNIFFER.indication command.

8.6.3 Media Type Definition

The MediaType parameter used in many of the MAC Wrapper primitives has the following meaning, depending on the primitive type:

Table 8-1. MediaType Definition

MediaType Value	Primitive Type		
	Request	Confirm	Indication
0x00	Power Line interface with Backup Radio Frequency interface	Power Line interface used by default	Power Line interface
0x01	Radio Frequency interface with Backup Power Line interface	Radio Frequency interface used by default	Radio Frequency interface
0x02	Both Power Line and Radio Frequency interfaces	Both Power Line and Radio Frequency interfaces used for transmission	Not used
0x03	Power Line interface with No backup interface	Power Line interface used as backup after failure on Radio Frequency interface	Not used
0x04	Radio Frequency interface with No backup interface	Radio Frequency interface used as backup after failure on Power Line interface	Not used

8.6.4 Backup Medium Usage

The MAC Wrapper implements a functionality that, when possible (both MAC layers present) and allowed (through ADP IB), performs a second transmission attempt using the backup medium (the one that was not primarily intended for transmission).

This procedure occurs when the MAC Wrapper sends a MCPS-DATA.request to the lower layer (PLC MAC or RF MAC) and it receives a MCPS-DATA.confirm with a status different from SUCCESS.

Therefore, the following two scenarios are possible:

- MacWrapper-DATA.request with MediaType = 0x00 (PLC). If the transmission over PLC medium fails (MCPS-DATA.confirm from PLC MAC with a status different from SUCCESS) the MAC Wrapper will check the RF POS table:
 - IF the DstAddr is in extended address mode or IF a valid entry is found for the DstAddr specified in the MacWrapper-DATA.request, send a MCPS-DATA.request to the RF MAC, and wait for the corresponding confirm.
 - IF the MCPS-DATA.confirm status is SUCCESS, generate a MacWrapper-DATA.confirm with SUCCESS status and MediaType = 0x04 (transmission over PLC failed).
 - ELSE, generate a MacWrapper-DATA.confirm with the status set to the proper error and MediaType = 0x04 (subsequent transmissions over PLC and RF failed, the status provides information for the RF transmission).
 - ELSE, generate a MacWrapper-DATA.confirm with the status set to the proper error and MediaType = 0x00.
- MacWrapper-DATA.request with MediaType = 0x01 (RF). If the transmission over the RF medium fails (MCPS-DATA.confirm from RF MAC with Status different from SUCCESS), the MAC Wrapper will check the PLC POS table:
 - IF the DstAddr is in extended address mode or IF a valid entry is found for the DstAddr specified in the MacWrapper-DATA.request, send a MCPS-DATA.request to the PLC MAC and wait for the corresponding confirm.
 - IF the MCPS-DATA.confirm status is SUCCESS, generate a MacWrapper-DATA.confirm with status SUCCESS and MediaType = 0x03 (transmission over RF failed).
 - ELSE, generate a MacWrapper-DATA.confirm with the status set to the proper error and MediaType = 0x03 (subsequent transmissions over RF and PLC failed, the status provides information for the PLC transmission).

- ELSE, generate a MacWrapper-DATA.confirm with the status set to the proper error and MediaType = 0x01.

8.7 MIB Objects Specification and Access

As the access to any of the MAC layers has to always be performed through the MAC Wrapper, this chapter enumerates objects using the name as defined in the *mac_wrapper_defs.h* file.

The default endianness of all MIB is little endian. In other cases, it will be explicitly specified.

8.7.1 MAC_WRP_PIB_MAX_BE (0x00000047)

Maximum value of back-off exponent. It should always be greater than macMinBE.

Access: R/W.

Value Range: 0–20.

Default Value: 8.

8.7.2 MAC_WRP_PIB_BSN (0x00000049)

Beacon frame sequence number.

Access: R/W.

Value Range: 0-255.

Default Value: random.

8.7.3 MAC_WRP_PIB_DSN (0x0000004C)

Data frame sequence number.

Access: R/W.

Value Range: 0-255.

Default Value: random.

8.7.4 MAC_WRP_PIB_MAX_CSMA_BACKOFFS (0x0000004E)

Maximum number of back-off attempts.

Access: R/W.

Value Range: 0-255.

Default Value: 50.

8.7.5 MAC_WRP_PIB_MIN_BE (0x0000004F)

Minimum value of back-off exponent.

Access: R/W.

Value Range: 0–20.

Default Value: 3.

8.7.6 MAC_WRP_PIB_PAN_ID (0x00000050)

PAN ID of the network.

Access: R/W.

Value Range: 0-65535.

Default Value: 65535.

8.7.7 **MAC_WRP_PIB_PROMISCUOUS_MODE (0x00000051)**

Promiscuous mode enabled as defined in IEEE 802.15.4 specification.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

8.7.8 **MAC_WRP_PIB_SHORT_ADDRESS (0x00000053)**

Device short address.

Access: R/W.

Value Range: 0-65535.

Default Value: 65535.

8.7.9 **MAC_WRP_PIB_MAX_FRAME_RETRIES (0x00000059)**

Maximum number of retransmission.

Access: R/W.

Value Range: 0-10.

Default Value: 5.

8.7.10 **MAC_WRP_PIB_TIMESTAMP_SUPPORTED (0x0000005C)**

MAC frame time stamp support enabled.

Access: Read-only.

Value Range: 0-1.

Default Value: 1

8.7.11 **MAC_WRP_PIB_SECURITY_ENABLED (0x0000005D)**

Security enabled.

Access: Read-only.

Value Range: 0-1.

Default Value: 1.

8.7.12 **MAC_WRP_PIB_KEY_TABLE (0x00000071)**

This attribute holds GMK keys required for MAC layer ciphering. The attribute can hold two 16-byte keys. The row index corresponds to the key identifier value. For security reasons, the key entries cannot be read, only written or deleted.

Access: Write only.

Number of entries: 2.

Entry format:

```
struct TMacSecurityKey {  
    bool m_bValid;  
    uint8_t m_au8Key[MAC_SECURITY_KEY_LENGTH]; // 16 bytes  
};
```

Default Value: empty table.

Add Entry: Call Set function with index [0-1], length = 16, value = 16-byte key value. Valid flag automatically set.

Delete Entry: Call Set function with index [0-1], length = 0: Key value and Valid flag automatically cleared.

8.7.13 MAC_WRP_PIB_FRAME_COUNTER (0x00000077)

The outgoing frame counter for this device. This value has to be restored across power fail.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.14 MAC_WRP_PIB_DUPLICATE_DETECTION_TTL (0x00000077)

Time a received tuple [source address + sequence-number] is retained for duplicate frame detection, in seconds.

Access: R/W.

Value Range: 0-255.

Default Value: 3.

8.7.15 MAC_WRP_PIB_HIGH_PRIORITY_WINDOW_SIZE (0x00000100)

The high priority contention window size, in number of slots.

Access: R/W.

Value Range: 1-7:

Default Value: 7.

8.7.16 MAC_WRP_PIB_TX_DATA_PACKET_COUNT (0x00000101)

Statistic counter of successfully transmitted unicast MSDUs.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.17 MAC_WRP_PIB_RX_DATA_PACKET_COUNT (0x00000102)

Statistic counter of successfully received unicast MSDUs.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.18 MAC_WRP_PIB_TX_CMD_PACKET_COUNT (0x00000103)

Statistic counter of successfully transmitted command packets.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.19 MAC_WRP_PIB_RX_CMD_PACKET_COUNT (0x00000104)

Statistic counter of successfully received command packets.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.20 **MAC_WRP_PIB_CSMA_FAIL_COUNT (0x00000105)**

Counts the number of times the CSMA back-offs reach macMaxCSMABackoffs.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.21 **MAC_WRP_PIB_CSMA_NO_ACK_COUNT (0x00000106)**

Counts the number of times an ACK is not received while transmitting a unicast data frame (The loss of ACK is attributed to collisions).

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.22 **MAC_WRP_PIB_RX_DATA_BROADCAST_COUNT (0x00000107)**

Statistic counter of successfully received broadcast frames.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.23 **MAC_WRP_PIB_TX_DATA_BROADCAST_COUNT (0x00000108)**

Statistic counter of the number of broadcast frames sent.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.24 **MAC_WRP_PIB_BAD_CRC_COUNT (0x00000109)**

Statistic counter of the number of frames received with bad CRC.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.25 **MAC_WRP_PIB_NEIGHBOUR_TABLE (0x0000010A)**

The neighbour table.

Access: R/W.

Number of entries:

The table size is configurable for each project by means of the *conf_tables.h* file. Default is 35 entries for Device and 500 entries for Coordinator.

Entry format:

```
struct TMacWrpNeighbourEntry {  
    TMacWrpShortAddress m_nShortAddress; // 2 bytes  
    struct TMacWrpToneMap m_ToneMap; // 3 bytes (only first used in Cenelec band-plan)  
    uint8_t m_nModulationType : 3;  
    uint8_t m_nTxGain : 4;  
};
```

```

uint8_t m_nTxRes : 1;
struct TMacWrpTxCoef m_TxCoef; // 6 bytes
uint8_t m_nModulationScheme : 1;
uint8_t m_nPhaseDifferential : 3;
uint8_t m_u8Lqi;
uint16_t m_u16TmrValidTime; // TMResponse parameters validity time (in seconds)
};

```

Figure 8-1. Neighbour Table fields

Short Address	Tone Map	TxRes	TxGain	Modulation Type	TxCoef	Not Used	Phase Differential	Modulation Scheme	Lqi	Tmr ValidTime
2 bytes	3 bytes	1 byte			6 bytes	1 byte			1 byte	2 bytes
		1 bit	4 bits	3 bits		4 bits	3 bits	1 bit		

This table is ordered according to m_u16TmrValidTime. The newest entries are placed at top of the table.

Default Value: empty table

Get Entry: Call Get function with index [0-34], returned value will have the format described above.

Add Entry: Call Set function with index [0-34], index will be ignored and new entry will be placed according to m_u16TmrValidTime as described above.

Delete Entry: Set entry with short address to be deleted and m_u16TmrValidTime equal to 0. Index is ignored, entry will be searched by short address and deleted.

To get all the active entries, the first step is to get the number of such active entries by means of [8.7.63. MAC_WRP_PIB_MANUF_NEIGHBOUR_TABLE_COUNT \(0x08000012\)](#). Then, because the table is ordered, all the entries can be obtained by iterating from 0 to the number of active entries.

8.7.26 MAC_WRP_PIB_FREQ_NOTCHING (0x0000010B)

The attribute was intended to set the S-FSK 63 to 74 kHz frequency notching. Even if available, this attribute is ignored. Use MAC_WRP_PIB_TONE_MASK instead.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

8.7.27 MAC_WRP_PIB_CSMA_FAIRNESS_LIMIT (0x0000010C)

Channel access fairness limit. Specifies after how many failed back-off attempts, back-off exponent is set to minBE. Should be at least (2×(macMaxBE-macMinBE)).

Access: R/W.

Value Range: 0-255.

Default Value: 15.

8.7.28 MAC_WRP_PIB_TMR_TTL (0x0000010D)

Maximum time to live of tone map parameters entry in the neighbor table, in minutes.

Access: R/W.

Value Range: 0-255.

Default Value: 2 in Spec'15, 10 in Spec'17.

8.7.29 MAC_WRP_PIB_NEIGHBOUR_TABLE_ENTRY_TTL (0x0000010E)

This attribute is not available from the April '17 G3 Specification, so it cannot be referenced when running mode compliant with Spec'17. In such case, the concept of Neighbour entries validity does not exist, and only the contents validity is taken into account, which is controlled by [8.7.28. MAC_WRP_PIB_TMR_TTL \(0x0000010D\)](#).

Maximum time to live for an entry in the neighbor table, in minutes.

Access: R/W.

Value Range: 0-255.

Default Value: 255.

8.7.30 MAC_WRP_PIB_POS_TABLE_ENTRY_TTL (0x0000010E)

This attribute is new with the April '17 G3 Specification, so it cannot be referenced when running mode compliant with Spec'15. Note that the numeric ID has been reused from MAC_WRP_PIB_NEIGHBOUR_TABLE_ENTRY_TTL of previous Specs. Be careful as it may lead to confusion.

Maximum time to live for an entry in the POS table, in minutes.

Access: R/W.

Value Range: 0-255.

Default Value: 255.

8.7.31 MAC_WRP_PIB_RC_COORD (0x0000010F)

Route cost to coordinator to be used in the beacon payload as RC_COORD.

Access: R/W.

Value Range: 0-65535.

Default Value: 65535.

8.7.32 MAC_WRP_PIB_TONE_MASK (0x00000110)

Defines the tone mask to use during symbol formation. It is represented using one bit per carrier. Holds up to 72 carriers (for FCC) so its size is 9 bytes. Bits set indicate carrier used; bits cleared indicate carrier masked. Depending on the band used, a different number of bits are taken into account.

This PIB is a 9-byte array coded little endian (LSB first). In the Cenelec-A band, the carriers are placed like this:

Byte 0								Byte 1								Byte 2	Byte 3	Byte 4								Byte 5	Byte 6	Byte 7	Byte 8
b7	b6	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0	...	...	b7	b6	b5	b4	b3	b2	b1	b0				
Carrier 30	Carrier 29	Carrier 28	Carrier 27	Carrier 26	Carrier 25	Carrier 24	Carrier 23	Carrier 38	Carrier 37	Carrier 36	Carrier 35	Carrier 34	Carrier 33	Carrier 32	Carrier 31	..	..	Not Used	Not Used	Not Used	Not Used	Carrier 58	Carrier 57	Carrier 56	Carrier 55	Not Used	Not Used	Not Used	Not Used

Access: R/W.

Value Range: 0x00-0xFF for each of 9 bytes.

Default Value: All bytes set to 0xFF (All carriers are enabled).

8.7.33 MAC_WRP_PIB_BEACON_RANDOMIZATION_WINDOW_LENGTH (0x00000111)

Duration time in seconds for beacon randomization.

Access: R/W.

Value Range: 1-254.

Default Value: 12.

8.7.34 MAC_WRP_PIB_A (0x00000112)

This parameter controls the adaptive CW linear decrease.

Access: R/W.

Value Range: 3-20.

Default Value: 8.

8.7.35 MAC_WRP_PIB_K (0x00000113)

Rate adaptation factor for channel access fairness limit.

Access: R/W.

Value Range: 1-macCSMAFairnessLimit.

Default Value: 5.

8.7.36 MAC_WRP_PIB_MIN_CW_ATTEMPTS (0x00000114)

Number of consecutive attempts while using minimum CW.

Access: R/W.

Value Range: 0-255

Default Value: 10.

8.7.37 MAC_WRP_PIB_CENELEC_LEGACY_MODE (0x00000115)

This read only attribute indicates the capability of the node. This means that, when set to 1, the following configuration is used:

- Full block interleaving
- Interleaver parameters n_i and n_j are swapped when $I(i,j) = 0$

Access: Read-only.

Value Range: 0-1.

Default Value: 1.

8.7.38 MAC_WRP_PIB_FCC_LEGACY_MODE (0x00000116)

This read only attribute indicates the capability of the device. This means that, when set to 1, the following configuration is used:

- Coherent FCH modulation
- Full block interleaving
- Interleaver parameters n_i and n_j are swapped when $I(i,j) = 0$
- Two RS blocks capability

Access: Read-only.

Value Range: 0-1.

Default Value: 1.

8.7.39 MAC_WRP_PIB_BROADCAST_MAX_CW_ENABLE (0x0000011E)

If enabled (1), MAC uses maximum contention window for Broadcast frames.

This attribute is new on G3 Specification April '17, so it cannot be referenced when running mode compliant with Spec'15.

Access: R/W.

Value Range: 0-1

Default Value: 0.

8.7.40 MAC_WRP_PIB_TRANSMIT_ATTEN (0x0000011F)

Attenuation of the output level in dB.

Access: R/W.

Value Range: 0-25

Default Value: 0.

8.7.41 MAC_WRP_PIB_POS_TABLE (0x00000120)

The POS table.

Access: R/W.

Number of entries: Default configuration is set to 100 for Device, 2000 for Coordinator. Anyway the Table size is configurable for each project by means of the *conf_tables.h* file.

Entry format:

```
struct TMacWrpPOSEntry {
    TMacWrpShortAddress m_nShortAddress; // 2 bytes
    uint8_t m_u8Lqi;
    uint16_t m_u16POSValidTime; // Entry validity time (in seconds)
};
```

This table is ordered according to m_u16POSValidTime. Newest entries are placed at top of the table.

Default Value: empty table.

Get Entry: Call Get function with index [0-99], returned value will have the format described above.

Add Entry: Call Set function with index [0-99], index will be ignored and new entry will be placed according to m_u16POSValidTime as described above.

Delete Entry: Set entry with the short address to be deleted and valid time equal to 0. Index is ignored and the entry will be searched by short address and deleted.

To get all the active entries, first step to get the number of such active entries by means of [8.7.75. MAC_WRP_PIB_MANUF_POS_TABLE_COUNT \(0x0800001E\)](#). Then, since the table is ordered, all entries can be obtained by iterating from 0 to the number of active entries.

8.7.42 MAC_WRP_PIB_POS_RECENT_ENTRY_THRESHOLD (0x00000121)

Threshold in minutes below which POS table entries are considered as recently refreshed.

Access: R/W.

Value Range: 1-255

Default Value: 120.

8.7.43 MAC_WRP_PIB_POS_RECENT_ENTRIES (0x00000122)

Number of POS table entries having been refreshed recently and which LQI is above *adpTrickleLQIThresholdLow*.

Access: R/W.

Value Range: 1-65535

Default Value: 1.

8.7.44 MAC_WRP_PIB_PLC_DISABLE (0x00000123)

Disable PLC PHY Tx and Rx.

Access: R/W.

Value Range: 0-1 (boolean)

Default Value: 0.

8.7.45 MAC_WRP_PIB_MANUF_DEVICE_TABLE (0x08000000)

The device table as specified in G3 Spec. Stores Frame Counter of neighbour nodes from which frames are received.

Access: Read only.

Number of entries: 128 by default. Can be modified through MAC_MAX_DEVICE_TABLE_ENTRIES definition in Mac Wrapper.

Entry format:

```
struct TDeviceTableEntry {
    TPanId m_nPanId; // 2 bytes
    TShortAddress m_nShortAddress; // 2 bytes
    uint32_t m_u32FrameCounter; // 4 bytes
};
```

Default Value: empty table.

Get Entry: Call Get function with index [0-127], returned value will have the format described above.

8.7.46 MAC_WRP_PIB_MANUF_EXTENDED_ADDRESS (0x08000001)

Extended address of the node. Format EUI64. (little endian encoding)

Access: R/W.

Value Range: 0-255 for each of 8 bytes.

Default value: All bytes set to 0.

8.7.47 MAC_WRP_PIB_MANUF_NEIGHBOUR_TABLE_ELEMENT (0x08000002)

This object has the same content as the neighbour table, but it is retrieved using the short address of the node, instead of the index inside the table. To do so, set the short address in the index field in the Get IB function.

Access: Read-only.

8.7.48 MAC_WRP_PIB_MANUF_BAND_INFORMATION (0x08000003)

This read only object returns information about the band.

The intention of this IB is more internal than for G3 user. It is used by G3 stack to perform calculations based on these parameters.

Entry format:

```
struct TPhyBandInformation {
    uint8_t m_u8Band; // 1 byte
    uint8_t m_u8Tones; // 1 byte
    uint8_t m_u8Carriers; // 1 byte
    uint8_t m_u8TonesInCarrier; // 1 byte
    uint8_t m_u8FlBand; // 1 byte
    uint16_t m_u16FlMax; // 2 bytes
};
```

```
uint8_t m_u8MaxRsBlocks; // 1 byte
uint8_t m_u8TxCoefBits; // 1 byte
uint8_t m_u8PilotsFreqSpa; // 1 byte
};
```

- 1 byte – band (CENELEC_A = 0, CENELEC_B = 1, FCC = 2, ARIB = 3)
- 1 byte – number of tones in band
- 1 byte – number of carrier groups
- 1 byte – tones in a carrier group
- 1 byte – FLBand. Symbol grouping in FCH coding (4 for CENELEC A & B, 1 for FCC & ARIB)
- 1 byte – reserved
- 2 byte – FLmax. Maximum symbols as coded in FCH (63 for CENELEC A & B, 511 for FCC & ARIB)
- 1 byte – maximum number of RS blocks (2 for FCC, 1 for other bands)
- 1 byte – number of bits in one entry of the TXCOEF table (4 for CENELEC A & B, 2 for FCC & ARIB)
- 1 byte – Pilot tones spacing in number of carriers (8 for CENELEC_B, 12 for other bands)

Access: Read-only.

8.7.49 MAC_WRP_PIB_MANUF_COORD_SHORT_ADDRESS (0x08000004)

Short address of the coordinator. Set after bootstrap process.

Access: Read-only.

Value Range: 0-65535.

Default Value: 0.

8.7.50 MAC_WRP_PIB_MANUF_MAX_MAC_PAYLOAD_SIZE (0x08000005)

Maximal length of an MSDU.

Access: Read-only.

Value Range: 0-65535.

Default value: 400.

8.7.51 MAC_WRP_PIB_MANUF_SECURITY_RESET (0x08000006)

Resets the device table. Intended for testing purposes, not to be used in final products.

Access: write only.

8.7.52 MAC_WRP_PIB_MANUF_FORCED_MOD_SCHEME (0x08000007)

Forces payload modulation scheme to Differential or Coherent mode for every transmitted frame, instead of using neighbour table info. For testing purposes.

Access: R/W.

Value Range: 0-2.

- 0 – Use Neighbour table info.
- 1 – Force Differential scheme.
- 2 – Force Coherent scheme.

Default value: 0.

8.7.53 MAC_WRP_PIB_MANUF_FORCED_MOD_TYPE (0x08000008)

Forces payload modulation type to any of the available modes for every transmitted frame, instead of using neighbour table info. For testing purposes.

Access: R/W.

Value Range: 0-4.

- 0 – Use Neighbour table info.
- 1 – Force ROBO.
- 2 – Force BPSK.
- 3 – Force QPSK.
- 4 – Force 8PSK.

Default value: 0.

8.7.54 MAC_WRP_PIB_MANUF_FORCED_TONEMAP (0x08000009)

Forces the payload tone map to any value for every transmitted frame, instead of using neighbour table information. For testing purposes.

Access: R/W.

Entry format:

```
struct TPhyToneMap {  
    uint8_t m_au8Tm[3];  
};
```

A value of “all zeros” means the tone map is not forced and the neighbour table information will be used.

Default value: [0x00, 0x00, 0x00].

Note: For the Cenelec A & B Bands, only first element of the array is used, as tone map length is 6 or 4 bits respectively.

8.7.55 MAC_WRP_PIB_MANUF_FORCED_MOD_SCHEME_ON_TMRESPONSE (0x0800000A)

Forces the Modulation Scheme bit to Differential or Coherent mode on Tone Map Response message, instead of using internal G3 stack information.

Access: R/W.

Value Range: 0-2.

- 0 – Use internal G3 stack information.
- 1 – Force Differential scheme.
- 2 – Force Coherent scheme.

Default value: 0.

8.7.56 MAC_WRP_PIB_MANUF_FORCED_MOD_TYPE_ON_TMRESPONSE (0x0800000B)

Forces modulation type bits to any of the available modes on the tone map response message, instead of using internal G3 stack info.

Access: R/W.

Value Range: 0-4.

- 0 – Use internal G3 stack info.
- 1 – Force ROBO.
- 2 – Force BPSK.
- 3 – Force QPSK.
- 4 – Force 8PSK.

Default value: 0.

8.7.57 MAC_WRP_PIB_MANUF_FORCED_TONEMAP_ON_TMRESPONSE (0x0800000C)

Forces the tone map field to any value on the tone map response message, instead of using internal G3 stack information.

Access: R/W.

Entry format:

```
struct TPhyToneMap {
    uint8_t m_au8Tm[3];
};
```

A value of “all zeros” means Tone Map field is not forced and internal G3 stack info will be used.

Default value: [0x00, 0x00, 0x00].

Note: For the Cenelec A & B Bands, only first element of the array is used, as tone map length is 6 or 4 bits respectively.

8.7.58 MAC_WRP_PIB_MANUF_LAST_RX_MOD_SCHEME (0x0800000D)

Gets Modulation scheme of the last received data frame.

Access: Read only.

Value Range: 0-1.

- 0 – Differential scheme.
- 1 – Coherent scheme.

Default value: 0.

8.7.59 MAC_WRP_PIB_MANUF_LAST_RX_MOD_TYPE (0x0800000E)

Gets modulation type of the last received data frame.

Access: Read only.

Value Range: 0-3.

- 0 – ROBO.
- 1 – BPSK.
- 2 – QPSK.
- 3 – 8PSK.

Default value: 0.

8.7.60 MAC_WRP_PIB_MANUF_LBP_FRAME_RECEIVED (0x0800000F)

Flag indicating whether LBP frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.61 MAC_WRP_PIB_MANUF_LNG_FRAME_RECEIVED (0x08000010)

Flag indicating whether LOADng frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.62 MAC_WRP_PIB_MANUF_BCN_FRAME_RECEIVED (0x08000011)

Flag indicating whether beacon frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.63 MAC_WRP_PIB_MANUF_NEIGHBOUR_TABLE_COUNT (0x08000012)

Gets the number of valid elements in the neighbour table.

Access: Read only.

Value Range: 0-65535.

Default value: 0.

8.7.64 MAC_WRP_PIB_MANUF_RX_OTHER_DESTINATION_COUNT (0x08000013)

Statistic counter of discarded packets due to other destination.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.65 MAC_WRP_PIB_MANUF_RX_INVALID_FRAME_LENGTH_COUNT (0x08000014)

Statistic counter of discarded packets due to invalid frame length.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.66 MAC_WRP_PIB_MANUF_RX_MAC_REPETITION_COUNT (0x08000015)

Statistic counter of discarded packets due to MAC repetition.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.67 MAC_WRP_PIB_MANUF_RX_WRONG_ADDR_MODE_COUNT (0x08000016)

Statistic counter of discarded packets due to Wrong Addressing Mode.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.68 MAC_WRP_PIB_MANUF_RX_UNSUPPORTED_SECURITY_COUNT (0x08000017)

Statistic counter of discarded packets due to unsupported security.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.69 MAC_WRP_PIB_MANUF_RX_WRONG_KEY_ID_COUNT (0x08000018)

Statistic counter of discarded packets due to wrong key ID.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.70 MAC_WRP_PIB_MANUF_RX_INVALID_KEY_COUNT (0x08000019)

Statistic counter of discarded packets due to invalid key.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.71 MAC_WRP_PIB_MANUF_RX_WRONG_FC_COUNT (0x0800001A)

Statistic counter of discarded packets due to wrong frame counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.72 MAC_WRP_PIB_MANUF_RX_DECRYPTION_ERROR_COUNT (0x0800001B)

Statistic counter of discarded packets due to decryption error.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.73 MAC_WRP_PIB_MANUF_RX_SEGMENT_DECODE_ERROR_COUNT (0x0800001C)

Statistic counter of discarded packets due to segment decode error.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.74 MAC_WRP_PIB_MANUF_ENABLE_MAC_SNIFFER (0x0800001D)

MAC Sniffer mode enabled. This mode processes all frames at MAC level, regardless of addressing, and outputs all correct frames by means of [MacMcpsMacSnifferIndication](#) callback. The difference with Promiscuous mode is that, in Sniffer mode, the device will still work as a regular node in the network, treating frames addressed to it like a normal device would do. Summarizing, Sniffer mode is the combination of Normal mode plus Promiscuous mode.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

8.7.75 MAC_WRP_PIB_MANUF_POS_TABLE_COUNT (0x0800001E)

Gets number of valid elements in the POS Table.

POS Table is new for the April '17 G3 Specification, so this attribute cannot be referenced when running mode compliant with Spec'15.

Access: Read only.

Value Range: 0-65535.

Default value: 0.

8.7.76 MAC_WRP_PIB_MANUF_RETRIES_LEFT_TO_FORCE_ROBO (0x0800001F)

This IB is used to force ROBO mode when a certain number of MAC retries are left before discarding the frame due to no ACK reception. It can be used to increase the robustness of communications after retries. Of course robustness increase causes throughput decrease.

Access: R/W.

Value Range: 0-macMaxFrameRetries.

Default value: 0.

8.7.77 MAC_WRP_PIB_MANUF_MAC_INTERNAL_VERSION (0x08000021)

Gets current MAC internal version number.

Entry format:

```
struct TMacSoftVersion {
    uint8_t m_u8Major;
    uint8_t m_u8Minor;
    uint8_t m_u8Revision;
    uint8_t m_u8Year; // year since 2000
    uint8_t m_u8Month;
    uint8_t m_u8Day;
};
```

Access: Read only.

Value Range: N/A.

Default value: Current MAC version.

8.7.78 MAC_WRP_PIB_MANUF_MAC_RT_INTERNAL_VERSION (0x08000022)

Gets current MAC Real Time internal version number.

Entry format:

```
struct TMacRtSoftVersion {
    uint8_t m_u8Major;
    uint8_t m_u8Minor;
    uint8_t m_u8Revision;
    uint8_t m_u8Year; // year since 2000
    uint8_t m_u8Month;
    uint8_t m_u8Day;
};
```

Access: Read only.

Value Range: N/A.

Default value: Current MAC Real Time version.

8.7.79 MAC_WRP_PIB_MANUF_RESET_MAC_STATS (0x08000023)

Resets all IBs related to MAC statistics at once.

Access: W.

Value Range: 8-bits. Any value triggers statistics reset.

Default Value: N/A.

8.7.80 MAC_WRP_PIB_MANUF_SLEEP_MODE (0x08000024)

This IB is used to set/clear the Sleep mode of the PL360, which is available under certain conditions. This mode is detailed in the PL360 Host Controller documentation.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

8.7.81 MAC_WRP_PIB_MANUF_DEBUG_SET (0x08000025)

This IB is used for the special Debug mode of the PL360. This mode is detailed in the PL360 Host Controller documentation.

8.7.82 MAC_WRP_PIB_MANUF_DEBUG_READ (0x08000026)

This IB is used for the special Debug mode of the PL360. This mode is detailed in the PL360 Host Controller documentation.

8.7.83 MAC_WRP_PIB_MANUF_POS_TABLE_ELEMENT (0x08000027)

This object has the same content as the POS table, but it is retrieved using the short address of the node, instead of the index inside the table. To do so, set the short address in the index field in the Get IB function.

Access: Read-only.

8.7.84 MAC_WRP_PIB_MANUF_TRICKLE_MIN_LQI (0x08000028)

Minimum LQI to consider a neighbour for Trickle algorithm.

Access: Write only.

Value Range: 0-255.

Default value: 80.

8.7.85 MAC_WRP_PIB_MANUF_NEIGHBOUR_LQI (0x08000029)

LQI for a given neighbour, which short address will be indicated by index in the Get IB function.

Access: Read only.

Value Range: 0-255.

Default value: N/A.

8.7.86 MAC_WRP_PIB_MANUF_BEST_LQI (0x0800002A)

Best LQI found in neighbour table.

Access: Read only.

Value Range: 0-255.

Default value: N/A.

8.7.87 MAC_WRP_PIB_MANUF_PLC_IFACE_AVAILABLE (0x0800002C)

PLC Interface availability.

Access: Read only.

Value Range: 0-1 (boolean).

Default value: N/A.

8.7.88 MAC_WRP_PIB_MANUF_LAST_FRAME_DURATION_PLC (0x0800002D)

Last PLC frame duration in ms.

Access: Read only.

Value Range: 0-65535.

Default value: N/A.

8.7.89 MAC_WRP_PIB_MANUF_PHY_PARAM (0x08000020)

Gives access to some PHY layer parameters and statistics. It is treated as a table, where the index determines which parameter to access. Available parameters and their corresponding indexes are the following:

A link to the corresponding PHY IB is provided for more detailed information.

MAC_WRP_PHY_PARAM_VERSION, index 0x010C. (7.1.3. PHY_ID_INFO_VERSION (0x010C))

Description: Retrieves PHY layer version number. 32 bits.

Access: Read only.

Default Value: N/A.

MAC_WRP_PHY_PARAM_TX_TOTAL, index 0x0110. (7.1.4. PHY_ID_TX_TOTAL (0x0110))

Description: Correctly transmitted frame count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_TOTAL_BYTES, index 0x0114. (7.1.5. PHY_ID_TX_TOTAL_BYTES (0x0114))

Description: Transmitted bytes count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_TOTAL_ERRORS, index 0x0118. (7.1.6. PHY_ID_TX_TOTAL_ERRORS (0x0118))

Description: Transmission errors count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_BAD_BUSY_TX, index 0x011C. (7.1.7. PHY_ID_BAD_BUSY_TX (0x011C))

Description: Transmission failure due to already in transmission. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_BAD_BUSY_CHANNEL, index 0x0120. (7.1.8. PHY_ID_TX_BAD_BUSY_CHANNEL (0x0120))

Description: Transmission failure due to busy channel. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_BAD_LEN, index 0x0124. (7.1.9. PHY_ID_TX_BAD_LEN (0x0124))

Description: Bad length in message (too short - too long). 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_BAD_FORMAT, index 0x0128. (7.1.10. PHY_ID_TX_BAD_FORMAT (0x0128))

Description: Message to transmit in bad format. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_TX_TIMEOUT, index 0x012C. (7.1.11. PHY_ID_TX_TIMEOUT (0x012C))

Description: Timeout error in transmission. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_TOTAL, index 0x0130. (7.1.12. PHY_ID_RX_TOTAL (0x0130))

Description: Correctly received messages count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_TOTAL_BYTES, index 0x0134. (7.1.13. PHY_ID_RX_TOTAL_BYTES (0x0134))

Description: Received bytes count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_RS_ERRORS, index 0x0138. (7.1.14. PHY_ID_RX_RS_ERRORS (0x0138))

Description: Reception RS errors count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_EXCEPTIONS, index 0x013C. (7.1.15. PHY_ID_RX_EXCEPTIONS (0x013C))

Description: Reception Exceptions count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_BAD_LEN, index 0x0140. (7.1.16. PHY_ID_RX_BAD_LEN (0x0140))

Description: Bad length in message (too short - too long). 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_BAD_CRC_FCH, index 0x0144. (7.1.17. PHY_ID_RX_BAD_CRC_FCH (0x0144))

Description: Bad CRC in received FCH. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_FALSE_POSITIVE, index 0x0148. (7.1.18. PHY_ID_RX_FALSE_POSITIVE (0x0148))

Description: Incomplete synchronization. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_RX_BAD_FORMAT, index 0x014C. (7.1.19. PHY_ID_RX_BAD_FORMAT (0x014C))

Description: Received message in bad format. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_PHY_PARAM_ENABLE_AUTO_NOISE_CAPTURE, index 0x0166. (7.1.25. PHY_ID_ENABLE_AUTO_NOISE_CAPTURE (0x0166))

Description: Noise autodetection and adaptation. 8 bits.

Access: R/W.

Default Value: 1.

MAC_WRP_PHY_PARAM_TIME_BETWEEN_NOISE_CAPTURES, index 0x0158. (7.1.20. PHY_ID_TIME_BETWEEN_NOISE_CAPTURES (0x0158))

Description: Time between noise captures (in ms). 32 bits.

Access: R/W.

Default Value: 60000.

MAC_WRP_PHY_PARAM_CFG_AUTODETECT_BRANCH, index 0x0161. (7.1.21. PHY_ID_CFG_AUTODETECT_IMPEDANCE (0x0161))

Description: Automatic Impedance detection by PHY layer. 8 bits treated as Boolean.

Access: R/W.

Default Value: 2.

MAC_WRP_PHY_PARAM_CFG_IMPEDANCE, index 0x0162. (7.1.22. PHY_ID_CFG_IMPEDANCE (0x0162))

Description: Impedance value to set in case Autodetect is disabled. 8 bits.

Access: R/W.

Default Value: 2 (VLO impedance).

MAC_WRP_PHY_PARAM_RRC_NOTCH_ACTIVE, index 0x0163. (7.1.23. [PHY_ID_RRC_NOTCH_ACTIVE \(0x0163\)](#))

Description: Indicate if notch filter is active or not. 8 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_RRC_NOTCH_INDEX, index 0x0164. (7.1.24. [PHY_ID_RRC_NOTCH_INDEX \(0x0164\)](#))

Description: Index of the notch filter. 8 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_DELAY_NOISE_CAPTURE_AFTER_RX, index 0x0167. (7.1.26. [PHY_ID_DELAY_NOISE_CAPTURE_AFTER_RX \(0x0167\)](#))

Description: Next noise capture is delayed if a frame is correctly received. 8 bits.

Access: R/W.

Default Value: 1.

MAC_WRP_PHY_PARAM_PLC_DISABLE, index 0x016A. (7.1.27. [PHY_ID_PLC_DISABLE \(0x016A\)](#))

Description: Disable PLC Tx/Rx. 8 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_NOISE_PEAK_POWER, index 0x016B. (7.1.28. [PHY_ID_NOISE_PEAK_POWER \(0x016B\)](#))

Description: Returns Detected Noise Peak Power on last Noise Capture, in case a Peak was detected (*MAC_WRP_PHY_PARAM_RRC_NOTCH_ACTIVE* == 1). If no Peak was detected, it returns the Noise average in band measured on last Noise Capture. 8 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_LAST_MSG_LQI, index 0x016C. (7.1.29. [PHY_ID_LAST_MSG_LQI \(0x016C\)](#))

Description: Returns LQI value calculated for last received frame. 8 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_LAST_MSG_RSSI, index 0x016D.

Description: Returns RSSI value calculated for last received frame. Only available in PL360 platform including MAC RT. 16 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_ACK_TX_CFM, index 0x016E.

Description: Counts the number of successful ACK transmissions. Only available in PL360 platform including MAC RT. 16 bits.

Access: R/W.

Default Value: 0.

MAC_WRP_PHY_PARAM_TONE_MAP_RSP_ENABLED_MODS, index 0x0174. (7.1.31. [PHY_ID_TONE_MAP_RSP_ENABLED_MODS \(0x0174\)](#))

Description: Enables / Disables modulations to be chosen by the PHY layer algorithm when a Tone Map Response has to be sent. 8 bits.

Access: R/W.

Default Value: 0xFF.

MAC_WRP_PHY_PARAM_RESET_PHY_STATS, index 0x0176. (7.1.33. [PHY_ID_RESET_PHY_STATS \(0x0176\)](#))

Description: Reset Phy Statistics. 8 bits.

Access: Write only.

Default Value: N/A.

MAC_WRP_PHY_PARAM_PREAMBLE_NUM_SYNC_P, index 0x0177. (7.1.34. [PHY_ID_PREAMBLE_NUM_SYNC_P \(0x0177\)](#))

Description: Number of SyncP symbols in preamble. 8 bits.

Access: R/W.

Default Value: 8.

8.7.90 MAC_WRP_PIB_DSN_RF (0x00000200)

Data frame sequence number.

Access: R/W.

Value Range: 0-255.

Default Value: random.

8.7.91 MAC_WRP_PIB_MAX_BE_RF (0x00000201)

Maximum value of back-off exponent. It should always be greater than macMinBERF.

Access: R/W.

Value Range: 3-8.

Default Value: 5.

8.7.92 MAC_WRP_PIB_MAX_CSMA_BACKOFFS_RF (0x00000202)

Maximum number of back-off attempts.

Access: R/W.

Value Range: 0-5.

Default Value: 4.

8.7.93 MAC_WRP_PIB_MAX_FRAME_RETRIES_RF (0x00000203)

Maximum number of retransmission.

Access: R/W.

Value Range: 0-7.

Default Value: 3.

8.7.94 MAC_WRP_PIB_MIN_BE_RF (0x00000204)

Minimum value of back-off exponent.

Access: R/W.

Value Range: 0-macMaxBeRf.

Default Value: 3.

8.7.95 MAC_WRP_PIB_TIMESTAMP_SUPPORTED_RF (0x00000205)

Support of the optional timestamping feature for incoming and outgoing Data frames.

Access: Read-only.

Value Range: Always set to False (Timestamp feature not supported).

Default Value: False.

8.7.96 MAC_WRP_PIB_DEVICE_TABLE_RF (0x00000206)

The device table as specified in the G3 Spec. It stores the Frame Counter of the neighbour nodes from which the frames are received on the RF medium.

Access: Read only.

Number of entries: 128 by default. Can be modified through MAC_MAX_DEVICE_TABLE_ENTRIES_RF definition in Mac Wrapper.

Entry format:

```
struct TDeviceTableEntry {
    TPanId m_nPanId; // 2 bytes
    TShortAddress m_nShortAddress; // 2 bytes
    uint32_t m_u32FrameCounter; // 4 bytes
};
```

Default Value: empty table.

Get Entry: Call PIB Get function with index [0-127], the returned value will have the format described above.

8.7.97 MAC_WRP_PIB_FRAME_COUNTER_RF (0x00000207)

The outgoing frame counter for this device. RF MAC layer maintains its own Frame Counter (independent from PLC one). This value has to be restored across power fail.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.98 MAC_WRP_PIB_DUPLICATE_DETECTION_TTL_RF (0x00000208)

Maximum time a received pair (source address + sequence-number) is retained for duplicate frame detection, in seconds.

Access: R/W.

Value Range: 0-255.

Default Value: 3.

8.7.99 MAC_WRP_PIB_COUNTER_OCTETS_RF (0x00000209)

The size of the MAC metrics counters in octets.

Access: Read-only.

Value Range: Always set to 4.

Default Value: 4.

8.7.100 MAC_WRP_PIB_RETRY_COUNT_RF (0x0000020A)

The number of transmitted frames that required exactly one retry before acknowledgment.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.101 MAC_WRP_PIB_MULTIPLE_RETRY_COUNT_RF (0x0000020B)

The number of transmitted frames that required more than one retry before acknowledgment.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.102 MAC_WRP_PIB_TX_FAIL_COUNT_RF (0x0000020C)

The number of transmitted frames that did not result in an acknowledgment after macMaxFrameRetries.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.103 MAC_WRP_PIB_TX_SUCCESS_COUNT_RF (0x0000020D)

The number of transmitted frames that were acknowledged successfully after the initial Data frame transmission.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.104 MAC_WRP_PIB_FCS_ERROR_COUNT_RF (0x0000020E)

The number of frames that were discarded due to an incorrect FCS.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.105 MAC_WRP_PIB_SECURITY_FAILURE_COUNT_RF (0x0000020F)

The number of frames that were returned from the security procedure with any Status other than SUCCESS.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.106 MAC_WRP_PIB_DUPLICATE_FRAME_COUNT_RF (0x00000210)

The number of frames that contained the same sequence number as a frame previously received.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.107 MAC_WRP_PIB_RX_SUCCESS_COUNT_RF (0x00000211)

The number of frames that were received correctly.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.108 MAC_WRP_PIB_USE_ENHANCED_BEACON_RF (0x00000213)

Use of Enhanced Beacon by this device.

Access: Read-only.

Value Range: Always set to True.

Default Value: True.

8.7.109 MAC_WRP_PIB_EB_HEADER_IE_LIST_RF (0x00000214)

List of IDs of Header IEs to include in the Enhanced Beacon. Only vendor-specific Header IEs are used in this implementation.

Access: Read-only.

Value Range: Always one element containing ID 0x00 (Vendor Specific IE).

Default Value: 0x00.

8.7.110 MAC_WRP_PIB_EB_PAYLOAD_IE_LIST_RF (0x00000215)

List of IDs of Payload IEs to include in the Enhanced Beacon. No Payload IEs are used in this implementation.

Access: Read-only.

Value Range: Always empty list (length 0).

Default Value: N/A.

8.7.111 MAC_WRP_PIB_EB_FILTERING_ENABLED_RF (0x00000216)

Indicates if devices should perform filtering in response to Enhanced Beacon Request command.

Access: Read-only.

Value Range: Always set to False.

Default Value: False.

8.7.112 MAC_WRP_PIB_EBSN_RF (0x00000217)

BSN used for Enhanced Beacon frames.

Access: R/W.

Value Range: 0-255.

Default Value: random.

8.7.113 MAC_WRP_PIB_EB_AUTO_SA_RF (0x00000218)

Indicates if Enhanced Beacon frames generated by the MAC in response to Enhanced Beacon frame include Source Address field.

Access: Read-only.

Value Range: Always set to Short Address identifier (2).

Default Value: 2.

8.7.114 MAC_WRP_PIB_SEC_SECURITY_LEVEL_LIST_RF (0x0000021A)

Security Level Descriptor of Frame Type indicated by index.

Access: Read-only.

Entry format:

```
struct TSecurityLevelDescriptorRF {
    uint8_t m_u8FrameType;
    uint8_t m_u8CommandId;
    uint8_t m_u8SecurityMinimum; // Fixed to 0
    bool m_bOverrideSecurityMinimum; // Fixed to False
};
```

Default Value: N/A.

8.7.115 MAC_WRP_PIB_POS_TABLE_RF (0x0000021D)

The POS table.

Access: R/W.

Number of entries: Default configuration is set to 100 for Device, 500 for Coordinator Lite, 2000 for Coordinator. The table size is configurable for each project by means of the *conf_tables.h* file.

Entry format:

```
struct TPOEntry {
    TShortAddress m_nShortAddress; // 2 bytes
    uint8_t m_u8ForwardLqi;
    uint8_t m_u8ReverseLqi;
    uint8_t m_u8DutyCycle;
    uint8_t m_u8ForwardTxPowerOffset;
    uint8_t m_u8ReverseTxPowerOffset;
    uint16_t m_u16POSValidTime; // Entry validity time (in seconds)
};
```

This table is ordered according to m_u16POSValidTime. Newest entries are placed at top of the table.

Default Value: empty table.

Get Entry: Call Get function with index [0-99], returned value will have the format described above.

Add Entry: Call Set function with index [0-99], index will be ignored and new entry will be placed according to m_u16POSValidTime as described above.

Delete Entry: Set entry with the short address to be deleted and valid time equal to 0. Index is ignored and the entry will be searched by short address and deleted.

To get all the active entries, first step to get the number of such active entries by means of [8.7.150. MAC_WRP_PIB_MANUF_POS_TABLE_COUNT_RF \(0x08000218\)](#). Then, because the table is ordered, all entries can be obtained by iterating from 0 to the number of active entries.

8.7.116 MAC_WRP_PIB_OPERATING_MODE_RF (0x0000021E)

The RF operating mode as defined in clause 19.3 of [IEEE 802.15.4-2020]. Tables 19.7 and 19.8.

Operating Mode defines Data Rate, Modulation, Modulation Index and Channel Spacing for a given working band.

Access: R/W.

Value Range: 1-4.

Default Value: 1.

8.7.117 MAC_WRP_PIB_CHANNEL_NUMBER_RF (0x0000021F)

The channel number as defined in clause 10.1.3.9 of [IEEE 802.15.4-2020] to be used for channel scanning and data transmission.

Available channels depend on working band, as well as channel spacing.

Access: R/W.

Value Range: 0-7279.

Default Value: 0.

8.7.118 MAC_WRP_PIB_DUTY_CYCLE_USAGE_RF (0x00000220)

Current usage of maximum allowed duty cycle in percent over the current sliding-window measurement period, i.e., $(t_{on} / \text{macDutyCycleLimit_RF}) * 100$.

Access: R/W.

Value Range: 0-100.

Default Value: 0.

8.7.119 MAC_WRP_PIB_DUTY_CYCLE_PERIOD_RF (0x00000221)

Duration of the measurement period in seconds, e.g., 3600 seconds as default for [ETSI EN 303 204].

Access: R/W.

Value Range: 1-65535.

Default Value: 3600.

8.7.120 MAC_WRP_PIB_DUTY_CYCLE_LIMIT_RF (0x00000222)

Duration of the allowed transmission time in seconds, e.g., 2.5%/10% of macDutyCyclePeriod_RF (3600 seconds as default for [ETSI EN 303 204]).



Tip: If this attribute is set equal to macDutyCyclePeriod_RF the value of macDutyCycleUsage_RF will be fixed to zero, and, thus, transmission will never be canceled due to Duty Cycle limits. This is the recommended setting for test environments, where no regulation is required and maximum throughput is desired.

Access: R/W.

Value Range: 1-65535.

Default Value: 90 (PAN Device), 360 (PAN Coordinator).

8.7.121 MAC_WRP_PIB_DUTY_CYCLE_THRESHOLD_RF (0x00000223)

Duty cycle threshold for stopping RF transmissions.

Access: R/W.

Value Range: 0-100.

Default Value: 90.

8.7.122 MAC_WRP_PIB_DISABLE_PHY_RF (0x00000224)

Disable RF PHY Tx and Rx.

Access: R/W.

Value Range: True / False.

Default Value: False.

8.7.123 MAC_WRP_PIB_FREQUENCY_BAND_RF (0x00000225)

Current operating frequency band identifier value as specified in [IEEE 802.15.4-2020] tables 7.21 and 7.22. Valid entries are those corresponding to band designations in Table H.5-1 Clause 10.1.2.

Access: R/W.

Value Range: 0-31.

Default Value: 4.

8.7.124 MAC_WRP_PIB_TRANSMIT_ATTEN_RF (0x00000226)

Attenuation of the RF transmit power in dB. The attenuation shall be applied to all transmissions, i.e. unicast as well as broadcast.

Access: R/W.

Value Range: 0-64.

Default Value: 0.

8.7.125 MAC_WRP_PIB_ADAPTIVE_POWER_STEP_RF (0x00000227)

Maximum change (in dB) during one step when adapting transmission power.

Access: R/W.

Value Range: 0-15.

Default Value: 3.

8.7.126 MAC_WRP_PIB_ADAPTIVE_POWER_HIGH_BOUND_RF (0x00000228)

Reverse LQI threshold above which Tx Power shall be reduced. Set to 255 to disable. This value shall be greater than macAdaptivePowerLowBound_RF by at least 20 dB. Default value is configured according to [ETSI EN 303 204] test for adaptive power control.

Access: R/W.

Value Range: 0-255.

Default Value: 104.

8.7.127 MAC_WRP_PIB_ADAPTIVE_POWER_LOW_BOUND_RF (0x00000229)

Reverse LQI threshold below which Tx Power shall be increased. Set to 0 to disable. This value shall be lower than macAdaptivePowerHighBound_RF by at least 20 dB.

Access: R/W.

Value Range: 0-255.

Default Value: 0.

8.7.128 MAC_WRP_PIB_POS_RECENT_ENTRIES_RF (0x0000022A)

Number of RF POS table entries having been refreshed recently and which LQI is above adpTrickleLQIThresholdLow_RF.

Access: Read only.

Value Range: 0-65535.

Default Value: N/A.

8.7.129 MAC_WRP_PIB_MANUF_SECURITY_RESET_RF (0x08000203)

Resets the device table. Intended for testing purposes, not to be used in final products.

Access: Write only.

8.7.130 MAC_WRP_PIB_MANUF_LBP_FRAME_RECEIVED_RF (0x08000204)

Flag indicating whether LBP frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.131 MAC_WRP_PIB_MANUF_LNG_FRAME_RECEIVED_RF (0x08000205)

Flag indicating whether LOADng frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.132 MAC_WRP_PIB_MANUF_BCN_FRAME_RECEIVED_RF (0x08000206)

Flag indicating whether Beacon frames are being received since last time MIB was set to 0. This IB is intended to be used for collision avoidance during bootstrap phase.

Access: R/W.

Value Range: 0-1.

Default value: 0.

8.7.133 MAC_WRP_PIB_MANUF_RX_OTHER_DESTINATION_COUNT_RF (0x08000207)

Statistic counter of discarded packets due to other destination.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.134 MAC_WRP_PIB_MANUF_RX_INVALID_FRAME_LENGTH_COUNT_RF (0x08000208)

Statistic counter of discarded packets due to invalid frame length.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.135 MAC_WRP_PIB_MANUF_RX_WRONG_ADDR_MODE_COUNT_RF (0x08000209)

Statistic counter of discarded packets due to Wrong Addressing Mode.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.136 MAC_WRP_PIB_MANUF_RX_UNSUPPORTED_SECURITY_COUNT_RF (0x0800020A)

Statistic counter of discarded packets due to unsupported security.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.137 MAC_WRP_PIB_MANUF_RX_WRONG_KEY_ID_COUNT_RF (0x0800020B)

Statistic counter of discarded packets due to wrong key ID.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.138 MAC_WRP_PIB_MANUF_RX_INVALID_KEY_COUNT_RF (0x0800020C)

Statistic counter of discarded packets due to invalid key.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.139 MAC_WRP_PIB_MANUF_RX_WRONG_FC_COUNT_RF (0x0800020D)

Statistic counter of discarded packets due to wrong frame counter.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.140 MAC_WRP_PIB_MANUF_RX_DECRYPTION_ERROR_COUNT_RF (0x0800020E)

Statistic counter of discarded packets due to decryption error.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.141 MAC_WRP_PIB_MANUF_TX_DATA_PACKET_COUNT_RF (0x0800020F)

Statistic counter of successfully transmitted unicast MSDUs.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.142 MAC_WRP_PIB_MANUF_RX_DATA_PACKET_COUNT_RF (0x08000210)

Statistic counter of successfully received unicast MSDUs.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.143 MAC_WRP_PIB_MANUF_TX_CMD_PACKET_COUNT_RF (0x08000211)

Statistic counter of successfully transmitted command packets.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.144 MAC_WRP_PIB_MANUF_RX_CMD_PACKET_COUNT_RF (0x08000212)

Statistic counter of successfully received command packets.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.145 MAC_WRP_PIB_MANUF_CSMA_FAIL_COUNT_RF (0x08000213)

Counts the number of times the CSMA back-offs reach macMaxCSMABackoffsRF.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.146 MAC_WRP_PIB_MANUF_RX_DATA_BROADCAST_COUNT_RF (0x08000214)

Statistic counter of successfully received broadcast frames.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.147 MAC_WRP_PIB_MANUF_TX_DATA_BROADCAST_COUNT_RF (0x08000215)

Statistic counter of the number of broadcast frames sent.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.148 MAC_WRP_PIB_MANUF_BAD_CRC_COUNT_RF (0x08000216)

Statistic counter of the number of frames received with bad CRC.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 0.

8.7.149 MAC_WRP_PIB_MANUF_ENABLE_MAC_SNIFFER_RF (0x08000217)

RF MAC Sniffer mode enabled. This mode processes all frames at MAC level, regardless of addressing, and outputs all correct frames by means of [MacMcpsMacSnifferIndicationRF](#) callback. The difference with Promiscuous mode is that, in Sniffer mode, the device will still work as a regular node in the network, treating frames addressed to it like a normal device would do. Summarizing, Sniffer mode is the combination of Normal mode plus Promiscuous mode.

Access: R/W.

Value Range: 0-1.

Default Value: 0.

8.7.150 MAC_WRP_PIB_MANUF_POS_TABLE_COUNT_RF (0x08000218)

Gets number of valid elements in the RF POS Table.

Access: Read only.

Value Range: 0-65535.

Default value: 0.

8.7.151 MAC_WRP_PIB_MANUF_MAC_INTERNAL_VERSION_RF (0x08000219)

Gets current RF MAC internal version number.

Entry format:

```
struct TMacSoftVersion {
    uint8_t m_u8Major;
    uint8_t m_u8Minor;
    uint8_t m_u8Revision;
    uint8_t m_u8Year; // year since 2000
    uint8_t m_u8Month;
    uint8_t m_u8Day;
};
```

Access: Read only.

Value Range: N/A.

Default value: Current RF MAC version.

8.7.152 MAC_WRP_PIB_MANUF_RESET_MAC_STATS_RF (0x0800021A)

Resets all IBs related to RF MAC statistics at once.

Access: W.

Value Range: 8-bits. Any value triggers statistics reset.

Default Value: N/A.

8.7.153 MAC_WRP_PIB_MANUF_POS_TABLE_ELEMENT_RF (0x0800021B)

This object has the same content as the RF POS table, but it is retrieved using the short address of the node, instead of the index inside the table. To do so, set the short address in the index field in the Get IB function.

Entry format:

```
struct TPOSEntry {
    TShortAddress m_nShortAddress; // 2 bytes
    uint8_t m_u8ForwardLqi;
    uint8_t m_u8ReverseLqi;
    uint8_t m_u8DutyCycle;
    uint8_t m_u8ForwardTxPowerOffset;
    uint8_t m_u8ReverseTxPowerOffset;
    uint16_t m_u16POSValidTime; // Entry validity time (in seconds)
};
```

Access: Read-only.

8.7.154 MAC_WRP_PIB_MANUF_ACK_TX_DELAY_RF (0x0800021C)

Time (in μ s) between end of reception and start of ACK transmission.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 2000.

8.7.155 MAC_WRP_PIB_MANUF_ACK_RX_WAIT_TIME_RF (0x0800021D)

Time (in μ s) to wait for an ACK reception after end of frame transmission before time-out.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 30000.

8.7.156 MAC_WRP_PIB_MANUF_ACK_CONFIRM_WAIT_TIME_RF (0x0800021E)

Time (in μ s) to wait for an ACK transmission confirm after TX request, before time-out.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 30000.

8.7.157 MAC_WRP_PIB_MANUF_DATA_CONFIRM_WAIT_TIME_RF (0x0800021F)

Time (in μ s) to wait for an Data Frame transmission confirm after TX request, before time-out.

Access: R/W.

Value Range: 0-4294967295.

Default Value: 90000.

8.7.158 MAC_WRP_PIB_MANUF_RF_IFACE_AVAILABLE (0x08000221)

RF Interface availability.

Access: Read only.

Value Range: True/False.

Default Value: N/A.

8.7.159 MAC_WRP_PIB_MANUF_LAST_FRAME_DURATION_RF (0x08000222)

Last PLC frame duration in ms.

Access: Read only.

Value Range: 0-65535.

Default Value: N/A.

8.7.160 MAC_WRP_PIB_MANUF_TRICKLE_MIN_LQI_RF (0x08000223)

Minimum LQI to consider a neighbour for Trickle.

Access: Write only.

Value Range: 0-255.

Default Value: 85.

8.7.161 MAC_WRP_PIB_MANUF_PHY_PARAM_RF (0x08000220)

Gives access to some RF PHY layer parameters and statistics. It is treated as a table, where the index determines which parameter to access. Available parameters and their corresponding indexes are the following:

MAC_WRP_RF_PHY_PARAM_DEVICE_ID, index 0x0000.

Description: RF device identifier. 16 bits.

Access: Read only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_FW_VERSION, index 0x0001.

Description: RF PHY layer firmware version number. 6 bytes.
{major, minor, revision, year, month, day}

Access: Read only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_DEVICE_RESET, index 0x0002.

Description: This IB performs an RF device reset by means of its Reset Pin. 8 bits.

Access: Write only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_TRX_RESET, index 0x0080.

Description: This IB performs an RF transceiver (RF09 or RF24) reset through an SPI command. It allows resetting only one of the two available transceivers. 8 bits.

Access: Write only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_TRX_SLEEP, index 0x0081.

Description: RF transceiver (RF09 or RF24) sleep. 8 bits.

Access: Write only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_PHY_CONFIG, index 0x0100.

Description: RF PHY configuration structure (see “at86rf_phy_cfg_t” in code).

Access: R/W.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_PHY_BAND_OPERATING_MODE, index 0x0101.

Description: RF PHY band and operating mode. 16 bits.

Access: R/W.

Default Value: AT86RF_SUN_FSK_BAND_863_OPM1.

MAC_WRP_RF_PHY_PARAM_PHY_CHANNEL_NUM, index 0x0120.

Description: RF channel number used for transmission and reception. 16 bits.

Access: R/W.

Default Value: 29 (866 MHz in default Band and Operating Mode).

MAC_WRP_RF_PHY_PARAM_PHY_CHANNEL_FREQ_HZ, index 0x0121.

Description: RF frequency in Hz used for transmission and reception. 32 bits.

Access: Read only.

Default Value: 866000000.

MAC_WRP_RF_PHY_PARAM_PHY_CCA_ED_CONFIG, index 0x0140.

Description: Configuration of Energy Detection for CCA. 3 bytes (see “at86rf_cca_ed_cfg_t” in code).
{duration, threshold}

Access: R/W.

Default Value: {160, -81}.

MAC_WRP_RF_PHY_PARAM_PHY_CCA_ED_DURATION, index 0x0141.

Description: Duration in us of Energy Detection for CCA. 16 bits.

Access: R/W.

Default Value: 160.

MAC_WRP_RF_PHY_PARAM_PHY_CCA_ED_THRESHOLD, index 0x0142.

Description: Threshold in dBm of for CCA with Energy Detection. 8 bits.

Access: R/W.

Default Value: -81.

MAC_WRP_RF_PHY_PARAM_PHY_TURNAROUND_TIME, index 0x0160.

Description: Turnaround time in us. 16 bits.

Access: Read only.

Default Value: 1000.

MAC_WRP_RF_PHY_PARAM_PHY_TX_PAY_SYMBOLS, index 0x0180.

Description: Number of payload symbols in last transmitted message. 16 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_PAY_SYMBOLS, index 0x0181.

Description: Number of payload symbols in last received message. 16 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_TOTAL, index 0x01A0.

Description: Successfully transmitted messages count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_TOTAL_BYTES, index 0x01A1.

Description: Successfully transmitted bytes count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_TOTAL, index 0x01A2.

Description: Total Transmission errors count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_BUSY_TX, index 0x01A3.

Description: Transmission errors count due to already in transmission. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_BUSY_RX, index 0x01A4.

Description: Transmission errors count due to already in reception. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_BUSY_CHN, index 0x01A5.

Description: Transmission errors count due to busy channel. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_BAD_LEN, index 0x01A6.

Description: Transmission errors count due to bad message length. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_BAD_FORMAT, index 0x01A7.

Description: Transmission errors count due to bad format. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_TIMEOUT, index 0x01A8.

Description: Transmission errors count due to timeout. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_ERR_ABORTED, index 0x01A9.

Description: Transmission aborted count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_TX_CFM_NOT_HANDLED, index 0x01AA.

Description: Transmission confirms not handled by upper layer count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_TOTAL, index 0x01B0.

Description: Successfully received messages count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_TOTAL_BYTES, index 0x01B1.

Description: Successfully received bytes count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_TOTAL, index 0x01B2.

Description: Total Reception errors count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_FALSE_POSITIVE, index 0x01B3.

Description: Reception false positive count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_BAD_LEN, index 0x01B4.

Description: Reception errors count due to bad message length. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_BAD_FORMAT, index 0x01B5.

Description: Reception errors count due to bad format or bad FCS in header. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_BAD_FCS_PAY, index 0x01B6.

Description: Reception errors count due to bad FCS in payload. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_ERR_ABORTED, index 0x01B7.

Description: Reception aborted count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_OVERRIDE, index 0x01B8.

Description: Reception override (another message with higher signal level) count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_RX_IND_NOT_HANDLED, index 0x01B9.

Description: Reception indications not handled by upper layer count. 32 bits.

Access: Read only.

Default Value: 0.

MAC_WRP_RF_PHY_PARAM_PHY_STATS_RESET, index 0x01C0.

Description: Reset Phy Statistics. 8 bits.

Access: Write only.

Default Value: N/A.

MAC_WRP_RF_PHY_PARAM_MAC_UNIT_BACKOFF_PERIOD, index 0x0200.

Description: Backoff period unit in us (aUnitBackoffPeriod in IEEE 802.15.4) used for CSMA-CA . 16 bits.

Access: Read only.

Default Value: 1160.

MAC_WRP_RF_PHY_PARAM_TX_FSK_FEC, index 0x8000.

Description: SUN FSK FEC enabled or disabled for transmission (phyFskFecEnabled in IEEE 802.15.4). 8 bits.

Access: R/W.

Default Value: 0 (Disabled).

MAC_WRP_RF_PHY_PARAM_TX_OFDM_MCS, index 0x8001.

Description: SUN OFDM MCS (Modulation and coding scheme) used for transmission. 8 bits.

Access: R/W.

Default Value: 0 (BPSK, 1/2 convolutional encoder rate, 4x frequency repetition).

8.8 ADP SAP

Functions detailed here (initialization and event handler) are not part of the G3 Spec.

These are functions specific to Microchip implementation of the G3 Stack. Layers above ADP are in charge of calling these functions properly.

8.8.1 ADP Initialization Function

Use this function to initialize the ADP layer. The ADP layer should be initialized before doing any other operation on the G3 Stack.

This function takes care of initialization of all lower layers.

```
void AdpInitialize(struct TAdpNotifications *pNotifications, enum TAdpBand band);
```

Parameters:

<i>pNotifications</i>	Structure with callbacks used to notify ADP specific events (if NULL the layer is de-initialized)
<i>band</i>	Working band (should be inline with the hardware)

```
struct TAdpNotifications {
    AdpDataConfirm fnctAdpDataConfirm;
    AdpDataIndication fnctAdpDataIndication;
    AdpDiscoveryConfirm fnctAdpDiscoveryConfirm;
    AdpDiscoveryIndication fnctAdpDiscoveryIndication;
    AdpNetworkStartConfirm fnctAdpNetworkStartConfirm;
    AdpResetConfirm fnctAdpResetConfirm;
    AdpSetConfirm fnctAdpSetConfirm;
    AdpMacSetConfirm fnctAdpMacSetConfirm;
    AdpGetConfirm fnctAdpGetConfirm;
    AdpMacGetConfirm fnctAdpMacGetConfirm;
    AdpRouteDiscoveryConfirm fnctAdpRouteDiscoveryConfirm;
    AdpPathDiscoveryConfirm fnctAdpPathDiscoveryConfirm;
    AdpNetworkStatusIndication fnctAdpNetworkStatusIndication;
    AdpBufferIndication fnctAdpBufferIndication;
    AdpPREQIndication fnctAdpPREQIndication;
    AdpUpdNonVolatileDataIndication fnctAdpUpdNonVolatileDataIndication;
    AdpRouteNotFoundIndication fnctAdpRouteNotFoundIndication;
};

enum TAdpBand {
    ADP_BAND_CENELEC_A = 0, ADP_BAND_CENELEC_B = 1, ADP_BAND_FCC = 2, ADP_BAND_ARIB = 3
};
```

8.8.2 ADP Event Handler Function

Event handler of the ADP layer. It must be called periodically, at the default 1ms rate or the one defined by the user as explained in [3.2.3.1. Time Control Initialization](#). This function calls the MacWrapperEventHandler function and the Event Handler call propagates through all lower layers. Returns *false* in case ADP Notification callbacks are not initialized, and *true* otherwise.

```
bool AdpEventHandler(void);
```

8.9 ADP Data Primitives

The ADPD is used to transport the NSDU to other devices on the network. Microchip implementation support all standard data primitives specified in G3-PLC Spec. The following chapters enumerate the ADPD functions.

All the functions and structures presented here are defined in the *AdpApi.h* and *AdpApiTypes.h* files which are available to the user. Refer to these files for details of structure contents and types.

8.9.1 ADPD-DATA.request

The ADPD-DATA.request primitive requests the transfer of an application PDU (i.e., IPv6 packet) to another device or multiple devices.

```
void AdpDataRequest(uint16_t u16NsduLength, const uint8_t *pNsdu, uint8_t u8NsduHandle, bool bDiscoverRoute, uint8_t u8QualityOfService);
```

Parameters:

<i>u16NsduLength</i>	The size of the NSDU, in bytes. [Valid range: 0-1280].
<i>pNsdu</i>	The NSDU to send.
<i>u8NsduHandle</i>	The handle of the NSDU to transmit. This parameter is used to identify in the <i>AdpDataConfirm</i> primitive which request it is concerned with. It can be randomly chosen by the application layer.
<i>bDiscoverRoute</i>	If TRUE, a route discovery procedure will be performed prior to sending the frame if a route to the destination is not available in the routing table. If FALSE, no route discovery is performed.
<i>u8QualityOfService</i>	The requested quality of service (QoS) of the frame to send. Allowed values are: [0x00 = normal priority; 0x01 = high priority].

When the execution of the *DataRequest* is finished, the callback *AdpDataConfirm* is called to inform the user about the status of the operation.

8.9.2 ADPD-DATA.confirm

The ADPD-DATA.confirm primitive reports the result of a previous ADPD-DATA.request primitive.

```
typedef void (*AdpDataConfirm)(struct TAdpDataConfirm *pDataConfirm);
```

The result is reported into the following structure:

```
struct TAdpDataConfirm {
    uint8_t m_u8Status;
    uint8_t m_u8NsduHandle;
};
```

Fields of the structure:

<i>u8Status</i>	The status code of a previous ADPD-DATA.request identified by its <i>NsduHandle</i> .
<i>u8NsduHandle</i>	The handle of the NSDU confirmed by this primitive.

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.9.3 ADPD-DATA.indication

The ADPD-DATA.indication primitive is used to transfer received data from the adaptation sublayer to the upper layer.

When user data is received by the modem, the callback *AdpDataIndication* is called:

```
typedef void (*AdpDataIndication)(struct TAdpDataIndication *pDataIndication);
```

The information is reported into the following structure:

```
struct TAdpDataIndication {
    uint16_t m_u16NsduLength;
    const uint8_t *m_pNsdu;
    uint8_t m_u8LinkQualityIndicator;
};
```

Fields of the structure:

<i>m_u16NsduLength</i>	The size of the NSDU, in bytes.
<i>m_pNsdu</i>	Pointer to received NSDU.
<i>m_u8LinkQualityIndicator</i>	The value of the link quality during the reception of the frame.

8.9.4 ADPD-NoIP-DATA.request

The ADPD-NoIP-DATA.request primitive requests the transfer of an application PDU, which is not an IPv6 packet, to another device or multiple devices.

```
void AdpNoIPDataRequest(uint16_t u16ApduLength, const uint8_t *pApdu, uint16_t u16DstAddr,
    uint8_t u8ApduHandle, bool bDiscoverRoute, uint8_t u8QualityOfService);
```

Parameters:

<i>u16ApduLength</i>	The size of the APDU, in bytes. [Valid range: 0-1280].
<i>pApdu</i>	The APDU to send; can be any App protocol.
<i>u16DstAddr</i>	The Destination Short Address. Can be a unicast address, a 6LowPAN Group Address, or the 6LowPAN BROADCAST ADDRESS (0x8001)
<i>u8ApduHandle</i>	The handle of the APDU to transmit. This parameter is used to identify in the AdpDataConfirm primitive which request it is concerned with. It can be randomly chosen by the application layer.
<i>bDiscoverRoute</i>	If TRUE, a route discovery procedure will be performed prior to sending the frame if a route to the destination is not available in the routing table. If FALSE, no route discovery is performed.
<i>u8QualityOfService</i>	The requested quality of service (QoS) of the frame to send. Allowed values are: [0x00 = normal priority; 0x01 = high priority].

When the execution of the NoIPDataRequest is finished, the callback *AdpDataConfirm* is called to inform the user about the status of the operation.

8.10 ADP Management Service Primitives

The ADPM allows the transport of command frames used for network maintenance. The following chapters enumerate the ADPM functions.

All the functions and structures presented here are defined in files *AdpApi.h* and *AdpApiTypes.h*, available to the user.

Refer to these files for details of structure contents and types.

8.10.1 ADPM-DISCOVERY.request

The ADPM-DISCOVERY.request primitive allows the upper layer to request the ADPM to scan for networks operating in its POS.

```
void AdpDiscoveryRequest(uint8_t u8Duration);
```

Parameter:

<i>u8Duration</i>	The number of seconds the scan shall last.
-------------------	--

8.10.2 ADPM-DISCOVERY.indication

The ADPM-Discovery.indication primitive is generated by the ADP layer upon Beacon reception. This primitive will be generated as many times as number of Beacons are received, thus, it is an application task to decide which PAN to join, depending on the info received on each of the indications.

```
typedef void (*AdpDiscoveryIndication) (struct TAdpPanDescriptor *pPanDescriptor);
```

The information is reported into the following structure:

```
struct TAdpPanDescriptor {
    uint16_t m_u16PanId;
    uint8_t m_u8LinkQuality;
    uint16_t m_u16LbaAddress;
    uint16_t m_u16RcCoord;
    uint8_t m_u8MediaType;
};
```

Fields of the structure:

<i>u16PanId</i>	PANId where Beacon comes from.
<i>u8LinkQuality</i>	Quality of Beacon reception.
<i>m_u16LbaAddress</i>	Short address of device sending the Beacon.
<i>m_u16RcCoord</i>	Route Cost to Coordinator reported by Device sending the Beacon.
<i>m_u8MediaType</i>	MediaType through which the Beacon was received. Will be further used when calling 8.10.6. ADPM-NETWORK-JOIN.request primitive.

8.10.3 ADPM-DISCOVERY.confirm

The ADPM-DISCOVERY.confirm primitive is generated by the ADP layer upon completion of a previous ADPM-DISCOVERY.request.

```
typedef void (*AdpDiscoveryConfirm) (uint8_t m_u8Status);
```

Parameter:

<i>m_u8Status</i>	Result of the associated DISCOVERY.request.
-------------------	---

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.4 ADPM-NETWORK-START.request

The ADPM-NETWORK-START.request primitive allows the upper layer to request the starting of a new network. It shall only be invoked by a device designated as the PAN coordinator during the factory process.

```
void AdpNetworkStartRequest(uint16_t u16PanId);
```

Parameter:

<i>u16PanId</i>	The PANId of the network to create; determined at the application level [Valid range: 0x0000 - 0xFFFF. Note: value must be ended with 0xFCFF according to G3 spec].
-----------------	---

8.10.5 ADPM-NETWORK-START.confirm

The ADPM-NETWORK-START.confirm primitive reports the status of an ADPM-NETWORK-START.request.

```
typedef void (*AdpNetworkStartConfirm) (struct TAdpNetworkStartConfirm *pNetworkStartConfirm);
```

Structure:

```
struct TAdpNetworkStartConfirm {
    uint8_t m_u8Status;
};
```

Parameter:

m_u8Status The result of Network Start request.

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.6 ADPM-NETWORK-JOIN.request

The ADPM-NETWORK-JOIN.request primitive allows the upper layer to join an existing network.

```
void AdpNetworkJoinRequest(uint16_t u16PanId, uint16_t u16LbaAddress, uint8_t u8MediaType);
```

Parameters:

u16PanId The 16-bit PAN identifier of the network to join.
u16LbaAddress The 16-bit short address of the device acting as a LoWPAN bootstrap agent as defined in Annex E of G3 Spec.
u8MediaType The Media through which the Joining procedure will be triggered.

8.10.7 ADPM-NETWORK-LEAVE.request

The ADPM-NETWORK-LEAVE.request primitive allows a non-coordinator device to remove itself from the network.

```
void AdpNetworkLeaveRequest(void);
```



Important: Regardless of the result of this request, the G3 stack will reset internally before calling ADPM-NETWORK-LEAVE.confirm.

8.10.8 ADPM-RESET.request

The ADPM-RESET.request primitive performs a reset of the adaptation sublayer and allows the resetting of the MIB attributes.

```
void AdpResetRequest(void);
```

8.10.9 ADPM-RESET.confirm

The ADPM-RESET.confirm primitive allows the upper layer to be notified of the completion of an ADPM-RESET.request primitive.

```
typedef void (*AdpResetConfirm)(struct TAdpResetConfirm *pResetConfirm);
```

Structure:

```
struct TAdpResetConfirm {
    uint8_t m_u8Status;
};
```

Field:

m_u8Status The status of the request.

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.10 ADPM-GET.request

The ADPM-GET.request primitive allows the upper layer to get the value of an attribute from the ADP information base.

```
void AdpGetRequest(uint32_t u32AttributeId, uint16_t u16AttributeIndex);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute to read.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute to read. This parameter is valid only for IB attributes that are tables.

A synchronous implementation of this function is also available, thus no confirm is generated, instead a pointer to confirm the structure has to be provided.

```
void AdpGetRequestSync(uint32_t u32AttributeId, uint16_t u16AttributeIndex, struct TAdpGetConfirm *pGetConfirm);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute to read.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute to read. This parameter is valid only for IB attributes that are tables.
<i>pGetConfirm</i>	Pointer to confirm structure to get the result. See structure definition in confirm primitive definition below.

8.10.11 ADPM-GET.confirm

The ADPM-GET.confirm primitive allows the upper layer to be informed of the status of a previously issued ADPM-GET.request primitive.

```
typedef void (*AdpGetConfirm)(struct TAdpGetConfirm *pGetConfirm);
```

Structure:

```
struct TAdpGetConfirm {
    uint8_t m_u8Status;
    uint32_t m_u32AttributeId;
    uint16_t m_u16AttributeIndex;
    uint8_t m_u8AttributeLength;
    uint8_t m_au8AttributeValue[64];
};
```

Fields:

<i>m_u8Status</i>	The status of the request.
<i>m_u32AttributeId</i>	The identifier of the IB attribute read.
<i>m_u16AttributeIndex</i>	The index within the table of the specified IB attribute to read. This parameter is valid only for IB attributes that are tables.
<i>m_u8AttributeLength</i>	The length of the value of the attribute read from the IB.
<i>m_au8AttributeValue</i>	The value of the attribute read from the IB.

Possible values of *m_u8Status* parameter are:

<i>G3_SUCCESS</i>	Value 0x00
<i>G3_INVALID_PARAMETER</i>	Value 0xA8
<i>G3_INVALID_INDEX</i>	Value 0xA7
<i>G3_UNSUPPORTED_ATTRIBUTE</i>	Value 0xB1

8.10.12 ADPM-MAC.GET.request

The ADPM-MAC.GET.request primitive allows the upper layer to get the value of an attribute from the MAC information base. The upper layer cannot directly access the MAC layer while ADP is running.

```
void AdpMacGetRequest(uint32_t u32AttributeId, uint16_t u16AttributeIndex);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the MAC IB attribute to read.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute to read. This parameter is valid only for IB attributes that are tables.

A synchronous implementation of this function is also available, thus no confirm is generated; instead, a pointer to confirm structure has to be provided.

```
void AdpMacGetRequestSync (uint32_t u32AttributeId, uint16_t u16AttributeIndex, struct TAdpMacGetConfirm *pGetConfirm);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute to read.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute to read. This parameter is valid only for IB attributes that are tables.
<i>pGetConfirm</i>	Pointer to confirm structure to get the result. See structure definition in confirm primitive definition below.

8.10.13 ADPM-MAC.GET.confirm

The ADPM-MAC.GET.confirm primitive allows the upper layer to be informed of the status of a previously issued ADPM-MAC.GET.request primitive.

```
typedef void (*AdpMacGetConfirm)(struct TAdpMacGetConfirm *pGetConfirm);
```

Structure:

```
struct TAdpMacGetConfirm {
    uint8_t m_u8Status;
    uint32_t m_u32AttributeId;
    uint16_t m_u16AttributeIndex;
    uint8_t m_u8AttributeLength;
    uint8_t m_a8AttributeValue[MAC_PIB_MAX_VALUE_LENGTH];
};
```

Fields:

<i>m_u8Status</i>	The status of the request.
<i>m_u32AttributeId</i>	The identifier of the IB attribute read.
<i>m_u16AttributeIndex</i>	The index within the table of the specified IB attribute read.

<i>m_u8AttributeLength</i>	The length of the value of the attribute read from the IB.
<i>m_u8AttributeValue</i>	The value of the attribute read from the IB.

Possible values of *m_u8Status* parameter are:

<i>MAC_STATUS_SUCCESS</i>	Value 0x00
<i>MAC_STATUS_UNSUPPORTED_ATTRIBUTE</i>	Value 0xF4
<i>MAC_STATUS_INVALID_INDEX</i>	Value 0xF9
<i>MAC_STATUS_UNAVAILABLE_KEY</i>	Value 0xF3
<i>MAC_STATUS_DENIED</i>	Value 0xE2
<i>MAC_STATUS_INVALID_PARAMETER</i>	Value 0xE8

8.10.14 ADPM-SET.request

The ADPM-SET.request primitive allows the upper layer to set the value of an attribute in the ADP information base.

```
void AdpSetRequest(uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLength, const uint8_t *pu8AttributeValue);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute set.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute.
<i>u8AttributeLength</i>	The length of the value of the attribute to set.
<i>pu8AttributeValue</i>	Pointer to the value of the attribute to set.

A synchronous implementation of this function is also available, thus no confirm is generated, instead a pointer to confirm structure has to be provided.

```
void AdpSetRequestSync(uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLength, const uint8_t *pu8AttributeValue, struct TAdpSetConfirm *pSetConfirm);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute set.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute.
<i>u8AttributeLength</i>	The length of the value of the attribute to set.
<i>pu8AttributeValue</i>	Pointer to the value of the attribute to set.
<i>pSetConfirm</i>	Pointer to confirm structure to get the result. See structure definition in confirm primitive definition below.

8.10.15 ADPM-SET.confirm

The ADPM-SET.confirm primitive allows the upper layer to be informed about a previous ADPM-SET.request primitive.

```
typedef void (*AdpSetConfirm)(struct TAdpSetConfirm *pSetConfirm);
```

Structure:

```
struct TAdpSetConfirm {
    uint8_t m_u8Status;
    uint32_t m_u32AttributeId;
    uint16_t m_u16AttributeIndex;
};
```

Fields:

<i>m_u8Status</i>	The status of the scan request.
<i>m_u32AttributeId</i>	The identifier of the IB attribute set.
<i>m_u16AttributeIndex</i>	The index within the table of the specified IB attribute.

Possible values of *m_u8Status* parameter are:

<i>G3_SUCCESS</i>	Value 0x00
<i>G3_INVALID_PARAMETER</i>	Value 0xA8
<i>G3_INVALID_INDEX</i>	Value 0xA7
<i>G3_UNSUPPORTED_ATTRIBUTE</i>	Value 0xB1
<i>G3_READ_ONLY</i>	Value 0xB0
<i>G3_FAILED</i>	Value 0xA2

8.10.16 ADPM-MAC.SET.request

The ADPM-MAC.SET.request primitive allows the upper layer to set the value of an attribute in the MAC information base. The upper layer cannot directly access the MAC layer while ADP is running.

```
void AdpMacSetRequest(uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLength, const uint8_t *pu8AttributeValue);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute set.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute.
<i>u8AttributeLength</i>	The length of the value of the attribute to set.
<i>pu8AttributeValue</i>	The value of the attribute to set.

A synchronous implementation of this function is also available, thus no confirm is generated, instead a pointer to confirm structure has to be provided.

```
void AdpMacSetRequestSync (uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLength, const uint8_t *pu8AttributeValue, struct TAdpMacSetConfirm *pSetConfirm);
```

Parameters:

<i>u32AttributeId</i>	The identifier of the ADP IB attribute set.
<i>u16AttributeIndex</i>	The index within the table of the specified IB attribute.
<i>u8AttributeLength</i>	The length of the value of the attribute to set.
<i>pu8AttributeValue</i>	Pointer to the value of the attribute to set.
<i>pSetConfirm</i>	Pointer to confirm structure to get the result. See structure definition in confirm primitive definition below.

8.10.17 ADPM-MAC.SET.confirm

The ADPM-MAC.SET.confirm primitive allows the upper layer to be informed about a previous ADPM-MAC.SET.request primitive.

```
typedef void (*AdpMacSetConfirm)(struct TAdpMacSetConfirm *pSetConfirm);
```

Structure:

```
struct TAdpMacSetConfirm {
    uint8_t m_u8Status;
    uint32_t m_u32AttributeId;
    uint16_t m_u16AttributeIndex;
```

```
};
```

Fields:

<i>m_u8Status</i>	The status of the request.
<i>m_u32AttributeId</i>	The identifier of the IB attribute set.
<i>m_u16AttributeIndex</i>	The index within the table of the specified IB attribute.

Possible values of *m_u8Status* parameter are:

<i>MAC_STATUS_SUCCESS</i>	Value 0x00
<i>MAC_STATUS_UNSUPPORTED_ATTRIBUTE</i>	Value 0xF4
<i>MAC_STATUS_INVALID_INDEX</i>	Value 0xF9
<i>MAC_STATUS_READ_ONLY</i>	Value 0xFB
<i>MAC_STATUS_INVALID_PARAMETER</i>	Value 0xE8

8.10.18 ADPM-NETWORK-STATUS.indication

The ADPM-NETWORK-STATUS.indication primitive allows the next higher layer of a PAN coordinator or a coordinator to be notified when a particular event occurs on the PAN.

```
typedef void (*AdpNetworkStatusIndication) (struct TAdpNetworkStatusIndication
*pNetworkStatusIndication);
```

Structure:

```
struct TAdpNetworkStatusIndication {
    uint16_t m_u16PanId;
    struct TAdpAddress m_SrcDeviceAddress;
    struct TAdpAddress m_DstDeviceAddress;
    uint8_t m_u8Status;
    uint8_t m_u8SecurityLevel;
    uint8_t m_u8KeyIndex;
    uint8_t m_u8MediaType;
};
```

8.10.19 ADPM-BUFFER.indication

The ADPM-BUFFER.indication primitive allows the upper layer to get info about the availability of buffers to hold a data transmission.

```
typedef void (*AdpBufferIndication) (TAdpBufferIndication *pBufferIndication);
```

Parameter:

```
typedef union {
    struct {
        uint8_t m_u8LargeBuffersAvailable : 1;
        uint8_t m_u8MediumBuffersAvailable : 1;
        uint8_t m_u8SmallBuffersAvailable : 1;
        /* Reserved */
        uint8_t : 5;
    };
    uint8_t m_u8BufferIndicationBitmap;
} TAdpBufferIndication;
```

This union represents a bitmap of buffers availability. The buffer availability can be accessed individually for Large, Medium and Small buffers using the structure defined in the TAdpBufferIndication union.

8.10.20 ADPM-PREQ.indication

The AdpPREQIndication primitive allows the next higher layer to be notified when a PREQ frame is received in Unicast mode with an originator address equal to the coordinator address, and with a destination address equal to the device address.

```
typedef void (*AdpPREQIndication) (void);
```

This primitive is generated by ADP layer in order to indicate upper layer that the device has been addressed by means of a PREQ frame.

8.10.21 ADPM-UPD_NON_VOLATILE_DATA.indication

The AdpUpdNonVolatileDataIndication primitive allows the next higher layer to be notified when any of the parameters to be saved/restored before/after a reset change; this way, the upper layer can maintain a record of these parameters that is always updated.

```
typedef void (*AdpUpdNonVolatileDataIndication) (void);
```

It is up to the upper layer how to handle the change of these parameters, and their storage if necessary.

Refer to the [3.4. Persistent Data Storage](#) for more details.

8.10.22 ADPM-ROUTE_NOT_FOUND.indication

The AdpRouteNotFoundIndication primitive is used to indicate the upper layer that a route is not available when a frame is to be sent. Absence of a route may be because there is no entry in the routing table when a data request comes from the upper layer, there is no entry in the routing table when a received frame has to be forwarded (in Unicast Addressing mode), or because a route has just broken due to no ACK reception for the frame being transmitted. This way, the upper layer knows that a route discover or repair process will be triggered to find a route to send such frame (always that IB configuration allows for triggering route discover/repair, and by default it does).

```
typedef void (*AdpRouteNotFoundIndication) (struct TAdpRouteNotFoundIndication
*pRouteNotFoundIndication);
```

Structure:

```
struct TAdpRouteNotFoundIndication{
    uint16_t m_u16SrcAddr;
    uint16_t m_u16DestAddr;
    uint16_t m_u16NextHopAddr;
    uint16_t m_u16PreviousHopAddr;
    uint16_t m_u16RouteCost;
    uint8_t m_u8HopCount;
    uint8_t m_u8WeakLinkCount;
    bool m_bRouteJustBroken;
    bool m_bCompressedHeader;
    uint16_t m_u16NsduLength;
    uint8_t *m_pNsdu;
};
```

Field:

<i>m_u16SrcAddr</i>	Originator address of the frame which found no route to be sent.
<i>m_u16DestAddr</i>	Final destination address of the frame which found no route to be sent.
<i>m_u16NextHopAddr</i>	If there was a route and it is just broken, next hop in broken route. In case there was no previous route, reported value is 0xFFFF.

<i>m_u16PreviousHopAddr</i>	Next hop in route to originator address. In case there is no route, reported value is 0xFFFF.
<i>m_u16RouteCost</i>	If there was a route and it is just broken, route cost of broken route. In case there was no previous route, reported value is 0xFFFF.
<i>m_u8HopCount</i>	If there was a route and it is just broken, hop count of broken route. In case there was no previous route, reported value is 0xFF.
<i>m_u8WeakLinkCount</i>	If there was a route and it is just broken, weak link count of broken route. In case there was no previous route, reported value is 0xFF.
<i>m_bRouteJustBroken</i>	Indicates whether there is no route because it is just broken (No ACK reception), or because there is no previous route.
<i>m_bCompressedHeader</i>	Indicates whether frame needing route, which is reported in this indication, has a compressed IPv6 header or a complete one.
<i>m_u16NsduLength</i>	Length of the frame needing route to be sent.
<i>*m_pNsdu</i>	Pointer to frame needing route to be sent.

8.10.23 ADPM-ROUTE-DISCOVERY.request

The ADPM-ROUTE-DISCOVERY.request primitive allows the upper layer to initiate a route discovery.

```
void AdpRouteDiscoveryRequest(uint16_t u16DstAddr, uint8_t u8MaxHops);
```

Parameters:

<i>u16DstAddr</i>	The short unicast destination address of the route discovery.
<i>u8MaxHops</i>	This parameter indicates the maximum number of hops allowed for the route discovery [Valid range: 0x01 - 0x0E].

8.10.24 ADPM-ROUTE-DISCOVERY.confirm

The ADPM-ROUTE-DISCOVERY.confirm primitive allows the upper layer to be informed about a previous ADPM-ROUTE-DISCOVERY.request primitive.

```
typedef void (*AdpRouteDiscoveryConfirm)(struct TAdpRouteDiscoveryConfirm  
*pRouteDiscoveryConfirm);
```

Structure:

```
struct TAdpRouteDiscoveryConfirm {  
    uint8_t m_u8Status;  
};
```

Field:

<i>m_u8Status</i>	The status of the route discovery.
-------------------	------------------------------------

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.25 ADPM-PATH-DISCOVERY.request

The ADPM-PATH-DISCOVERY.request primitive allows the upper layer to initiate a path discovery.

```
void AdpPathDiscoveryRequest(uint16_t u16DstAddr, uint8_t u8MetricType);
```

Parameters:

<i>u16DstAddr</i>	The short unicast destination address of the path discovery.
<i>u8MetricType</i>	The metric type to be used for the path discovery. (Range: 0x00 - 0x0F).

8.10.26 ADPM-PATH-DISCOVERY.confirm

The ADPM-PATH-DISCOVERY.confirm primitive allows the upper layer to be informed of the completion of a path discovery (ADPM-PATH-DISCOVERY.request).

```
typedef void (*AdpPathDiscoveryConfirm)(struct TAdpPathDiscoveryConfirm
*pPathDiscoveryConfirm);
```

Structure:

```
struct TAdpPathDiscoveryConfirm {
    uint8_t m_u8Status;
    uint16_t m_u16DstAddr;
    uint16_t m_u16OrigAddr;
    uint8_t m_u8MetricType;
    uint8_t m_u8ForwardHopsCount;
    uint8_t m_u8ReverseHopsCount;
    struct THopDescriptor m_aForwardPath[16];
    struct THopDescriptor m_aReversePath[16];
};
```

Fields:

<i>m_u8Status</i>	The status of the path discovery (status can be INCOMPLETE and the other parameters contain the discovered path).
<i>m_u16DstAddr</i>	The short unicast destination address of the path discovery.
<i>m_u16Originator</i>	The originator of the path reply.
<i>m_u8PathMetricType</i>	Path metric type.
<i>m_u8HopsCount</i>	Number of path hops.
<i>m_pau16HopAddressTable</i>	Table with the address of each hop (according to m_u8HopsCount).
<i>m_pau16HopAddressTable</i>	Table with mns field of each hop (according to m_u8HopsCount).
<i>m_pau8LinkCostTable</i>	Table with cost information related to each hop (according to m_u8HopsCount).

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.27 ADPM-LBP.request

The ADPM-LBP.request primitive allows the upper layer of the client to send the LBP message to the server modem.

This primitive is used by the LBP module and it is not intended to be used by Application layer.

```
void AdpLbpRequest(const struct TAdpAddress *pDstAddr, uint16_t u16NsduLength, uint8_t
*pNsdu, uint8_t u8NsduHandle, uint8_t u8MaxHops, bool bDiscoveryRoute, uint8_t
u8QualityOfService, bool bSecurityEnable);
```

Parameters:

<i>pDstAddr</i>	16-bit address of LBA or LBD or 64-bit address (extended address of LBD).
<i>u16NsduLength</i>	The size of the NSDU, in bytes.
<i>pNsdu</i>	The NSDU to send.
<i>u8NsduHandle</i>	The handle of the NSDU to transmit. This parameter is used to identify in the AdpLbpConfirm primitive which request is concerned. It can be randomly chosen by the application layer.
<i>u8MaxHops</i>	The number of times the frame will be repeated by network routers.

<i>bDiscoveryRoute</i>	If TRUE, a route discovery procedure will be performed prior to sending the frame if a route to the destination is not available in the routing table. If FALSE, no route discovery is performed.
<i>u8QualityOfService</i>	The requested quality of service (QoS) of the frame to send. Allowed values are: • 0x00 = standard priority • 0x01 = high priority
<i>bSecurityEnable</i>	If TRUE, this parameter enables the MAC layer security for sending the frame.

8.10.28 ADPM-LBP.confirm

The ADPM-LBP.confirm primitive reports the result of a previous ADPM-LBP.request primitive.

This callback is set and captured by the LBP module and thus it is not part of the Notifications to be set by Application layer.

```
typedef void (*AdpLbpConfirm)(struct TAdpLbpConfirm *pLbpConfirm);
```

Structure:

```
struct TAdpLbpConfirm {
    uint8_t m_u8Status;
    uint8_t m_u8NsduHandle;
};
```

Fields:

<i>m_u8Status</i>	The status code of a previous AdpLbpRequest identified by its NsduHandle.
<i>m_u8NsduHandle</i>	The handle of the NSDU confirmed by this primitive.

Status codes are detailed in [8.12. ADP Status Codes Description](#).

8.10.29 ADPM-LBP.indication

The ADPM-LBP.indication primitive is used to transfer a received LBP frame from the ADP layer to the upper layer.

This callback is set and captured by the LBP module and thus it is not part of the Notifications to be set by Application layer.

```
typedef void (*AdpLbpIndication)(struct TAdpLbpIndication *pLbpIndication);
```

Structure:

```
struct TAdpLbpIndication {
    uint16_t m_u16SrcAddr;
    uint16_t m_u16NsduLength;
    uint8_t *m_pNsdu;
    uint8_t m_u8LinkQualityIndicator;
    bool m_bSecurityEnabled;
};
```

Fields:

<i>m_SrcAddr</i>	16 bits final destination address.
<i>m_u16NsduLength</i>	The size of the NSDU, in bytes; Range: 0 – 1280.
<i>m_pNsdu</i>	The NSDU received.

<i>m_u8LinkQualityIndicator</i>	The value of the link quality during reception of the frame.
<i>m_bSecurityEnabled</i>	TRUE if the frame was received with a security level greater or equal to <i>adpSecurityLevel</i> , FALSE otherwise.

8.11 ADPIB Objects Specification and Access

Microchip G3-PLC stacks supports all the information base (IB) attributes of the Adaptation Sublayer defined in the G3-PLC spec. The default endianness of all ADPIB is little endian; in other cases it will be explicitly specified.

In addition, there are some Microchip proprietary IB attributes of the Adaptation Sublayer with extra functionalities. All of them are described here.

8.11.1 ADP_IB_SECURITY_LEVEL = 0x00000000

This attribute indicates the minimum security level to be used for incoming and outgoing adaptation frames. Messages not respecting this level are dropped. The size of the attribute is 1 byte. The default value is 5 (ENC-MIC-32).

8.11.2 ADP_IB_PREFIX_TABLE = 0x00000001

This attribute contains the list of prefixes defined on this PAN. It is assumed that the link local IPv6 address exists independently and is not affected by the prefixes defined in the prefix table.

This attribute is a table of two elements.

Each element is encoded as following:

- Byte 0: indicates the prefix length in bits
- Byte 1: indicates the “on-link flag” (0 = not set, 1 = set)
- Byte 2: indicates the “autonomous address-configuration flag” (0 = not set, 1 = set)
- Bytes 3 - 6: indicate the valid lifetime in seconds
- Bytes 7 - 10: indicate the preferred lifetime in seconds
- Bytes 11 - 26 (variable length): indicate the prefix value

If the attribute length is set to 0, the prefix is deleted.

8.11.3 ADP_IB_BROADCAST_LOG_TABLE_ENTRY_TTL = 0x00000002

This attribute indicates the maximum time to live of an *adpBroadcastLogTable* entry (in minutes). The size of the attribute is 2 bytes. The value of the attribute is a 16 bits integer encoded little endian (LSB first). (e.g. 0x0200 = 2 minutes). Default value is 2 minutes.

8.11.4 ADP_IB_METRIC_TYPE = 0x00000003

This attribute defines the metric type to be used for routing purposes. The size of the attribute is 1 byte. The default value is 0x0F.

Possible values for this IB are:

- 0x00: TYPE_HOP_COUNT
- 0x0E: TYPE_METRIC_CAPACITY
- 0x0F: TYPE_METRIC_COMPOSITE

8.11.5 ADP_IB_LOW_LQI_VALUE = 0x00000004

The low LQI value defines the LQI value, used in metric computation, below which a link to a neighbour is considered as an unreliable link. The size of the attribute is 1 byte. The default value is 0x00.

8.11.6 ADP_IB_HIGH_LQI_VALUE = 0x00000005

The high LQI value defines the LQI value, used in metric computation, above which a link to a neighbour is considered the most reliable. The size of the attribute is 1 byte. The default value is 0xFF.

8.11.7 ADP_IB_RREP_WAIT = 0x00000006

An RREP shall be generated after a delay of adpRREPWait seconds after either the arrival of the first RREQ or the transmission of the latest RREP. The size of the attribute is 1 byte. The default value is 4.

8.11.8 ADP_IB_CONTEXT_INFORMATION_TABLE = 0x00000007

This attribute contains the context information associated to each CID extension field. This attribute is a table of 16 elements. The CID field from the ICMPv6 / 6CO message indicates the index in the table.

Each element is encoded as following:

- Bytes 0 - 1: indicate the valid time in minutes. The value is a 16 bits integer encoded little endian (LSB first)
- Byte 2: indicates the validity of the CID for compression purposes (corresponds to C field from the ICMPv6 / 6CO message). A value of 0 means false, 1 means true
- Byte 3: indicates the context length in bits
- Bytes 4 - 19 (variable length): indicate the context value

If the attribute length is set to 0 the context is deleted.

This table is not ordered using any criteria and entries can be added at any index position, thus to get all the active entries, the whole table has to be read, or iterate until [8.11.85. ADP_IB_MANUF_CONTEXT_INFORMATION_TABLE_COUNT = 0x080000E1](#) entries have been found.

8.11.9 ADP_IB_COORD_SHORT_ADDRESS = 0x00000008

This attribute defines the short address of the coordinator. The size of the attribute is 2 bytes. The value is a 16 bits integer encoded little endian (LSB first). The default value is 0x0000.

On Coordinator, this value is used to set its own Short Address when [8.10.4. ADPM-NETWORK-START.request](#) primitive is called.

On Devices, it is used to address frames targeted to PAN Coordinator.

8.11.10 ADP_IB_RLC_ENABLED = 0x00000009

This attribute controls the sending of RLCREQ frame by the device. The size of the attribute is 1. The value 0 indicates mechanism is disabled, value 1 mechanism enabled. The default value is 0.

8.11.11 ADP_IB_ADD_REV_LINK_COST = 0x0000000A

This attribute represents an additional cost to take into account a possible asymmetry in the link. The size of the attribute is 1 byte. The default value is 0.

8.11.12 ADP_IB_BROADCAST_LOG_TABLE = 0x0000000B

This attribute contains the broadcast log table. This attribute is read-only. The table contains 20 elements. Each element is 5 bytes in length:

- Bytes 0 - 1: indicate the source address of the broadcast packet (the address of the broadcast initiator). The address is encoded little endian (LSB first).
- Byte 2: indicates the sequence number.
- Bytes 3 - 4: indicate the remaining time in minutes until this entry in the broadcast log table is considered valid (encoded little endian, LSB first).

This table is not ordered using any criteria, therefore to get all the active entries the whole table must be read, or iterate until [8.11.84. ADP_IB_MANUF_BROADCAST_LOG_TABLE_COUNT = 0x080000E0](#) entries have been found.

8.11.13 ADP_IB_ROUTING_TABLE = 0x0000000C

This attribute contains the routing table.

The routing table default sizes are 150 elements for Devices, 400 for Coordinator Lite and 2000 for Full Coordinator. Anyway the Table size is configurable for each project by means of the *conf_tables.h* file.

Each element has 10 bytes (all integers are encoded little endian, LSB first):

- Bytes 0 - 1: indicate the destination address
- Bytes 2 - 3: indicate the next hop address
- Bytes 4 - 5: indicate the route cost
- Byte 6, bits 0 - 3: indicate the weak links count
- Byte 6, bits 4 - 7: indicate the hop count
- Byte 7: indicate the MediaType use to reach next hop (0 PLC, 1 RF)
- Bytes 8 - 9: indicate the time this entry is valid in minutes

This attribute can also be written for testing purposes.

This table is ordered according to valid time. Newest entries are placed at the top of the table.

To get all the active entries, the first step is to get the number of such active entries by means of [8.11.61. ADP_IB_MANUF_ROUTING_TABLE_COUNT = 0x080000C8](#). Then, since the table is ordered, all entries can be obtained by iterating from 0 to the number of active entries.

8.11.14 ADP_IB_UNICAST_RREQ_GEN_ENABLE = 0x0000000D

This attribute controls the unicast RREQ generation. The length of the attribute is 1 byte. Value 0 indicates the mechanism is disabled, value 1 the mechanism is enabled. The default value is 1 (enabled).

8.11.15 ADP_IB_GROUP_TABLE = 0x0000000E

This attribute contains the group addresses to which the device belongs. The table size is 16. Note that the "All nodes address", group 0x8001 is predefined and it doesn't need to be added to this attribute.

The length of each element is 2 bytes, defining the group address encoded little endian (lsb first). If the attribute element is set with the length 0, the group is deleted.

When reading the element, an extra byte is added indicating if the entry is valid or not: 0 means invalid, 1 means valid.

This table is not ordered using any criteria, thus to get all the active entries the whole table has to be read, or iterate until [8.11.86. ADP_IB_MANUF_GROUP_TABLE_COUNT = 0x080000E2](#) entries have been found.

8.11.16 ADP_IB_MAX_HOPS = 0x0000000F

This attribute defines the maximum number of hops to be used by the routing algorithm. The length of the attribute is 1 byte. The default value is 8.

8.11.17 ADP_IB_DEVICE_TYPE= 0x00000010

This attribute defines the type of device connected to the modem. The length of the attribute is 1 byte.

- 0: PAN device
- 1: PAN coordinator
- 2: Not defined (Default)

8.11.18 ADP_IB_NET_TRAVERSAL_TIME = 0x00000011

This attribute defines the maximum time in seconds that a packet is expected to take to reach any node from any node. The length of the attribute is 1 byte. The default value is 20.

8.11.19 ADP_IB_ROUTING_TABLE_ENTRY_TTL = 0x00000012

This attribute defines the maximum time-to-live of a routing table entry (in minutes). The length of the attribute is 2 bytes. The value is encoded little endian. Default value: 360.

8.11.20 ADP_IB_KR = 0x00000013

This attribute defines a weight factor for robust mode to calculate link cost. The length of the attribute is 1 byte. The default value is 0.

8.11.21 ADP_IB_KM = 0x00000014

This attribute defines a weight factor for modulation to calculate link cost. The length of the attribute is 1 byte. The default value is 0.

8.11.22 ADP_IB_KC = 0x00000015

This attribute defines a weight factor for number of active tones to calculate link cost. The length of the attribute is 1 byte. The default value is 0.

8.11.23 ADP_IB_KQ = 0x00000016

This attribute defines a weight factor for LQI to calculate route cost. The length of the attribute is 1 byte. Default value: 10.

8.11.24 ADP_IB_KH = 0x00000017

This attribute defines a weight factor for hop to calculate link cost. The length of the attribute is 1 byte. Default value is 4.

8.11.25 ADP_IB_RREQ_RETRIES = 0x00000018

This attribute defines the number of RREQ retransmissions in case of a RREP reception time out. The length of the attribute is 1 byte. The default value is 0.

8.11.26 ADP_IB_RREQ_WAIT = 0x00000019

This attribute defines the time in seconds to wait between two consecutive RREQ generations. The length is 1 byte. The default value is 30.

8.11.27 ADP_IB_WEAK_LQI_VALUE = 0x0000001A

This attribute indicates the weak link value which defines the LQI value below which a link to a neighbour is considered as a weak link. The length is 1 byte. The default value is 52 (SNR of 3 dB).

8.11.28 ADP_IB_KRT = 0x0000001B

This attribute indicates a weight factor for the number of active routes in the routing table to calculate link cost. The length is 1 byte. The default value is 0.

8.11.29 ADP_IB_SOFT_VERSION = 0x0000001C

This attribute indicates the software version. This is a read-only attribute with a length of 6 bytes:

- Byte 0: major version number

- Byte 1: minor version number
- Byte 2: revision number
- Byte 3: version date: year (since 2000)
- Byte 4: version date: month
- Byte 5: version date: day

8.11.30 ADP_IB_SNIFFER_MODE = 0x0000001D

Note: This attribute is not supported in the current version of the stack.

Sniffer mode at ADP level activation/deactivation. The length of the attribute is 1 byte.

- 0: Disabled (Default)
- 1: Enabled

8.11.31 ADP_IB_BLACKLIST_TABLE = 0x0000001E

This attribute contains the list of the blacklisted neighbours.

The size of the table is 20 elements by default. This size can be modified if needed in the *RoutingWrapper.c* file.

Each element has 4 bytes:

- Bytes 0 - 1: the short address of the blacklisted neighbour
- Bytes 2 - 3: the remaining time in minutes while this entry is valid

If the attribute length is set to 0 the entry is deleted.

This table is not ordered using any criteria, thus to get all the active entries the whole table has to be read, or iterate until [8.11.83. ADP_IB_MANUF_BLACKLIST_TABLE_COUNT = 0x080000DF](#) entries have been found.

8.11.32 ADP_IB_BLACKLIST_TABLE_ENTRY_TTL = 0x0000001F

This attribute indicates the maximum time-to-live of a blacklisted neighbour entry (in minutes). The length of the attribute is 2 bytes. The value is encoded little endian (lsb first). The default value is 10.

8.11.33 ADP_IB_MAX_JOIN_WAIT_TIME = 0x00000020

This attribute indicates the network join timeout in seconds for LBD. The size of the attribute is 2 bytes. The value is encoded little endian (lsb first). The default value is 20.

8.11.34 ADP_IB_PATH_DISCOVERY_TIME = 0x00000021

This attribute indicates the timeout for path discovery in seconds. The size of the attribute is 1 byte. The default value is 40.

8.11.35 ADP_IB_ACTIVE_KEY_INDEX = 0x00000022

This attribute indicates the index of the active GMK to be used for data transmission. The size of the attribute is 1 byte. The default value is 0.

8.11.36 ADP_IB_DESTINATION_ADDRESS_SET = 0x00000023

This attribute contains the Destination Address Set, which contains the list of the addresses of the device for which this LOADng router is providing connectivity.

The size of the table is 1 element. Each element has 2 bytes:

- Bytes 0 - 1: the short address of the target destination

If the attribute length is set to 0 the entry is deleted.

8.11.37 ADP_IB_DEFAULT_COORD_ROUTE_ENABLED = 0x00000024

If TRUE, the adaptation layer adds a default route to the coordinator after successful completion of the bootstrapping procedure. If FALSE, no default route will be created.

The size of the attribute is 1 byte. The default value is 0 (FALSE).

8.11.38 ADP_IB_DELAY_LOW_LQI = 0x00000025

Delay in ms before retransmitting an RREQ in an intermediate node when the LQI of the received RREQ is below adpRREQJitterLowLQI.

The size of the attribute is 2 bytes. The default value is 0.

8.11.39 ADP_IB_DELAY_HIGH_LQI = 0x00000026

Delay in ms before retransmitting an RREQ in an intermediate node when the LQI of the received RREQ is above adpRREQJitterHighLQI.

The size of the attribute is 2 bytes. The default value is 0.

8.11.40 ADP_IB_RREQ_JITTER_LOW_LQI = 0x00000027

LQI value below which RREQ retransmission is delayed by adpDelayLowLQI.

The size of the attribute is 1 byte. The default value is 52.

8.11.41 ADP_IB_RREQ_JITTER_HIGH_LQI = 0x00000028

LQI value above which RREQ retransmission is delayed by adpDelayHighLQI.

The size of the attribute is 1 byte. The default value is 120.

8.11.42 ADP_IB_TRICKLE_DATA_ENABLED = 0x00000029

Enables Trickle for data broadcast frames.

The size of the attribute is 1 byte. The default value is 0 (FALSE).

8.11.43 ADP_IB_TRICKLE_LQI_THRESHOLD_LOW = 0x0000002A

Low LQI threshold used to schedule broadcast messages. Also defines the minimum LQI value above which POS table entries are considered for the calculation of the redundancy constant Ki.

The size of the attribute is 1 byte. The default value is 80.

8.11.44 ADP_IB_TRICKLE_STEP = 0x0000002B

Indication of the desired redundancy related to the reception of broadcast frames per tier of recent POS table entries.

The size of the attribute is 1 byte. The default value is 1.

8.11.45 ADP_IB_TRICKLE_I_MIN = 0x0000002D

Minimum Trickle interval size in milliseconds.

The size of the attribute is 2 bytes. The default value is 1200 for CENELEC bandplans, 600 for FCC bandplans.

8.11.46 ADP_IB_TRICKLE_MAX_KI = 0x0000002E

Maximum redundancy constant allowed in the PAN.

The size of the attribute is 1 byte. Allowed values are from 1 to 5. The default value is 3.

8.11.47 ADP_IB_TRICKLE_ADAPTIVE_I_MIN = 0x0000002F

If activated, the minimum trickle interval size is not taken from `adpTrickleImin`, but adaptively computed.

The size of the attribute is 1 byte. The default value is 1 (TRUE).

8.11.48 ADP_IB_TRICKLE_ADAPTIVE_KI = 0x00000030

If disabled (FALSE), Ki is statically set to `adpTrickleMaxKi`.

The size of the attribute is 1 byte. The default value is 0 (FALSE).

8.11.49 ADP_IB_CLUSTER_TRICKLE_ENABLED = 0x00000031

Enables Trickle for RREQ messages.

The size of the attribute is 1 byte. The default value is 0 (FALSE).

8.11.50 ADP_IB_CLUSTER_MIN_LQI = 0x00000032

The minimum LQI value above which a link is considered within the same cluster.

The size of the attribute is 1 byte. The default value is 90.

8.11.51 ADP_IB_CLUSTER_TRICKLE_K = 0x00000033

The redundancy constant used in the cluster trickle algorithm.

The size of the attribute is 1 byte. Allowed values are from 1 to 10. The default value is 3.

8.11.52 ADP_IB_CLUSTER_RREQ_ROUTE_COST_DEVIATION = 0x00000034

Positive or negative deviation of the route cost of the stored RREQ compared to the route cost of the incoming RREQ.

The size of the attribute is 1 byte. The default value is 4.

8.11.53 ADP_IB_CLUSTER_TRICKLE_I = 0x00000035

Trickle interval time in milliseconds. Recommended to be set to $\text{adpClusterTrickleK} * 3 * \text{duration(RREQ)}$, where the latter is the duration of an RREQ frame in the used bandplan.

The size of the attribute is 2 bytes. Allowed values are up to 4096. The default value is $\text{adpClusterTrickleK} * 3 * \text{duration(RREQ)}$.

8.11.54 ADP_IB_TRICKLE_LQI_THRESHOLD_HIGH = 0x00000036

High LQI threshold used for Trickle Counter incrementation.

The size of the attribute is 1 byte. The default value is 90.

8.11.55 ADP_IB_DISABLE_DEFAULT_ROUTING = 0x000000F0

This attribute indicates if LOADng protocol should or should not be used. The size of the attribute is 1 byte. If 1 (true), the default routing (LOADng) is disabled. If 0 (false), the default routing (LOADng) is enabled. The default value is 0.

8.11.56 ADP_IB_MANUF_REASSEMBLY_TIMER = 0x080000C0

This manufacturer attribute can be used to specify the reassembly timer (the maximum delay between the reception of two fragments from a message at ADP level). The size of the attribute is 2 bytes. The value is encoded little endian. Default value is 60 seconds.

8.11.57 ADP_IB_MANUF_IPV6_HEADER_COMPRESSION = 0x080000C1

This manufacturer attribute can be used to enable / disable the compression of the IPv6 headers. This applies only on the messages sent using the DataRequest service. The size of the attribute

is 1 byte. The value 0 indicates the compression is disabled. Value 1 indicates the compression is enabled. The default value is 1.

8.11.58 ADP_IB_MANUF_PAN_ID = 0x080000C3

This manufacturer attribute specifies the PAN ID of the Joined Network. It is set by the LBP module on Devices when the Join process is successfully completed. It is not intended to be used by upper layers.

The size is 2 bytes. The default value is 0xFFFF.

8.11.59 ADP_IB_MANUF_BROADCAST_SEQUENCE_NUMBER = 0x080000C4

This manufacturer attribute can be used to get/set the broadcast sequence number. The size of the attribute is 1 byte.

8.11.60 ADP_IB_MANUF_DATAGRAM_TAG = 0x080000C6

This manufacturer attribute can be used to specify the datagram tag to be used for the fragmented transfers. The size of the attribute is 2 bytes. The value is encoded little endian. The default value is 0, which means the datagram will be randomly chosen.

8.11.61 ADP_IB_MANUF_ROUTING_TABLE_COUNT = 0x080000C8

This read-only manufacturer attribute can be used to get the number of active entries in the routing table. The size of the attribute is 4 bytes. The value represents a 32 bits integer encoded little endian (lsb first).

8.11.62 ADP_IB_MANUF_DISCOVER_SEQUENCE_NUMBER = 0x080000C9

This manufacturer attribute can be used to get/set the discover sequence number. The size of the attribute is 2 bytes. The value is encoded little endian.

8.11.63 ADP_IB_MANUF_FORCED_NO_ACK_REQUEST = 0x080000CA

This manufacturer attribute can be used to force all frames to be sent with No ACK request. The size of the attribute is 1 byte. The default value is 0, which means that the ACK request is decided by G3 mechanisms. Any other value forces all frames to be sent without ACK request.

8.11.64 ADP_IB_MANUF_LQI_TO_COORD = 0x080000CB

This manufacturer attribute retrieves LQI of the link which is the next hop in the route to coordinator. This link can be to coordinator itself if coordinator is a neighbour, or to another device if a route is needed to reach it.

The size of the attribute is 1 byte. The default value is 0, which means that coordinator is not a neighbour and there is no route to reach it (typically before starting to exchange any data with it).

8.11.65 ADP_IB_MANUF_BROADCAST_ROUTE_ALL = 0x080000CC

This manufacturer attribute can be used to activate specific mode where all the broadcast messages are propagated (in respect with broadcast sequence number and number of hops) even if the modem is not part of the destination group of the message.

The normal behaviour is to drop broadcast messages if device is not part of the destination group, thus default value is 0.

The size of the attribute is 1 byte, interpreted as Boolean.

8.11.66 ADP_IB_MANUF_ADP_INTERNAL_VERSION = 0x080000CE

This attribute indicates the internal ADP version. This is a read-only attribute and the length is 6 bytes:

- Byte 0: major version number
- Byte 1: minor version number
- Byte 2: revision number
- Byte 3: version date: year (since 2000)
- Byte 4: version date: month
- Byte 5: version date: day

8.11.67 ADP_IB_MANUF_CIRCULAR_ROUTES_DETECTED = 0x080000CF

This manufacturer attribute can be used to get the number of circular routes (routing loops) detected and corrected by device. The size of the attribute is 2 bytes.

8.11.68 ADP_IB_MANUF_LAST_CIRCULAR_ROUTE_ADDRESS = 0x080000D0

This manufacturer attribute can be used to get the next hop address involved in the last routing loop detected and corrected, it is provided for statistical purposes. The size of the attribute is 2 bytes.

8.11.69 ADP_IB_MANUF_IPV6_ULA_DEST_SHORT_ADDRESS = 0x080000D1

This manufacturer attribute is intended to be used in a G3 Gateway implementation. It allows upper layers to set the destination short address of the next data frame to be transmitted, which will be derived from an ULA address. This allows the ULA / short address translation to be managed by the gateway application, out of G3 stack scope. The size of the attribute is 2 bytes.

8.11.70 ADP_IB_MANUF_MAX_REPAIR_RESEND_ATTEMPTS = 0x080000D2

This manufacturer attribute sets the number of attempts of transmission for a frame when repair has been performed. The concept “attempt” refers to the whole cycle of transmission fail and successful repair. The aim of this attribute is to avoid an infinite loop of transmission fail + repair successful, limiting this cycle to a number of iterations. Default value is 5 and attribute size is 1 byte.

8.11.71 ADP_IB_MANUF_DISABLE_AUTO_RREQ = 0x080000D3

This manufacturer attribute allows controlling the stack behaviour when a route is not available for sending a frame. This situation happens if an existing route is broken due to No ACK reception after sending a frame, or if previous route does not exist or is no longer valid. The default behaviour is to automatically trigger a route discover or repair mechanism (thus default IB value is false), but the user can change this to discard the frame sending if there is no route available. When this IB is set, only route discovery processes requested by upper layer by means of AdpRouteDiscoveryRequest primitive will be executed. Please note that this behaviour is out of G3 spec, and may lead to interoperability issues. Attribute size is 1 byte. The default value is 0.

8.11.72 ADP_IB_MANUF_SHORT_ADDRESS = 0x080000D4

This manufacturer attribute specifies the Short Address that a Device is given after Joining a Network. It is set by the LBP module on Devices when the Join process is successfully completed. It is not intended to be used by upper layers.

The size is 2 bytes. The default value is 0xFFFF.

8.11.73 ADP_IB_MANUF_ALL_NEIGHBORS_BLACKLISTED_COUNT = 0x080000D5

As a protection, when a device detects that all its neighbours are blacklisted, the blacklist table is cleared so there is a chance to find routes again through any of them.

This manufacturer attribute counts how many times this protection has been triggered. Attribute size is 2 bytes. The default value is 0.

8.11.74 ADP_IB_MANUF_QUEUED_ENTRIES_REMOVED_TIMEOUT_COUNT = 0x080000D6

The ADP transmission queue implements a timeout so frames not released for transmission for a given time are discarded. Time-out is set to 2*ADP_IB_NET_TRAVERSAL_TIME.

This manufacturer attribute counts how many frames have been discarded due to time-out. Attribute size is 2 bytes. The default value is 0.

8.11.75 ADP_IB_MANUF_QUEUED_ENTRIES_REMOVED_ROUTE_ERROR_COUNT = 0x080000D7

When a route request/repair fails, any element in ADP transmission queue with the same destination as the one for which the route process has failed is purged. This prevents immediately starting a new route discovery process to the same destination that just failed, and thus avoids channel congestion due to the flooding of RREQs in the network.

This manufacturer attribute counts how many frames have been discarded due to the above explained reason. Attribute size is 2 bytes. The default value is 0.

8.11.76 ADP_IB_MANUF_PENDING_DATA_IND_SHORT_ADDRESS = 0x080000D8

This manufacturer attribute is used for inter-communication between ADP and Routing libraries. It is not intended to be used by the final user application.

Attribute size is 2 bytes. The default value is 0.

8.11.77 ADP_IB_MANUF_GET_BAND_CONTEXT_TONES = 0x080000D9

This manufacturer attribute is used for inter-communication between ADP and Routing libraries. It is not intended to be used by the final user application.

Attribute size is 1 byte. The default value is 0.

8.11.78 ADP_IB_MANUF_UPDATE_NON_VOLATILE_DATA = 0x080000DA

This manufacturer attribute is used for inter-communication between ADP and Routing libraries. It is not intended to be used by the final user application.

Attribute size is 1 byte. The default value is 0.

8.11.79 ADP_IB_MANUF_DISCOVER_ROUTE_GLOBAL_SEQ_NUM = 0x080000DB

This manufacturer attribute is used for inter-communication between ADP and Routing libraries. It is not intended to be used by the final user application.

Attribute size is 2 bytes. The default value is 0.

8.11.80 ADP_IB_MANUF_FRAGMENT_DELAY = 0x080000DC

This manufacturer attribute is used to set a delay between 6LoWPAN fragments at originator. The intention of such delay is to avoid collisions when fragments are retransmitted through subsequent hops.

The effect of this attribute depends on other related ones as follows:

When [8.11.81. ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_ENABLED = 0x080000DD](#) is false, it is the delay between fragments.

When [8.11.81. ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_ENABLED = 0x080000DD](#) is set, this IB sets a ceiling value to apply to the formula used by [8.11.82. ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_FACTOR = 0x080000DE](#).

If this IB is set to 0, no delay is applied in any case.

Attribute size is 2 bytes. The default value is 0.

8.11.81 ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_ENABLED = 0x080000DD

This manufacturer attribute is used to enable dynamic 6LowPAN fragments delay. When true, dynamic delay between fragments is used, otherwise static delay is applied.

Attribute size is 1 byte. The default value is 0 (false).

8.11.82 ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_FACTOR = 0x080000DE

This manufacturer attribute is used to set a factor in order to calculate the dynamic 6LowPAN fragments delay.

When [8.11.81. ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_ENABLED = 0x080000DD](#) is set, delay between fragments is calculated depending on number of hops to destination:

*Delay = (ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_FACTOR * num_hops_to_destination), when num_hops_to_destination > 2. Otherwise Delay = 0.*

This IB has no effect when [8.11.81. ADP_IB_MANUF_DYNAMIC_FRAGMENT_DELAY_ENABLED = 0x080000DD](#) is false.

Attribute size is 2 bytes. The default value is 800.

8.11.83 ADP_IB_MANUF_BLACKLIST_TABLE_COUNT = 0x080000DF

This read-only manufacturer attribute can be used to get the number of active entries in the Blacklist table [8.11.31. ADP_IB_BLACKLIST_TABLE = 0x0000001E](#). The size of the attribute is 2 bytes. The value represents a 16 bits integer encoded little endian (LSB first).

8.11.84 ADP_IB_MANUF_BROADCAST_LOG_TABLE_COUNT = 0x080000E0

This read-only manufacturer attribute can be used to get the number of active entries in the Broadcast Log table [8.11.12. ADP_IB_BROADCAST_LOG_TABLE = 0x0000000B](#). The size of the attribute is 2 bytes. The value represents a 16 bits integer encoded little endian (LSB first).

8.11.85 ADP_IB_MANUF_CONTEXT_INFORMATION_TABLE_COUNT = 0x080000E1

This read-only manufacturer attribute can be used to get the number of active entries in the Context Information table [8.11.8. ADP_IB_CONTEXT_INFORMATION_TABLE = 0x00000007](#). The size of the attribute is 2 bytes. The value represents a 16 bits integer encoded little endian (LSB first).

8.11.86 ADP_IB_MANUF_GROUP_TABLE_COUNT = 0x080000E2

This read-only manufacturer attribute can be used to get the number of active entries in the Group table [8.11.15. ADP_IB_GROUP_TABLE = 0x0000000E](#). The size of the attribute is 2 bytes. The value represents a 16 bits integer encoded little endian (LSB first).

8.11.87 ADP_IB_MANUF_ROUTING_TABLE_ELEMENT = 0x080000E3

This object has the same content as the Routing table, but it is retrieved using the short address of the node, instead of the index inside the table. To do so, set the short address in the index field in the Get IB function.

This attribute is Read-only.

8.11.88 ADP_IB_MANUF_HYBRID_PROFILE = 0x080000E5

This read-only manufacturer attribute is used to read from upper layers whether the underlying G3 stack implements the Hybrid Profile or not.

The attribute size is 1 byte. The Value is set to '0' (false) on PLC-only projects, and to '1' (true) on projects supporting the Hybrid Profile.

8.11.89 ADP_IB_MANUF_LAST_PHASEDIFF = 0x080000E6

This manufacturer attribute stores the computed Phase Differential for the last received frame. It is used by LOADng to retrieve the value and set it as part of the PREQ/PREP frames.

Attribute size is 1 byte. The default value is 0.

8.11.90 ADP_IB_LOW_LQI_VALUE_RF = 0x000000D0

The low LQI value defines the LQI value, used in metric computation, below which a link to a neighbour is considered an unreliable RF link. This value will be lower than adpHighLQIValueRF. The size of the attribute is 1 byte. The default value is 0x00.

8.11.91 ADP_IB_HIGH_LQI_VALUE_RF = 0x000000D1

The low LQI value defines the LQI value, used in metric computation, above which a link to a neighbour is considered the most reliable RF link. This value will be higher than adpLowLQIValueRF and lower than 255. The size of the attribute is 1 byte. The default value is 254.

8.11.92 ADP_IB_KQ_RF = 0x000000D2

This attribute defines a weight factor for LQI to calculate link cost. The length of the attribute is 1 byte. Default value: 10.

8.11.93 ADP_IB_KH_RF = 0x000000D3

This attribute defines a weight factor for Hop Count to calculate link cost. The length of the attribute is 1 byte. Default value: 4.

8.11.94 ADP_IB_KRT_RF = 0x000000D4

This attribute defines a weight factor for Number of Active Routes to calculate link cost. The length of the attribute is 1 byte. Default value: 0.

8.11.95 ADP_IB_KDC_RF = 0x000000D5

This attribute defines a weight factor for Duty Cycle to calculate link cost. The length of the attribute is 1 byte. Default value: 10.

8.11.96 ADP_IB_USE_BACKUP_MEDIA = 0x000000D6

This attribute controls if retransmission can use the backup medium in case of transmission failure. The length of the attribute is 1 byte (boolean). Default value: True.

8.11.97 ADP_IB_WEAK_LQI_VALUE_RF = 0x000000D7

The weak link value defines the LQI value below which an RF link to a neighbour is considered as a weak link. It should be set to sensitivity threshold plus sufficient margin (which depends on FSK operating mode).

The size of the attribute is 1 byte. The default value is 85 (FSK 50kbit/s).

8.11.98 ADP_IB_TRICKLE_LQI_THRESHOLD_LOW_RF = 0x000000D8

Low LQI threshold used to schedule broadcast messages. Also defines the minimum LQI value above which RF POS table entries are considered for the calculation of the redundancy constant Ki.

The size of the attribute is 1 byte. The default value is 85.

8.11.99 ADP_IB_DELAY_LOW_LQI_RF = 0x000000D9

Delay in ms before retransmitting an RREQ in an intermediate node when the LQI of the received RREQ from RF medium is below adpRREQJitterLowLQI_RF.

The size of the attribute is 2 bytes. The default value is 1500.

8.11.100 ADP_IB_DELAY_HIGH_LQI_RF = 0x000000DA

Delay in ms before retransmitting an RREQ in an intermediate node when the LQI of the received RREQ from RF medium is above `adpRREQJitterHighLQI_RF`.

The size of the attribute is 2 bytes. The default value is 300.

8.11.101 ADP_IB_RREQ_JITTER_LOW_LQI_RF = 0x000000DB

LQI value from RF medium below which RREQ retransmission is delayed by `adpDelayLowLQI_RF`.

The size of the attribute is 1 byte. The default value is 60.

8.11.102 ADP_IB_RREQ_JITTER_HIGH_LQI_RF = 0x000000DC

LQI value from RF medium above which RREQ retransmission is delayed by `adpDelayHighLQI_RF`.

The size of the attribute is 1 byte. The default value is 180.

8.11.103 ADP_IB_CLUSTER_TRICKLE_I_RF = 0x000000DD

Trickle interval time in milliseconds.

The size of the attribute is 2 bytes. Allowed values are up to 4096. The default value is $adpClusterTrickleK_RF * 3 * duration(RREQ)$.

8.11.104 ADP_IB_CLUSTER_TRICKLE_K_RF = 0x000000DE

The redundancy constant used in the cluster trickle algorithm for RF media type.

The size of the attribute is 1 byte. Allowed values are from 1 to 10. The default value is 3.

8.11.105 ADP_IB_CLUSTER_MIN_LQI_RF = 0x000000DF

The minimum LQI value above which an RF link is considered within the same cluster.

The size of the attribute is 1 byte. The default value is 90.

8.11.106 ADP_IB_LAST_GASP = 0x000000E0

Controls the status of Last Gasp mode. To be enabled when the device loses the AC power source.

The size of the attribute is 1 byte. The default value is 0 (FALSE).

8.11.107 ADP_IB_TRICKLE_LQI_THRESHOLD_HIGH_RF = 0x000000E2

High LQI threshold used for Trickle Counter incrementation.

The size of the attribute is 1 byte. The default value is 95.

8.12 ADP Status Codes Description

Following is the list of status codes and a brief description of each one, as reported by ADP:

8.12.1 G3_SUCCESS (0x00)

Returned when the last requested operation succeeded, may it be a data request, a get or set request, etc.

8.12.2 G3_INVALID_REQUEST (0xA1)

This value is reported in the following situations:

- Data request is called before device has joined a network. Suggested action: Wait for Bootstrap process to complete before calling data request.
- `AdpNetworkJoinRequest` is called when the device is already part of a network. Suggested action: Call `AdpNetworkLeaveRequest` and wait for the end of this process before trying to join a network again.

- AdpNetworkLeaveRequest is called when the device is not part of a network. Suggested action: Assume device is not part of a network, even before leave request was called.

8.12.3 G3_FAILED (0xA2)

This value is reported in the following situations:

- When setting ADP_IB_MANUF_REGISTER_DEVICE and the information is correctly set at the ADP layer, but not at the MAC level. Suggested action: Reset G3 stack and configure the MAC and ADP layers again.
- Data request is correctly handled by ADP layer, but MAC layer has no resources in such moment to handle it. Suggested action: wait a random time in application level to let the MAC layer free some resources and try sending again.
- On PAN coordinator, AdpNetworkStartConfirm returns this value when AdpNetworkStartRequest is not able to request information from the MAC layer (short and extended addresses), or when the short address at MAC layer does not match the value in ADP_IB_COORD_SHORT_ADDRESS. Suggested action: Reset G3 stack and configure MAC and ADP layers again.
- On the device, AdpNetworkJoinConfirm returns this value when AdpNetworkJoinRequest is not able to request information from the MAC layer (extended address and PAN ID). Suggested action: Reset G3 stack and configure MAC and ADP layers again.
- On the device, AdpNetworkJoinConfirm returns this value when setting MAC parameters during the joining process fails in some intermediate state while exchanging data with coordinator. Suggested action: Reset G3 stack and configure the MAC and ADP layers again. Then start the network join process again.
- On the device, AdpDiscoveryConfirm returns this value when a scan confirm is received from the MAC layer, but the ADP layer was not expecting it. Suggested action: Ignore confirm and start the discovery process again if necessary.

8.12.4 G3_INVALID_IPV6_FRAME (0xA3)

This value is returned when the buffer passed to AdpDataRequest for sending does not contain a valid IPv6 frame. Suggested action: Check the correctness of headers at the IP level and try sending again.

8.12.5 G3_NOT_PERMITTED (0xA4)

This value is returned when a device is not accepted in the network it is trying to join. It may be due to a DECLINE frame reception or to a lack or wrong reception of the ACCEPTED frame. Suggested action: Check correctness of values used in network join, or try to join another network in case the device is banned in the current one.

8.12.6 G3_ROUTE_ERROR (0xA5)

This value is returned in the following situations:

- When AdpPathDiscoveryRequest primitive is called, and there is no route for the intended destination. Suggested action: Call AdpRouteDiscoveryRequest first to create the route, and then call AdpPathDiscoveryRequest to get the path.
- When AdpDataRequest is called for a destination to which there is no route, and one of the following conditions is met:
 - Route discovery is not allowed due to request parameters or IB configuration. Suggested action: Allow route discovery, or perform AdpRouteDiscoveryRequest from upper layer before calling AdpDataRequest.
 - Route discovery is allowed, and such discovery fails to find a route. Suggested action: Wait for a route to be available (or explicitly look for it using AdpRouteDiscoveryRequest primitive), and then retry sending data.

8.12.7 G3_TIMEOUT (0xA6)

This value is returned in the following situations:

- When a join process is not completed after the time-out specified in ADP_IB_MAX_JOIN_WAIT_TIME parameter. Suggested action: Increase time-out.
- When a path discover process is not completed after the time-out specified in ADP_IB_PATH_DISCOVERY_TIME parameter. Suggested action: Increase time-out.

8.12.8 G3_INVALID_INDEX (0xA7)

This value is returned when trying to set/get an ADP IB object with an index out of range of the requested object. Suggested action: Check object range and verify the requested index.

8.12.9 G3_INVALID_PARAMETER (0xA8)

This value is returned when trying to set an ADP IB object with a different length than expected. Suggested action: Check that IB length is according to set request.

Note: A particular case is ADP_IB_SNIFFER_MODE, which is not implemented and this value is returned when trying to get/set it.

8.12.10 G3_NO_BEACON (0xA9)

Not used in the current stack.

8.12.11 G3_READ_ONLY (0xB0)

This value is returned when trying to set an ADP IB object which cannot be written. Suggested action: Check IB access rights.

8.12.12 G3_UNSUPPORTED_ATTRIBUTE (0xB1)

This value is returned when trying to set/get an ADP IB object with an ID which is not part of the available set. Suggested action: Check available IDs and verify the requested one.

8.12.13 G3_INCOMPLETE_PATH (0xB2)

This value is returned when the path returned after an AdpPathDiscoveryRequest has not the complete information, because a problem was encountered at some point of the path. Suggested action: Call AdpRouteDiscoveryRequest first to create the route, and then call AdpPathDiscoveryRequest to get the path.

8.12.14 G3_BUSY (0xB3)

This value is returned in the following situations:

When the AdpDiscoveryRequest primitive is called when a previous discovery process is already in progress. Suggested action: Wait for previous discovery process to finish (reception of corresponding AdpDiscoveryConfirm) to request a new one.

8.12.15 G3_NO_BUFFERS (0xB4)

This value is returned when the AdpDataRequest primitive is called, but the stack has no free resources to handle it. It can be due to the fact that all transmission buffers are being used, or because a route discovery is needed before data sending and there are no resources to perform the discovery (other discovery processes running are using the resources). Suggested action: wait a random time in application level to let the stack free some resources and try sending again.

8.12.16 G3_TRICKLE_REDUNDANT (0xB5)

This value is returned when the Trickle algorithm detects that a frame has been relayed by other nodes and there is no need to further propagate it. This value does not reach upper layers, as it is reported for frame relaying managed internally by ADP layer.

8.12.17 G3_ERROR_INTERNAL (0xFF)

This value is returned in the following situations:

- When trying to set a MAC IB using a length greater than the attribute length. Suggested action: Check that attribute length is according to set length.
- When trying to get a MAC IB and its length is greater than the container to return it. Suggested action: Report this issue, as it is a stack internal error that has to be fixed in the library.

8.12.18 MAC_WRP_STATUS_ALTERNATE_PANID_DETECTION (0x80)

This value is reported by [8.10.18. ADPM-NETWORK-STATUS.indication](#) when a frame is received from a PAN ID different than the network which the node belongs to. Actions to be taken when an alternate PAN ID is detected are left to the application.

8.12.19 MAC_WRP_STATUS_COUNTER_ERROR (0xDB)

This value is reported when Frame Counter reaches 0xFFFFFFFF. Suggested action: Reset Device and Reset Frame Counter, as security has to be reinitialized.

8.12.20 MAC_WRP_STATUS_UNSUPPORTED_SECURITY (0xDF)

This value is reported when there is an Encryption Error. Suggested action: Check that ADP_IB_SECURITY_LEVEL has a correct value, otherwise reset Device, as security has to be reinitialized.

8.12.21 MAC_WRP_STATUS_CHANNEL_ACCESS_FAILURE (0xE1)

This value is reported when CSMA retries are exhausted. Suggested action: Retry transmission after a random time.

8.12.22 MAC_WRP_STATUS_DENIED (0xE2)

This value is reported when Tx is aborted by MAC layer. Suggested action: Retry transmission after a random time.

8.12.23 MAC_WRP_STATUS_SECURITY_ERROR (0xE4)

This value is reported when there is an Encryption Error. Suggested action: Reset Device, as security has to be reinitialized.

8.12.24 MAC_WRP_STATUS_FRAME_TOO_LONG (0xE5)

This value is reported when MAC frame is too long (should not happen as ADP layer will split frames correctly). Suggested action: Report error to Microchip providing App layer frame transmitted, as ADP layer has to be debugged.

8.12.25 MAC_WRP_STATUS_NO_ACK (0xE9)

This value is reported in case ACK is not received and route discovery is not allowed when calling AdpDataRequest. If discovery is allowed, a route will be created and data sent, or in case route is not found, a G3_ROUTE_ERROR will be reported. Suggested action: Trigger a Route Discovery process from application, as it has not been automatically triggered by the stack.

8.12.26 MAC_WRP_STATUS_TRANSACTION_OVERFLOW (0xF1)

This value is reported when Device is already transmitting. Suggested action: Retry transmission after a random time.

8.12.27 MAC_WRP_STATUS_UNAVAILABLE_KEY (0xF3)

This value is reported when there is an Encryption Error. Suggested action: Check that ADP_IB_ACTIVE_KEY_INDEX has a correct value, otherwise reset Device, as security has to be reinitialized.

8.12.28 MAC_WRP_STATUS_LIMIT_REACHED (0xFA)

This value is reported when MAC layer ran out of transmitting resources. Suggested action: Retry transmission after a random time.

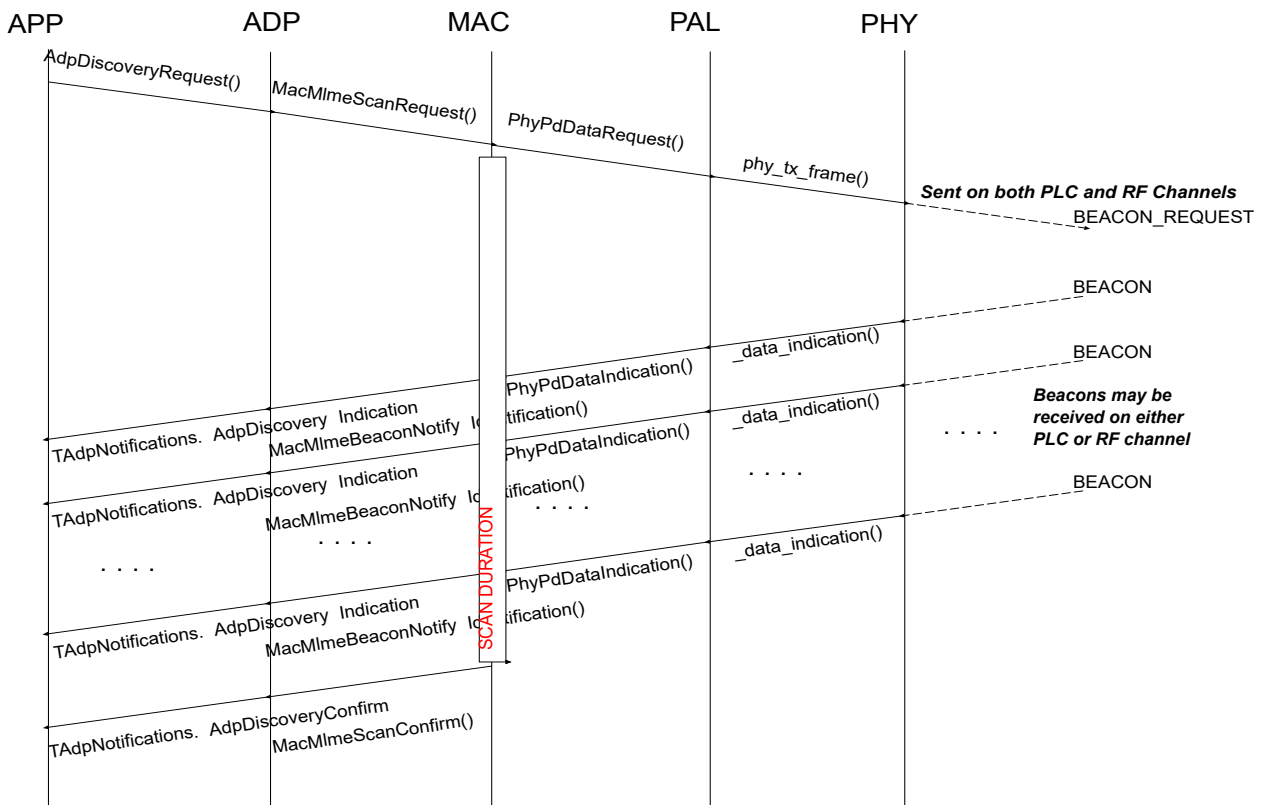
9. Data Link Processes

This chapter shows flow diagrams for different Data Link layer processes to help the user understand the flow of messages through the stack and the behavior of a G3-PLC or Hybrid network.

9.1 Network Discovery Request

The ADPM-DISCOVERY.request primitive allows the upper layer to request the ADPM to scan for networks operating in its POS. The ADPM-DISCOVERY.confirm primitive is generated by the ADP layer upon completion of a previous ADPM-DISCOVERY.request.

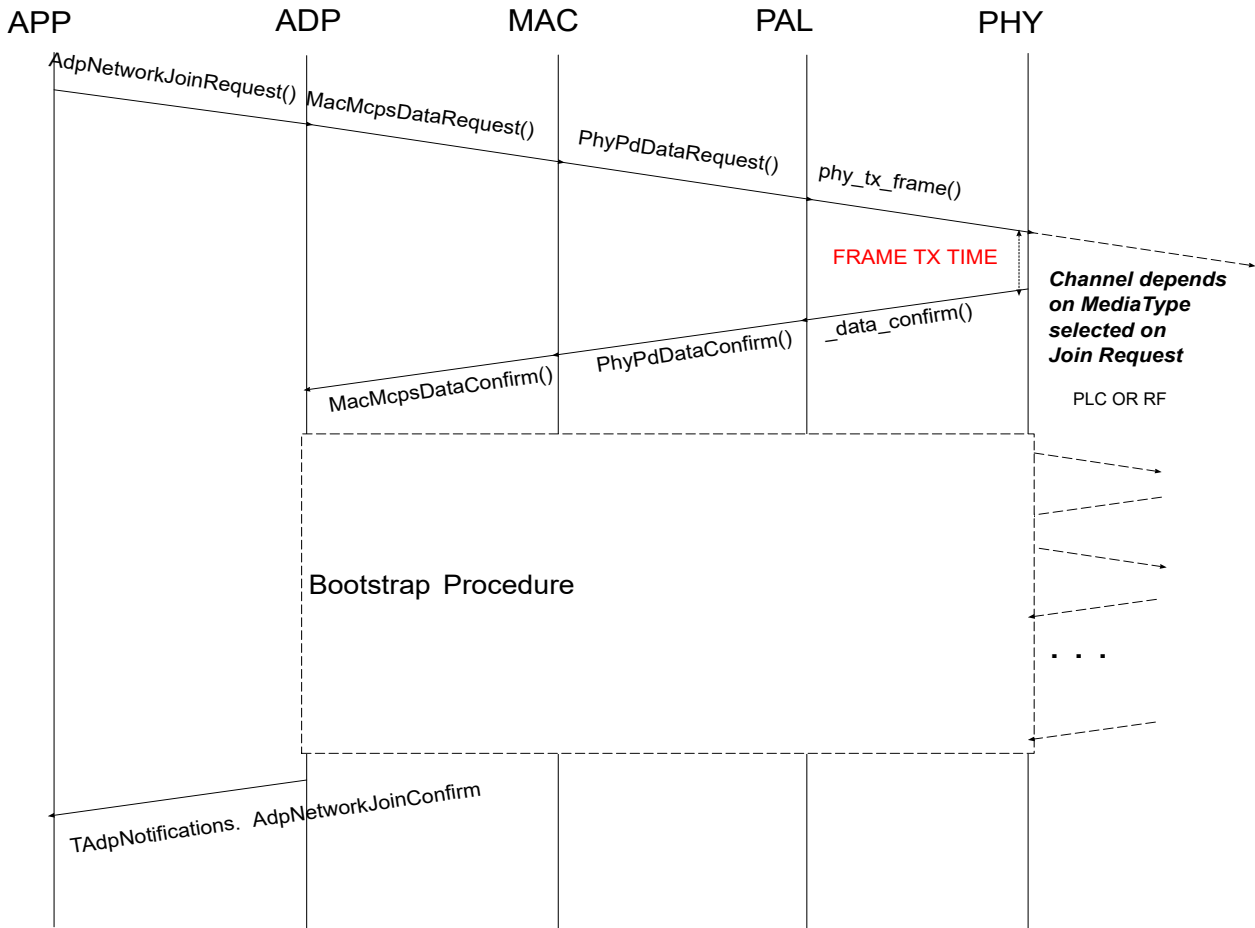
Figure 9-1. Network Discovery Request Flow Chart



9.2 Network Join Request

The ADPM-NETWORK-JOIN.request primitive allows the upper layer to join an existing network. The ADPM-NETWORK-JOIN.confirm primitive is generated by the ADP layer to indicate the completion status of a previous ADPM-NETWORK-JOIN.request.

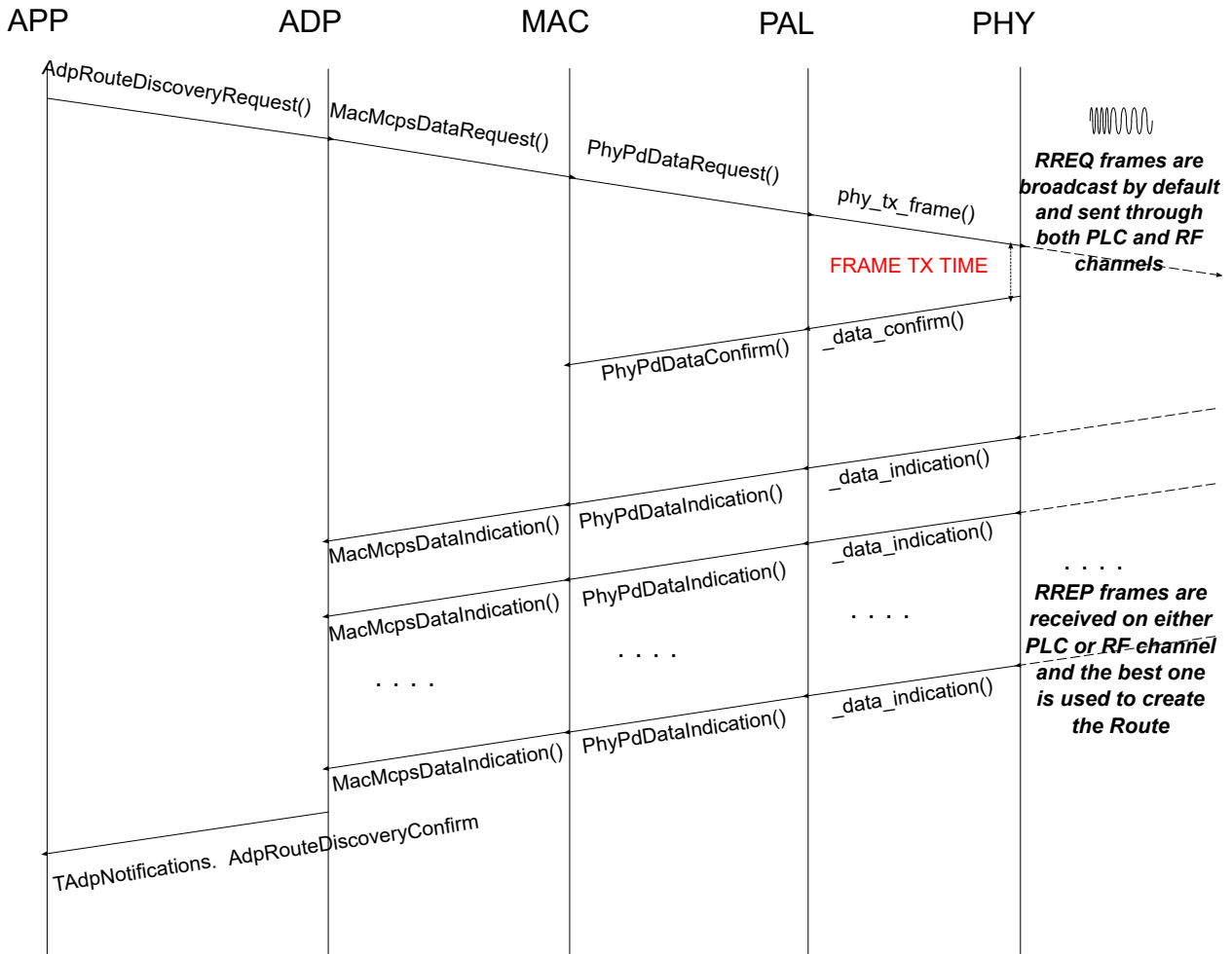
Figure 9-2. Network Join Request Flow Chart



9.3 Route/Path Discovery Request

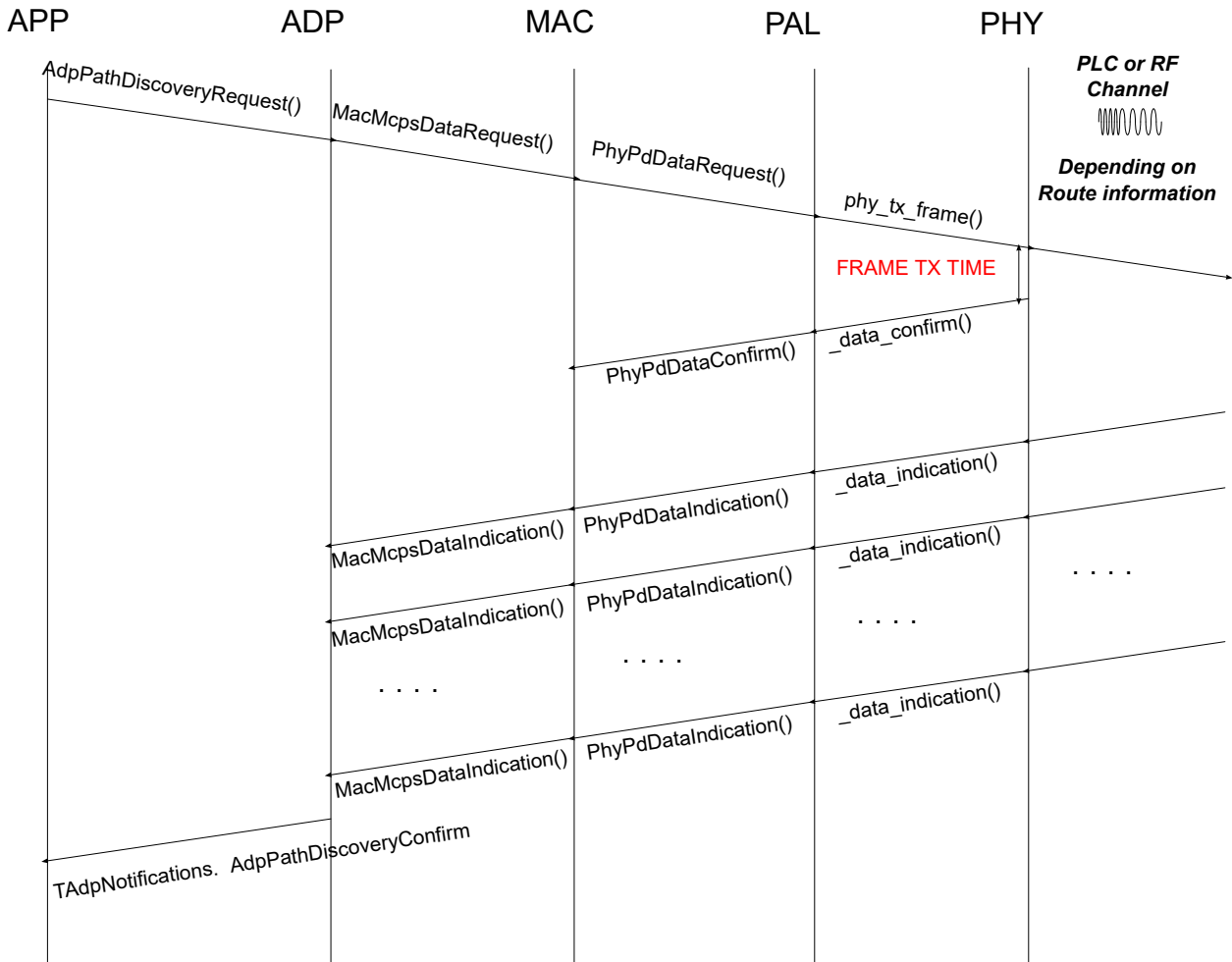
The ADPM-ROUTE-DISCOVERY.request primitive allows the upper layer to initiate a route discovery. The ADPM-ROUTE-DISCOVERY.confirm primitive allows the upper layer to be informed about a previous ADPM-ROUTE-DISCOVERY.request primitive.

Figure 9-3. Route Discovery Request Flow Chart



The ADPM-PATH-DISCOVERY.request primitive allows the upper layer to initiate a path discovery. The ADPM-PATH-DISCOVERY.confirm primitive allows the upper layer to be informed of the completion of a path discovery (ADPM-PATH-DISCOVERY.request).

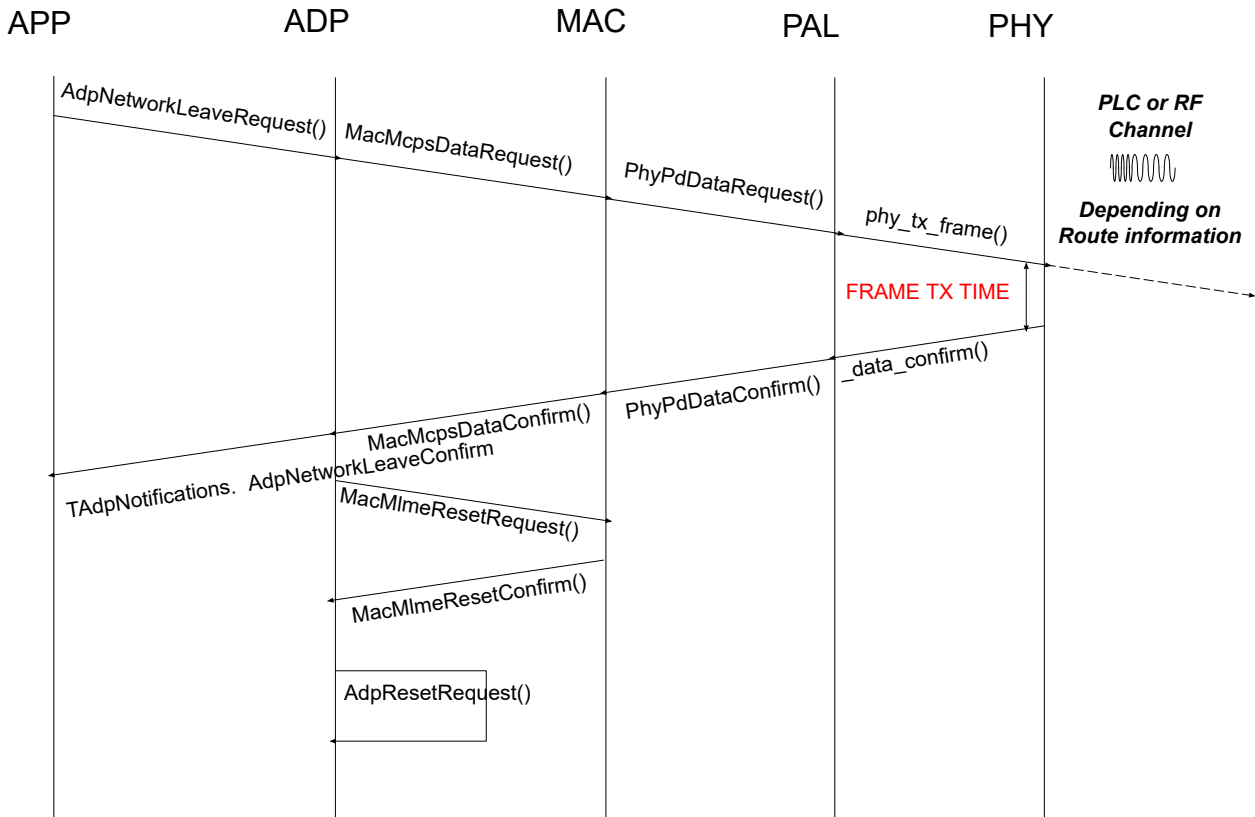
Figure 9-4. Path Discovery Request Flow Chart



9.4 Network Leave

The ADPM-NETWORK-LEAVE.request primitive allows a non-coordinator device to remove itself from the network. The ADPM-NETWORK-LEAVE.confirm primitive allows the upper layer to be informed of the status of its previous ADPM-NETWORK-LEAVE.request.

Figure 9-5. Network Leave Flow Chart



9.5 Data Request

The ADPD-DATA.request primitive requests the transfer of an application PDU (i.e., IPv6 packet) to another device or multiple devices. The ADPD-DATA.confirm primitive reports the result of a previous ADPD-DATA.requestprimitive. The ADPD-DATA.indication primitive is used to transfer received data from the adaptation sublayer to the upper layer.

Figure 9-6. ADP Data Request Flow Chart

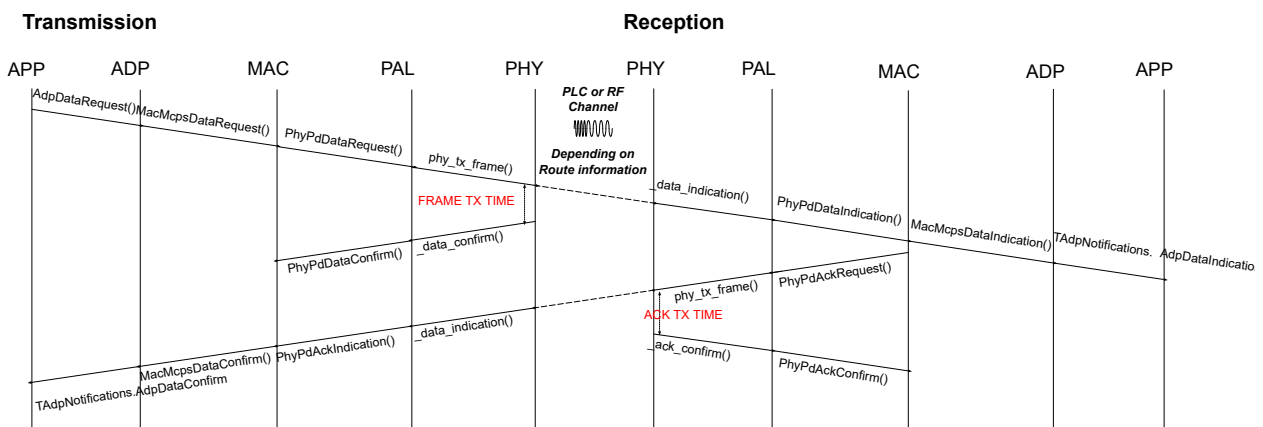
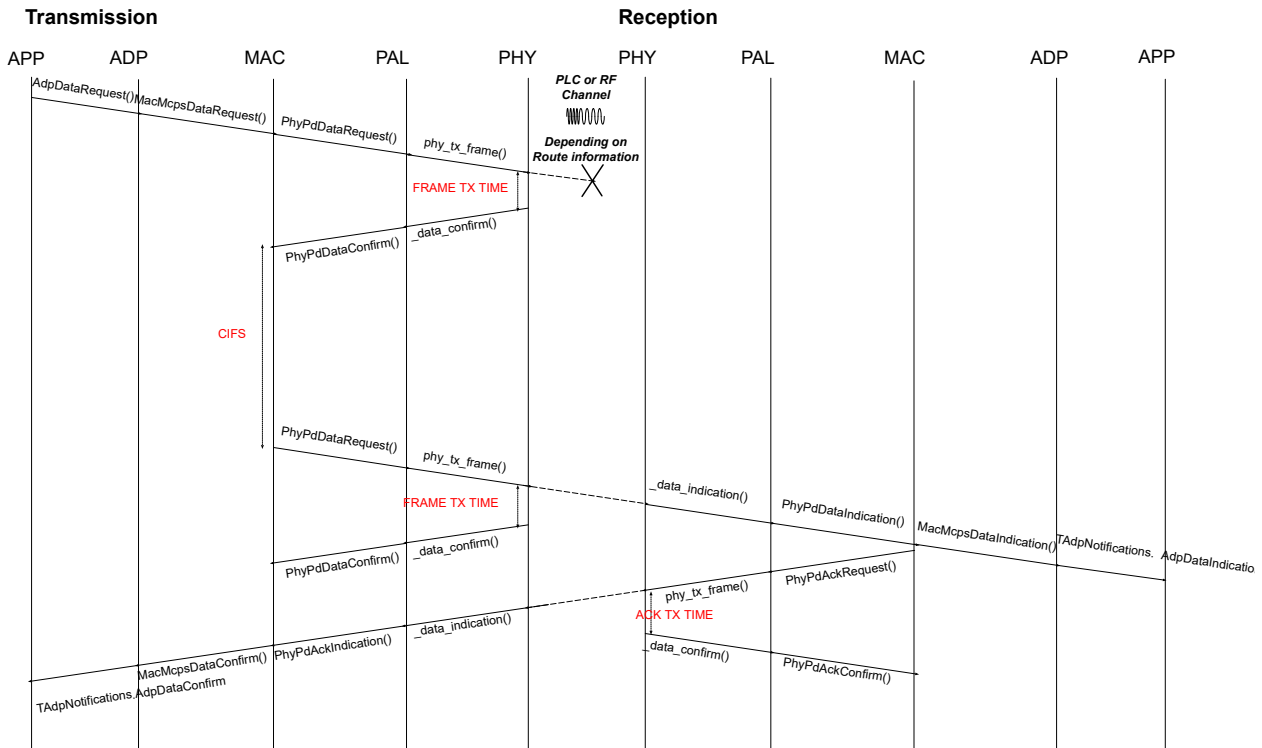


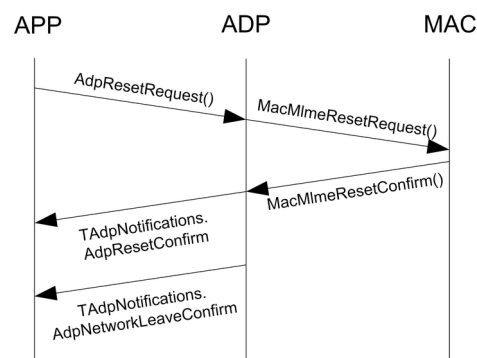
Figure 9-7. ADP Data Request Flow Chart. Retransmission due to ACK Fail



9.6 ADP Reset

The ADPM-RESET.request primitive performs a reset of the adaptation sublayer and allows the resetting of the MIB attributes. The ADPM-RESET.confirm primitive allows the upper layer to be notified of the completion of an ADPM-RESET.request primitive.

Figure 9-8. ADP Reset Flow Chart



9.7 GET/SET MIBs

The ADPM-GET.request primitive allows the upper layer to get the value of an attribute from the ADP information base. The ADPM-GET.confirm primitive allows the upper layer to be informed of the status of a previously issued ADPM-GET.request primitive.

The ADPM-SET.request primitive allows the upper layer to set the value of an attribute in the ADP information base. The ADPM-SET.confirm primitive allows the upper layer to be informed about a previous ADPM-SET.request primitive.

Figure 9-9. ADP GET Request and ADP SET Request Flow Charts



The ADPM-MAC.GET.request primitive allows the upper layer to get the value of an attribute from the MAC information base. The upper layer cannot access directly the MAC layer while ADP is running. The ADPM-MAC.GET.confirm primitive allows the upper layer to be informed of the status of a previously issued ADPM-MAC.GET.request primitive.

The ADPM-MAC.SET.request primitive allows the upper layer to set the value of an attribute in the MAC information base. The upper layer cannot access directly the MAC layer while ADP is running. The ADPM-MAC.SET.confirm primitive allows the upper layer to be informed about a previous ADPM-MAC.SET.request primitive.

Figure 9-10. ADPMAC GET Request and ADPMAC SET Request Flow Charts



10. LBP Module

LBP stands for LoWPAN Bootstrap Protocol, and its role is to manage the Creation of a Network (in Coordinator) or the Joining to the Network (in Devices), by performing tasks such as authentication and encryption key distribution.

The module has some shared functionality for both the Coordinator and the Device, such as encoding/decoding of the LBP frames, and some separated modules which are present or not depending on the node type, to manage the Joining process from both sides.

This module interacts with both the Coordinator Module (if present) and the ADP Library.

The following sections describe the API of this module for both Coordinator and Device.

11. LBP Module

LoWPAN Bootstrap Protocol (LBP) and its role is to manage the creation of a network (in coordinator) or the joining to the network (in devices), by performing tasks, such as authentication and encryption key distribution.

The module has some shared functionality for both the coordinator and the device, such as encoding/decoding of the LBP frames, and some separated modules which are present or not depending on the node type, to manage the Joining process from both sides.

This module interacts with both the Coordinator Module (if present) and the ADP Library.

11.1 LBP Coordinator

The LBP module in a Coordinator has the following roles:

- Managing the Joining and Leaving devices in the Network. This includes notifying upper layers about nodes willing to join the network (so upper layers can accept or reject them, kicking nodes out of the network, and notify upper layers about nodes leaving the network).
- Managing the change of the Network GMK, which is used by MAC layer to encrypt/decrypt frames.

11.1.1 LBP Init Function

The LBP_InitCoord primitive initializes Coordinator LBP module.

```
void LBP_InitCoord(bool aribBand);
```

Parameter:

<i>aribBand</i>	Indicates whether ARIB band is used for PLC communication. This has some implication in parameters during the bootstrap procedure, so LBP module has to be aware of it.
-----------------	---

11.1.2 LBP Process Function

The LBP_UpdateBootstrapSlots primitive updates the slots that control bootstrap protocol. It has to be called periodically in order to update the state machine that controls the bootstrapping phase of Joining nodes.

```
void LBP_UpdateBootstrapSlots(void);
```

The macro

```
#define BOOTSTRAP_NUM_SLOTS 5
```

Defines how many bootstrapping slots can be managed in parallel by Coordinator.

Note: Increasing the number of slots has an impact in data footprint, and it does not necessarily improve performance, because more nodes joining in parallel may cause network congestion and thus frames may be lost.

11.1.3 LBP Set Notifications Function

The LBP_SetNotificationsCoord primitive sets the callbacks to be invoked when events have to be notified.

```
void LBP_SetNotificationsCoord(struct TLbpNotificationsCoord *pNotifications);
```

Parameter:

pNotifications Structure containing pointers to functions that will be invoked when events have to be notified to upper layer.

```
struct TLbpNotificationsCoord {
    LBP_JoinRequestIndication fnctJoinRequestIndication;
    LBP_JoinCompleteIndication fnctJoinCompleteIndication;
    LBP_LeaveIndication fnctLeaveIndication;
};
```

*typedef void (*LBP_JoinRequestIndication)(uint8_t* au8LbdAddress);* Allows the upper layer to be notified of the intention of a device to Join the Network, allowing to reject it or accept it assigning a network short address. Contains the Extended Address of device willing to Join the network as parameter.

*typedef void (*LBP_JoinCompleteIndication)(uint8_t* au8LbdAddress, uint16_t u16AssignedAddress);* Allows the upper layer to be notified of the successful completion of a Network Join for a certain device. Contains the Extended and Short Addresses of device that joined the network as parameters.

*typedef void (*LBP_LeaveIndication)(uint16_t u16NetworkAddress);* Allows the upper layer to be notified when a device leaves the network. Contains the 16-bit network address of device leaving the network.

11.1.4 LBP Assign Address Function

The LBP_ShortAddressAssign primitive allows the upper layer to assign a short address to a device willing to Join.

This function is intended to be called by the upper layer upon reception of the JoinRequestIndication callback. This way, the bootstrapping process can continue with the provided Short Address.

The special (invalid) value 0xFFFF is used to indicate that a node has to be rejected, immediately terminating the bootstrapping process for such node.

```
void LBP_ShortAddressAssign(uint8_t* au8ExtAddress, uint16_t u16AssignedAddress);
```

Parameter:

au8ExtAddress The Extended address of device willing to Join.
u16AssignedAddress The Network address to be assigned. Set to 0xFFFF to reject device.

11.1.5 LBP Kick Device Function

The LBP_KickDevice primitive allows the upper layer to remove a device from the Network.

```
bool LBP_KickDevice(uint16_t us_short_address, struct TAdpExtendedAddress *pEUI64Address);
```

Parameter:

us_short_address The Network address of device to be removed.
pEUI64Address The Extended address of device to be removed. It is used to build the LBP frame contents.

Returns True if Kick is sent to device, otherwise False.

11.1.6 LBP Set Rekey Phase Function

The LBP_SetRekeyPhase primitive allows the upper layer to start or stop a rekey phase in LBP module.

This primitive has to be called by upper layer before start sending the new GMK to nodes in the Network, and after all nodes have received the new GMK.

```
void LBP_SetRekeyPhase(bool bRekeyStart);
```

Parameter:

bRekeyStart Indicates whether Rekey phase starts or finishes.

11.1.7 LBP Send Key Function

The LBP_Rekey primitive allows the upper layer to perform a rekey process for a device in the Network.

```
void LBP_Rekey(uint16_t us_short_address, struct TAdpExtendedAddress *pEUI64Address, bool bDistribute);
```

Parameter:

us_short_address The Network address of device to be rekeyed.
pEUI64Address The Extended address of device to be rekeyed.
bDistribute True in distribution phase, False in activation phase.

11.1.8 LBP Activate Key Function

The LBP_activate_new_key primitive activates the new GMK in the Coordinator itself after a Rekeying process, when all nodes have the new GMK.

```
void LBP_activate_new_key(void);
```

11.1.9 LBP Set Coordinator IB Function

The LBP_SetParamCoord primitive allows the upper layer to set the value of a parameter in LBP IB.

```
void LBP_SetParamCoord(uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLen, const uint8_t* pu8AttributeValue, struct TLbpSetParamConfirm *pSetConfirm);
```

Parameter:

u32AttributeId IB attribute ID. See 11.3. LBP IB List for details.
u16AttributeIndex Index when attribute is a list.
u8AttributeLen Length of value to set.
pu8AttributeValue Pointer to array containing value.
pSetConfirm Result of the Set operation.

11.2 LBP Device

The LBP module in a device has the following roles:

- Joining and leaving a network.
- Notifying the upper layer when the device is kicked from the network by PAN coordinator.

11.2.1 LBP Init Function

The LBP_InitDev primitive initializes the bootstrap protocol on a Device.

```
void LBP_InitDev(void);
```

11.2.2 LBP Set Notifications Function

The LBP_SetNotificationsDev primitive sets the callbacks to be invoked when events have to be notified.

```
void LBP_SetNotificationsDev(struct TLbpNotificationsDev *pNotifications);
```

Parameter:

pNotifications Structure containing pointers to functions that will be invoked when events have to be notified to upper layer.

```
struct TLbpNotificationsDev {
    AdpNetworkJoinConfirm fncJoinConfirm;
    AdpNetworkLeaveConfirm fncLeaveConfirm;
    AdpNetworkLeaveIndication fncLeaveIndication;
};
```

These Callbacks are part of the ADP API and are described in the next sections.

11.2.2.1 ADPM-NETWORK-JOIN.confirm

The ADPM-NETWORK-JOIN.confirm primitive is generated by the ADP layer to indicate the completion status of a previous ADPM-NETWORK-JOIN.request.

```
typedef void (*AdpNetworkJoinConfirm)(struct TAdpNetworkJoinConfirm *pNetworkJoinConfirm);
```

Structure:

```
struct TAdpNetworkJoinConfirm {
    uint8_t m_u8Status;
    uint16_t m_u16NetworkAddress;
    uint16_t m_u16PanId;
};
```

Fields:

<i>m_u8Status</i>	The status of the Join request.
<i>m_u16NetworkAddress</i>	The 16-bit network address that was allocated to the device. If the allocation fails, this address is equal to 0xFFFF.
<i>m_u16PanId</i>	The 16-bit address of the PAN of which the device is now a member.

Status codes are detailed in [8.12. ADP Status Codes Description](#).

11.2.2.2 ADPM-NETWORK-LEAVE.confirm

The ADPM-NETWORK-LEAVE.confirm primitive allows the upper layer to be informed of the status of its previous ADPM-NETWORK-LEAVE.request.

```
typedef void (*AdpNetworkLeaveConfirm)(struct TAdpNetworkLeaveConfirm *pLeaveConfirm);
```

Structure:

```
struct TAdpNetworkLeaveConfirm {
    uint8_t m_u8Status;
};
```

Field:

<i>m_u8Status</i>	The status of the request.
-------------------	----------------------------

Status codes are detailed in [8.12. ADP Status Codes Description](#).

11.2.2.3 ADPM-NETWORK-LEAVE.indication

ADPM-NETWORK-LEAVE.indication primitive is generated by the ADP layer of a non-coordinator device to inform the upper layer that it has been unregistered from the network by the coordinator.

```
typedef void (*AdpNetworkLeaveIndication)(void);
```

11.2.3 LBP Set Device IB Function

The LBP_SetParamDev primitive allows the upper layer to set the value of a parameter in LBP IB.

```
void LBP_SetParamDev(uint32_t u32AttributeId, uint16_t u16AttributeIndex, uint8_t u8AttributeLen, const uint8_t *pu8AttributeValue, struct TlbpSetParamConfirm *pSetConfirm);
```

Parameter:

<i>u32AttributeId</i>	IB attribute ID. See 11.3. LBP IB List for details.
<i>u16AttributeIndex</i>	Index when attribute is a list.
<i>u8AttributeLen</i>	Length of value to set.
<i>pu8AttributeValue</i>	Pointer to array containing value.
<i>pSetConfirm</i>	Result of the Set operation.

11.2.4 LBP Force Register Function

The LBP_ForceRegister function is used for testing purposes to force the device register in the network without going through the bootstrap process.

```
void LBP_ForceRegister(struct TAdpExtendedAddress *pEUI64Address, uint16_t u16ShortAddress, uint16_t u16PanId, struct TGroupMasterKey *pGMK);
```

Parameter:

<i>pEUI64Address</i>	Pointer to EUI64 address of the node.
<i>u16ShortAddress</i>	The 16-bit network address to be set.
<i>u16PanId</i>	The 16-bit PAN ID to be set.
<i>pGMK</i>	Pointer to Group Master Key to set.

Note: As stated on function description, this primitive is added only for testing purposes. Its usage avoids going through the regular bootstrapping process, which implies a security breach.

11.3 LBP IB List

The following configuration parameters are available in the LBP Information Base (IB).

11.3.1 LBP_IB_IDS = 0x00000000

This object represents the Network Access Identifier of the EAP server, thus only available on Coordinator LBP module. Its size depends on the used band (8 bytes for Cenelec and FCC, and 34 bytes for ARIB).

Access is write-only.

11.3.2 LBP_IB_IDP = 0x00000001

This object represents the Network Access Identifier of the EAP peer, thus only available on Device LBP module. Its size depends on the used band (8 bytes for Cenelec and FCC, and 36 bytes for ARIB).

Access is write-only.

11.3.3 LBP_IB_PSK = 0x00000002

With this IB, the PSK of the LBP procedure can be specified. Its size is 16 bytes. This attribute is available for both Coordinator and Device.

Access is write-only.

11.3.4 LBP_IB_GMK = 0x00000003

With this IB, the GMK to be distributed by Coordinator on the Bootstrapping phase can be specified. Its size is 16 bytes.

Access is write-only.

11.3.5 LBP_IB_REKEY_GMK = 0x00000004

With this IB, the GMK to be distributed by Coordinator on the Rekeying phase can be specified. Its size is 16 bytes.

Access is write-only.

11.3.6 LBP_IB_RANDP = 0x00000005

This object represents a random test array to be ciphered by the EAP peer, thus only available on Device LBP module. Size is 16 bytes.

Access is write-only.

11.3.7 LBP_IB_MSG_TIMEOUT = 0x00000006

This IB is only available in the Coordinator LBP and contains a time-out, in seconds, to wait for the next bootstrap packet for a joining device. In case time-out expires, the slot for such joining device is cleared and a new joining device will be assigned to it. Parameter size is 2 bytes, and default value is 40.

Access is write-only.

12. Coordinator Module

This module is only present in the G3 coordinator applications. This module is beyond the G3 specification and it is provided as an example for illustrative purposes.

Note: The provided module is a basic example, not intended to be a complete module in a real coordinator. This module, as stated, is out of the G3 specification and it is provided for testing purposes.

This section describes the API and implementation of the coordinator functionality provided as example.

12.1 Functional Description

The Coordinator module implements the following functionalities:

- Accepting/Rejecting Joining Devices
- Manage the Short Addressing of Joining Devices
- Maintain a list of Joined Devices in the Network
- Trigger the Kick of a Device out of the Network
- Trigger a Rekeying procedure to update the GMK used in the Network
- Interact with LBP module to send LBP frames during Join/Rekey phases

12.2 API Listing

12.2.1 Coordinator Module Initialization and Maintenance functions

The module initialization function sets up all the variables and structures needed for this functionality. It must be called at system start.

```
void coord_init (bool aribBand)
```

Parameters:

aribBand Specifies whether ARIB band is used in PLC. This has an impact when interacting with LBP module.

A maintenance function must be called periodically:

```
void coord_process (void)
```

This function is in charge of calling the LBP function that maintains its State Machine, to allow attending to Joining nodes in the Network.

12.2.2 Rekeying

In order to launch a rekeying procedure, the following function must be called:

```
void coord_launch_rekeying (void)
```

Before launching the rekeying, a new GMK has to be set in LBP module by means of LBP_IB_REKEY_GMK = 0x00000006.

Once the rekeying is triggered, all the process will be handled by both the Coordinator and LBP modules.

12.2.3 Joined Devices Table Access

The table of devices that have joined the network can be accessed, as well as its size.

The number of devices that have successfully joined the network is returned by the following function:

```
uint16_t coord_get_llds_counter (void)
```

And the table entries can be accessed through the IB [12.2.5.1. COORD_IB_DEVICE_LIST = 0x00000000](#) by means of [12.2.5. IB Access Functions](#).

There is an additional function related to the Device List, which obtains the Short Address of a Device, knowing its Extended Address:

```
bool coord_get_short_addr_by_ext (uint8_t *puc_extended_address, uint16_t *pus_short_address)
```

Parameters:

<i>uint8_t *puc_extended_address</i>	Pointer to Extended Address of known Device
<i>uint16_t *pus_short_address</i>	Pointer to which corresponding Short Address will be copied to

The function returns true in case the requested entry exists, and false otherwise.

12.2.4 Kick

As described in the G3 specification [\[ITU G.9903\]](#), the kick procedure can be initiated by the device or by the coordinator.

For the coordinator to start a kick procedure, the following function must be used:

```
void coord_kick_device (uint16_t us_short_address)
```

Parameters:

<i>uint16_t us_short_address</i>	Short address of the device to be kicked
----------------------------------	--

12.2.5 IB Access Functions

The Coordinator module offers a set of IBs which can be read and/or written using the following functions:

```
void coord_get_param (uint32_t ul_attribute_id, uint16_t us_attribute_idx, struct t_coord_get_param_confirm *p_get_confirm)
```

Parameters:

<i>uint32_t ul_attribute_id</i>	ID of attribute to be read
<i>uint16_t us_attribute_idx</i>	Index to read (in case it is a list)
<i>t_coord_get_param_confirm *p_get_confirm</i>	Pointer to confirm structure where result will be returned

```
void coord_set_param (uint32_t ul_attribute_id, uint16_t us_attribute_idx, uint8_t uc_attribute_len, const uint8_t *puc_attribute_value, struct t_coord_set_param_confirm *p_set_confirm)
```

Parameters:

<i>uint32_t ul_attribute_id</i>	ID of attribute to be written
<i>uint16_t us_attribute_idx</i>	Index to write (in case it is a list)
<i>uint8_t uc_attribute_len</i>	Length of attribute to be written
<i>const uint8_t *puc_attribute_value</i>	Value to be written
<i>struct t_coord_set_param_confirm *p_set_confirm</i>	Pointer to confirm structure where result will be returned

12.2.5.1 COORD_IB_DEVICE_LIST = 0x00000000

This Coordinator IB is used to read or write an element in the Device List.

Attribute size is 10 bytes:

- 2 bytes - Short Address of Device
- 8 bytes - Extended Address of Device (EUI64)

12.2.5.2 COORD_IB_INITIAL_SHORT_ADDRESS = 0x00000001

This Coordinator IB is used to read or write the first Short Address that will be assigned to a Joining Device. Following Devices will be given sequentially incremented addresses from this initial address.

Attribute size is 2 bytes containing the initial Address. Note that Address 0x0000 is not allowed as it is reserved for Coordinator.

12.2.5.3 COORD_IB_ADD_DEVICE_TO_BLACKLIST = 0x00000002

This Write Only Coordinator IB is used to add a Device to the Blacklist. When a Device is added to the Blacklist, it will not be allowed to join the Network.

Attribute size is 8 bytes, containing the Device Extended Address (EUI64).

12.2.5.4 COORD_IB_SHORT_ADDRESS_FROM_EXTENDED = 0x00000003

This Coordinator IB is a flag to force the Coordinator to set the Short Address of a Joining Device equal to last 2 bytes of its EUI64.

It is implemented just for illustration purposes, on how to change the behaviour of Short Address assignment in a Coordinator.

Attribute size is 1 byte (boolean).

12.2.6 Callback Integration with Upper and Lower Layers

As explained in the [10. LBP Module](#), it provides a set of callback functions that are captured by the Coordinator Module, they are configured by means of [11.1.3. LBP Set Notifications Function](#).

This callbacks capture the following events:

- Join Request indication. Upon reception, Coordinator module either assigns a Short Address to Device, or rejects it Joining.
- Join Complete indication. Invoked by LBP when a Joining procedure has finished. The provided Coordinator example notifies upper layer of the new Joined Device.
- Leave indication. Invoked by LBP when a Device leaves the Network. The provided Coordinator example notifies upper layer of the Leaving Device.

As stated above, the Coordinator module notifies the upper layer of Joining and Leaving events. To do so, upper layer must set callbacks in order to be notified, by means of the functions:

```
void coord_leave_ind_set_cb(pf_app_leave_ind_cb_t pf_handler)
```

Parameters:

pf_app_leave_ind_cb_t pf_handler Pointer to function which will be called when a device leaves the network

```
void coord_join_ind_set_cb(pf_app_join_ind_cb_t pf_handler)
```

Parameters:

pf_app_join_ind_cb_t pf_handler Pointer to function which will be called when a device joins the network

12.3 Brief of Provided Coordinator Module Example

Microchip provides an example implementation of the Coordinator module to be used in the G3 PAN Coordinator in order to accept devices willing to join the network, and therefore make them accessible to upper layers for data exchange.

12.3.1 Initialization Function

When upper layer calls the Coordinator initialization function, the following steps are performed:

- LBP module for Coordinator is initialized
- LBP notifications are set to be captured by Coordinator module
- Device List and Blacklist are initialized to empty lists
- Initial address to give to devices joining the network is set to 1

All these values are available as either constants or initialized variables in the same Coordinator module, and can be changed by the user to fit specific needs.

12.3.2 Registered Devices List

A registered devices list is managed by the Coordinator module. This list has a number of elements defined by the constant `MAX_LBDS`. Each element represents a device that has joined the network, and the information contained in the list entry is the EUI64 of the joined device. The short address of the device is given by the result of the following operation: $(index\ in\ list) + u16InitialShortAddr$. This initial short address is initialized to 1 by default, but can be changed to fit the user needs by either changing the default value or by means of setting the [12.2.5.2. COORD_IB_INITIAL_SHORT_ADDRESS = 0x00000001](#) parameter.

In the provided implementation, the Coordinator module assigns short addresses sequentially, starting with `u16InitialShortAddr` and adding 1 each time a device wants to join the network. If a device bootstraps more than once, a new short address is given, even if it was already present in the network before and the old entry is removed from the list. In case the end of the list is reached, the algorithm looks for removed entries to reuse them. If the list is full, no additional devices are accepted in the network.

12.3.3 Coordinator Blacklist

A blacklist of devices is implemented in the module. When a device is present in the blacklist, it is not accepted when it tries to join the network. Entries can be added to the blacklist from the upper layers using [12.2.5.3. COORD_IB_ADD_DEVICE_TO_BLACKLIST = 0x00000002](#).

12.3.4 Rekeying

The rekeying of the network is triggered by calling the *coord_launch_rekeying* function.

Note: As stated in the API description, before triggering the rekeying process, the new GMK must be set in LBP module by means of `LBP_IB_REKEY_GMK = 0x00000006`.

After triggering, the process is guided by the Coordinator module, making use of LBP module, which is the one in charge of sending and receiving all the required frames.

Refer to the [11.1. LBP Coordinator](#) section for details of the internal LBP processes.

When this process ends, all the network will be using the distributed key, and the module is ready for a new rekeying procedure by changing the value in `LBP_IB_REKEY_GMK = 0x00000006` attribute and calling *coord_launch_rekeying* primitive again.

13. Serialization with Embedded USI

The Embedded USI is a wrapper that provides the interface between the G3 FW stack and the serial communications channel.

For serial transmissions from the G3 stack, the Embedded USI provides a function that packs and sends each message via the serial link to the external application. For serial receptions from the serial link, the Embedded USI provides a function that unpacks the received message and passes it to the G3 FW stack.

The equivalent wrapper in the external application is the provided USI Host, which is also in charge of coding and decoding the messages. If users want to develop their own USI Host application, they will have to take into account the following operation of the Embedded USI to make it compatible:

- USI frame format
- USI G3 protocols
- USI Configuration

For more information about available services and the provided USI Host, see Application Note *PLC Universal Serial Interface* and the *USI Host User Guide*.

13.1 USI Frame Format

The USI frame format is based on the HDLC specification used along with DLMS, and is shown in the following figure.

Figure 13-1. USI Frame Format

7E (1 byte)	MSG LENGTH (10 bits)	PROTOCOL ID (6 bits)	MESSAGE DATA	CRC (variable)	7E (1 byte)
-----------------------	--------------------------------	--------------------------------	---------------------	--------------------------	-----------------------

The frame starts and ends with 0x7E. The following is the description of each field:

- **MSG LENGTH:** command length in bytes (protocol command byte plus message data bytes)
- **PROTOCOL ID:** protocol in the frame (see USI Protocols and Associated CRC Size table below)
- **MESSAGE DATA:** variable field with the data of the exchanged message
- **CRC:** error correction code for the message. The CRC field can have a different length depending on the protocol (see USI Protocols and Associated CRC Size table below)

Table 13-1. USI Protocols and Associated CRC Size

Protocol	Protocol ID	CRC size (bits)
PROTOCOL_PHY_ATPL2X0 ¹	0x22	16
PROTOCOL_SNIF_G3	0x23	16
PROTOCOL_MAC_G3	0x24	16
PROTOCOL_ADP_G3	0x25	16
PROTOCOL_COORD_G3	0x26	16
PROTOCOL_USER_DEFINED ²	0xFE	Defined by the user

Notes:

1. The PROTOCOL_PHY_ATPL2X0 is used by the PLC PHY Tester PC tool that Microchip provides with the evaluation kit in order to serialize the API of the PHY layer. Despite this denomination, this protocol is also used in the PL360 platform for the same purpose.
2. Defined by the user for their own proprietary protocol, if necessary.

13.2 USI G3 Protocols

The USI is able to serialize the following G3 interfaces and services:

- G3 PHY layer. Used between Phy Tester Tool Embedded FW and PC Application.
- G3 Sniffer. Used between Phy Sniffer Tool Embedded FW and PC Application.
- G3 MAC layer. Used between ADP_MAC_Serialized Embedded FW and Example Scripts.
- G3 ADP layer. Used between ADP_MAC_Serialized Embedded FW and Example Scripts.
- G3 COORD layer. Used between ADP_MAC_Serialized Embedded FW and Example Scripts.
- User Application. Through the available User Defined USI Protocol.

13.3 Embedded USI Configuration

The Embedded USI must be configured according to the user requirements. This configuration consists of indicating the protocols to be serialized and which port will be used by each protocol.

13.3.1 USI Ports Definition and Configuration

Users can define the ports to be used and their configurations in the *conf_usi.h* file.

```
/* Port Communications configuration */
#define NUM_PORTS      2
#define PORT_0 CONF_PORT(UART_TYPE, 0, 115200, TX_UART_BUF0_SIZE, RX_UART_BUF0_SIZE)
#define PORT_1 CONF_PORT(USART_TYPE, 1, 230400, TX_USART_BUF1_SIZE, RX_USART_BUF1_SIZE)
```

NUM_PORTS defines the number of ports to be used. After that, every PORT_x must be configured following a sequential order (PORT_0, PORT_1, etc.). The input parameters of the port configuration are shown in the following table.

Table 13-2. USI Port Configuration Parameters

Parameter	Description	Valid Values
Type	Type of link	UART_TYPE for UART USART_TYPE for USART USB_TYPE for USB
Channel	Instance	0: UART0/USART0 1: UART1/USART1 2: UART2/USART2
Speed	Baudrate	9600, 19200, 38400, 57600, 115200, 230400, 256000, 921600
TX_size	Size of transmission buffer	Size of buffer must be (at least) twice the size of the bigger message payload linked to this port.
RX_size	Size of reception buffer	Size of buffer must be (at least) twice the size of the bigger message payload linked to this port.



Important: Both Embedded USI and the external application must use the same baud rate and protocols. Messages from protocols not serialized in both sides of the serial communications channel are discarded.



Important: In the SAM4C microcontroller family, the UART1 has no access to the DMA so it cannot be chosen for the USI operation.

13.3.2 Linking of G3 Sniffer

To link the embedded G3 sniffer to a USI port defined in the Embedded USI, file *conf_usi.h* must be edited.

An example of enabling embedded sniffer is provided below.

```

/* Enable Embedded Sniffer */
#define ENABLE_SNIFFER

#define USI_PORT_0      0

/* Port Communications configuration */
#define NUM_PORTS      1
#define PORT_0 CONF_PORT(UART_TYPE, USI_PORT_0, 230400, TX_UART_BUF0_SIZE, RX_UART_BUF0_SIZE)

/* Select PORT to sniffer phy iface */
#define PHY_SNIFFER_SERIAL_PORT      USI_PORT_0

```

13.3.3 Linking of G3 API

To link any (or all) of the G3 APIs (MAC, ADP, COORD) to a USI port defined in the Embedded USI, file *conf_usi.h* must be edited.

An example of enabling all APIs Serialization, as done in the Adp_Mac_Serialized example application, is provided below.

```

#define USI_PORT_0      0

/* Port Communications configuration */
#define NUM_PORTS      1
#define PORT_0 CONF_PORT(UART_TYPE, USI_PORT_0, 230400, TX_UART_BUF0_SIZE, RX_UART_BUF0_SIZE)

/* Select PORT to serialize layers */
#define ADP_SERIAL_PORT      USI_PORT_0
#define MAC_SERIAL_PORT      USI_PORT_0
#define COORD_SERIAL_PORT    USI_PORT_0

```

In the case that a user wants to add Sniffer capabilities apart from controlling the G3 stack through the same Serial Channel, the Sniffer Port has to be defined also (as seen on previous section) by adding the following line:

```

#define PHY_SNIFFER_SERIAL_PORT      USI_PORT_0

```

14. Supported Platforms

This chapter describes which hardware platforms are currently supported in the Microchip G3-PLC software package. A platform is usually comprised of three major components:

- An MCU
- A transceiver chip. This may be integrated into the MCU (i.e.:PL485)
- A specific board or even several boards that contain the MCU and the transceiver chip

14.1 Supported MCU Families

Currently, the supported MCU families are the SAM4C, SAME70, SAMG55, and PIC32CX platforms.

The dedicated code for each device of the family can be found in the corresponding sub-directories of the FW package.

14.2 Supported Devices

Currently, the supported devices are PL360, PL460 and PL485 for PLC, and AT86RF215 for RF.

14.3 Supported Boards

The currently supported boards and platforms are the following:

- PIC32CXMTG-EK + PL460-EK on Xplained Port
- PIC32CXMTSH-DB + PL460-EK on Xplained Port
- PIC32CXMTG-DB + PL460-EK on Xplained Port
- PIC32CXMTG-EK + PL460-EK on Xplained Port + ATREB215-EK on mikroBUS Port
- PIC32CXMTSH-DB + PL460-EK on Xplained Port + ATREB215-EK on mikroBUS Port
- PIC32CXMTG-DB + PL460-EK on Xplained Port + ATREB215-EK on mikroBUS Port
- PL360MB
- SAM4CMS-DB + PL460-EK on Xplained Port
- SAME70 Xplained + PL460-EK on Xplained Port
- SAME70 Xplained + PL460-EK on Xplained Port + ATREB215-EK on Xplained Port
- PL360BN
- SAMG55 Xplained + PL460-EK on Xplained Port
- SAMG55 Xplained + PL460-EK on Xplained Port + ATREB215-EK on Xplained Port
- PL360G55CF-EK
- PL360G55CB-EK
- PL360G55CF-EK + ATREB215-EK on Xplained Port
- PL360G55CB-EK + ATREB215-EK on Xplained Port
- PL485-EK
- PL485-EK + ATREB215-EK on Xplained Port

15. Toolchain

The following sections describe the required tools and toolchain for the development and build process and how the provided example applications can be built.

15.1 General Prerequisites

The following tools and toolchains are used for building the applications from this FW package:

- Microchip Studio (see www.microchip.com/en-us/development-tools-tools-and-software/microchip-studio-for-avr-and-sam-devices)
- IAR Systems® Embedded Workbench (see <http://www.iar.com/>)
- Keil® MDK-ARM Microcontroller Development Kit (see <http://www.keil.com/arm/mdk.asp>)

15.2 Building the Applications

15.2.1 Using Microchip Studio

Each application can be rebuilt with Microchip Studio directly, using version 7.

15.2.2 Using IAR Embedded Workbench®

Each application can be rebuilt with IAR Embedded Workbench directly, using version 9.20.

15.2.3 Using Keil®

Each application can be rebuilt with Keil µVision® directly, using version 5.

16. PICS

This chapter lists the conformance of the Microchip G3-PLC implementation with the requirements and optional features as defined by the G3-PLC specification document.

A Protocol Implementation Conformance Statement (PICS), as defined by ISO 9646 [2], is a declaration listing the capabilities and options supported by an implementation. The PICS is based on a list of options and values, defined in the G3 specification and in the test suites used by the certification process.

When an implementation is ready for certification, the implementer must provide the PICS corresponding to the implementation capability.

The PICS values are included in the device certificate, in order to indicate the conditions of the certification in terms of implemented options.

16.1 Band-plan Support

This PICS entry is used to describe the band-plan supported by the device under test.

If a physical device is able to support several band-plans, it will be tested once for each band-plan, setting the following PICS value accordingly.

Current implementation supports Cenelec A-Band, Cenelec B-Band, ARIB and FCC band-plans.

16.2 Feature Implementation Statement

Microchip G3 PLC stack implements the following optional features:

- Hybrid Profile.
- Coherent Modulation.
- Routing: LOADng implementation supporting composite metrics and capacity metrics.

16.3 Other Features

Microchip provides a basic PAN Coordinator module with limited features, such as creating the network and accepting all network join from the devices. This way, it is possible to build a network and exchange data.

This is just a basic implementation for demonstration purposes, and it should not be taken as a complete module to build a full-featured PAN Coordinator.

17. Abbreviations

ADP	Adaptation Sublayer
ADPD	Adaptation Sublayer Data Service
ADPIB	Adaptation Sublayer Information Base
ADPM	Adaptation Sublayer Management Service
API	Application Programming Interface
APP	Application
ASF	Advanced Software Framework
FCS	Frame Check Sequence
FW	Firmware
HAL	Hardware Abstraction Layer
MAC	Medium Access Control
MAC RT	MAC Real Time
MCPS	MAC Common Part Sub-layer
MCU	Microcontroller Unit
MIB	MAC Information Base
MIC	Message Integrity Code
MLME	MAC Sub-layer Management Entity
MSDU	MAC Service Data Unit
OSS	Operating System Support
PAL	PHY Abstraction Layer
PAN	Personal Area Network
PDU	Physical layer Data Unit
PIB	Protocol Information Base
PLME	Physical Layer Management Entity
POS	Personal Operating Space
SPI	Serial Peripheral Interface
TRX	Transceiver

18. References

1. Smart Energy Products: <http://www.microchip.com/design-centers/smart-energy-products/overview>
2. Power Line Communications: <http://www.microchip.com/design-centers/smart-energy-products/power-line-communications/overview>
3. Microchip Design Support: <http://www.microchip.com/support/hottopics.aspx>
4. ITU G.9903: <https://www.itu.int/rec/T-REC-G.9903/en>
5. ITU G.9901: <http://www.itu.int/rec/T-REC-G.9901/en>
6. G3-PLC Alliance: <http://www.g3-plc.com/>
7. Advanced Software Framework: [http://www.microchip.com/development-tools/atmel-studio-7/advanced-software-framework-\(asf\)](http://www.microchip.com/development-tools/atmel-studio-7/advanced-software-framework-(asf))
8. Microchip Studio: www.microchip.com/en-us/development-tools-tools-and-software/microchip-studio-for-avr-and-sam-devices

19. Revision History

Note: This document starts from G3-PLC Firmware Stack rev. C (04/2020).

19.1 Revision B - 09/2023

All Sections	Added references to PL460 where missing
All Sections	Removed references to ATPL250
All Sections	Removed obsolete information
8. API of DATA LINK Layer	Removed obsolete Get Request/Confirm mechanisms from MAC layers (only Sync primitives used) Merged MacWrapper and HyAL content, as now they are a unique layer Added a new MAC IBs Reviewed PHY parameters Added new ADP IBs
8.8. ADP SAP	Removed obsolete callbacks now handled by LBP module
8.9. ADP Data Primitives	Added AdpNoIPDataRequest primitive
10. LBP Module	New Chapter is added
12. Coordinator Module	Reviewed Coordinator functionalities and API
Gateway Configuration	Removed this Chapter
14. Supported Platforms	Added a new supported platforms and removed obsolete.

19.2 Rev A - 11/2021

1. General Architecture	Added/Modified sections to reflect Hybrid Profile.
2.1. G3-PLC Workspace Structure	Added/Modified sections to reflect Hybrid Profile.
3.3. Configuration Files	Added and described new files. Review conf_clock.h and conf_pplc_if.h files description.
5. Example Applications	Added Meter/Modem Apps description. Added RF PHY Examples.
5.4. Clock Configuration Example for Low Requirements Module	Added this section.
6.6. Voltage and Thermal Monitor for PLC Driver	Added this section.
8. API of DATA LINK Layer	Added MAC SAP and MAC Primitives chapters for RF MAC. Added RF MAC MIB Objects definition. Introduced Hybrid Abstraction Layer. Updated ADPM-DISCOVERY.indication, ADPM-NETWORK-JOIN.request and ADPM-NETWORK-STATUS.indication to reflect changes introduced on Hybrid Profile. Added New ADP IB Objects definition.
8.7. MIB Objects Specification and Access	Added MAC_WRP_PIB_MANUF_ACK_TX_DELAY_RF, MAC_WRP_PIB_MANUF_ACK_RX_WAIT_TIME_RF, MAC_WRP_PIB_MANUF_ACK_CONFIRM_WAIT_TIME_RF, MAC_WRP_PIB_MANUF_DATA_CONFIRM_WAIT_TIME_RF.
9. Data Link Processes	Review Diagrams to add RF channel.
14. Supported Platforms	Added new supported platforms.
15. Toolchain	Introduced Microchip Studio.
Gateway Configuration	Imported Topic.

Microchip Information

The Microchip Website

Microchip provides online support via our website at www.microchip.com/. This website is used to make files and information easily available to customers. Some of the content available includes:

- **Product Support** – Data sheets and errata, application notes and sample programs, design resources, user's guides and hardware support documents, latest software releases and archived software
- **General Technical Support** – Frequently Asked Questions (FAQs), technical support requests, online discussion groups, Microchip design partner program member listing
- **Business of Microchip** – Product selector and ordering guides, latest Microchip press releases, listing of seminars and events, listings of Microchip sales offices, distributors and factory representatives

Product Change Notification Service

Microchip's product change notification service helps keep customers current on Microchip products. Subscribers will receive email notification whenever there are changes, updates, revisions or errata related to a specified product family or development tool of interest.

To register, go to www.microchip.com/pcn and follow the registration instructions.

Customer Support

Users of Microchip products can receive assistance through several channels:

- Distributor or Representative
- Local Sales Office
- Embedded Solutions Engineer (ESE)
- Technical Support

Customers should contact their distributor, representative or ESE for support. Local sales offices are also available to help customers. A listing of sales offices and locations is included in this document.

Technical support is available through the website at: www.microchip.com/support

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip product is strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is "unbreakable". Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure

that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP "AS IS". MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Trademarks

The Microchip name and logo, the Microchip logo, Adaptec, AVR, AVR logo, AVR Freaks, BesTime, BitCloud, CryptoMemory, CryptoRF, dsPIC, flexPWR, HELDO, IGLOO, JukeBlox, KeeLoq, Kleer, LANCheck, LinkMD, maXStylus, maXTouch, MediaLB, megaAVR, Microsemi, Microsemi logo, MOST, MOST logo, MPLAB, OptoLyzer, PIC, picoPower, PICSTART, PIC32 logo, PolarFire, Prochip Designer, QTouch, SAM-BA, SenGenuity, SpyNIC, SST, SST Logo, SuperFlash, Symmetricom, SyncServer, Tachyon, TimeSource, tinyAVR, UNI/O, Vectron, and XMEGA are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

AgileSwitch, APT, ClockWorks, The Embedded Control Solutions Company, EtherSynch, Flashtec, Hyper Speed Control, HyperLight Load, Libero, motorBench, mTouch, Powermite 3, Precision Edge, ProASIC, ProASIC Plus, ProASIC Plus logo, Quiet-Wire, SmartFusion, SyncWorld, Temux, TimeCesium, TimeHub, TimePictra, TimeProvider, TrueTime, and ZL are registered trademarks of Microchip Technology Incorporated in the U.S.A.

Adjacent Key Suppression, AKS, Analog-for-the-Digital Age, Any Capacitor, AnyIn, AnyOut, Augmented Switching, BlueSky, BodyCom, Clockstudio, CodeGuard, CryptoAuthentication, CryptoAutomotive, CryptoCompanion, CryptoController, dsPICDEM, dsPICDEM.net, Dynamic Average Matching, DAM, ECAN, Espresso T1S, EtherGREEN, GridTime, IdealBridge, In-Circuit Serial Programming, ICSP, INICnet, Intelligent Paralleling, IntelliMOS, Inter-Chip Connectivity, JitterBlocker, Knob-on-Display, KoD, maxCrypto, maxView, memBrain, Mindi, MiWi, MPASM, MPF, MPLAB Certified logo, MPLIB, MPLINK, MultiTRAK, NetDetach, Omniscient Code Generation, PICDEM, PICDEM.net, PICkit, PICtail, PowerSmart, PureSilicon, QMatrix, REAL ICE, Ripple Blocker, RTAX, RTG4, SAM-ICE, Serial Quad I/O, simpleMAP, SimpliPHY, SmartBuffer, SmartHLS, SMART-I.S., storClad, SQL, SuperSwitcher, SuperSwitcher II, Switchtec, SynchroPHY, Total Endurance, Trusted Time, TSHARC, USBCheck, VariSense, VectorBlox, VeriPHY, ViewSpan, WiperLock, XpressConnect, and ZENA are trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

SQTP is a service mark of Microchip Technology Incorporated in the U.S.A.

The Adaptec logo, Frequency on Demand, Silicon Storage Technology, and Symmcom are registered trademarks of Microchip Technology Inc. in other countries.

GestIC is a registered trademark of Microchip Technology Germany II GmbH & Co. KG, a subsidiary of Microchip Technology Inc., in other countries.

All other trademarks mentioned herein are property of their respective companies.

© 2021, Microchip Technology Incorporated and its subsidiaries. All Rights Reserved.

ISBN: 978-1-6683-3173-6

Quality Management System

For information regarding Microchip's Quality Management Systems, please visit www.microchip.com/quality.

Worldwide Sales and Service

AMERICAS	ASIA/PACIFIC	ASIA/PACIFIC	EUROPE
Corporate Office 2355 West Chandler Blvd. Chandler, AZ 85224-6199 Tel: 480-792-7200 Fax: 480-792-7277 Technical Support: www.microchip.com/support Web Address: www.microchip.com	Australia - Sydney Tel: 61-2-9868-6733 China - Beijing Tel: 86-10-8569-7000 China - Chengdu Tel: 86-28-8665-5511 China - Chongqing Tel: 86-23-8980-9588 China - Dongguan Tel: 86-769-8702-9880 China - Guangzhou Tel: 86-20-8755-8029 China - Hangzhou Tel: 86-571-8792-8115 China - Hong Kong SAR Tel: 852-2943-5100 China - Nanjing Tel: 86-25-8473-2460 China - Qingdao Tel: 86-532-8502-7355 China - Shanghai Tel: 86-21-3326-8000 China - Shenyang Tel: 86-24-2334-2829 China - Shenzhen Tel: 86-755-8864-2200 China - Suzhou Tel: 86-186-6233-1526 China - Wuhan Tel: 86-27-5980-5300 China - Xian Tel: 86-29-8833-7252 China - Xiamen Tel: 86-592-2388138 China - Zhuhai Tel: 86-756-3210040	India - Bangalore Tel: 91-80-3090-4444 India - New Delhi Tel: 91-11-4160-8631 India - Pune Tel: 91-20-4121-0141 Japan - Osaka Tel: 81-6-6152-7160 Japan - Tokyo Tel: 81-3-6880-3770 Korea - Daegu Tel: 82-53-744-4301 Korea - Seoul Tel: 82-2-554-7200 Malaysia - Kuala Lumpur Tel: 60-3-7651-7906 Malaysia - Penang Tel: 60-4-227-8870 Philippines - Manila Tel: 63-2-634-9065 Singapore Tel: 65-6334-8870 Taiwan - Hsin Chu Tel: 886-3-577-8366 Taiwan - Kaohsiung Tel: 886-7-213-7830 Taiwan - Taipei Tel: 886-2-2508-8600 Thailand - Bangkok Tel: 66-2-694-1351 Vietnam - Ho Chi Minh Tel: 84-28-5448-2100	Austria - Wels Tel: 43-7242-2244-39 Fax: 43-7242-2244-393 Denmark - Copenhagen Tel: 45-4485-5910 Fax: 45-4485-2829 Finland - Espoo Tel: 358-9-4520-820 France - Paris Tel: 33-1-69-53-63-20 Fax: 33-1-69-30-90-79 Germany - Garching Tel: 49-8931-9700 Germany - Haan Tel: 49-2129-3766400 Germany - Heilbronn Tel: 49-7131-72400 Germany - Karlsruhe Tel: 49-721-625370 Germany - Munich Tel: 49-89-627-144-0 Fax: 49-89-627-144-44 Germany - Rosenheim Tel: 49-8031-354-560 Israel - Ra'anana Tel: 972-9-744-7705 Italy - Milan Tel: 39-0331-742611 Fax: 39-0331-466781 Italy - Padova Tel: 39-049-7625286 Netherlands - Drunen Tel: 31-416-690399 Fax: 31-416-690340 Norway - Trondheim Tel: 47-72884388 Poland - Warsaw Tel: 48-22-3325737 Romania - Bucharest Tel: 40-21-407-87-50 Spain - Madrid Tel: 34-91-708-08-90 Fax: 34-91-708-08-91 Sweden - Gothenberg Tel: 46-31-704-60-40 Sweden - Stockholm Tel: 46-8-5090-4654 UK - Wokingham Tel: 44-118-921-5800 Fax: 44-118-921-5820